Universidad Nacional de Córdoba
Facultad de Ciencias Exactas, Físicas y Naturales
Ingeniería en Computación

Plataforma de Automatización de Red Multivendor

</AnsiAuth>

Alumnos:

- Martin, Tomas - DNI: 39326227
  mail: tomas.martin@unc.edu.ar
  teléfono: +54 9 3541 37-5980
- Endrek, Marco - DNI: 38798589
  mail: mendrek@unc.edu.ar
  teléfono: +54 9 3515 10-8479

Dirección y Codirección:

- Prof. Ing. Solinas, Miguel A.
- Ing. Bettucci, Marcos

AGRADECIMIENTOS

Quiero agradecer a la Universidad Nacional de Córdoba y a la Facultad de Ciencias Exactas, Físicas y Naturales por la educación pública y de calidad que me permitió llegar hasta aquí. A los docentes que me formaron y a mis directores, por su guía y disposición durante el desarrollo de este proyecto. También a la Prosecretaría de Informática, y en especial a mis compañeros de Redes, por abrirme las puertas y por convertir el trabajo diario en un espacio de aprendizaje permanente. Gran parte de lo que se refleja en este trabajo nació de esa experiencia.

A mis amigos de toda la vida y a los que me regaló esta etapa, por ser refugio en los días difíciles y alegría en los buenos, y por recordarme siempre que no estaba solo.

A mi familia, por su apoyo incondicional, su amor y la confianza que depositaron en mí desde el primer día. Sin ellos no lo hubiera logrado. Y a mi novio, por acompañarme y alentarme en esta última etapa. A todos, gracias por celebrar cada avance como propio. Este logro también es de ustedes.

Tomás.

Quiero expresar mi agradecimiento a la Universidad Nacional de Córdoba por la formación que me brindó, y a los profesores que formaron parte de este camino, en especial a Miguel Solinas, por su guía y dedicación durante el desarrollo de este proyecto integrador. Agradezco también al equipo de Redes de la PSI, por todo lo que me enseñaron y por hacer del trabajo un espacio en el que siempre fue un gusto estar y pertenecer.

A mi familia, por su apoyo constante y por haber sido un pilar fundamental a lo largo de toda mi carrera, y en especial a mis padres, por brindarme la posibilidad de dedicarme al estudio. Asimismo, a Agustina y a su familia, por su compañía y por motivarme en todo momento a alcanzar mis metas.

A mis amigos, que estuvieron presentes, celebrando conmigo cada logro y sosteniéndome en los momentos más difíciles. Y, de manera muy especial, a Fernando, la estrella que me guía desde el comienzo de este camino. Este logro también es tuyo.

Marco.

Índice

1. Introducción.....7

1.1. Motivación..... 7

1.2. Objetivos..... 8

1.3. Estructura del Informe.....8

2. Marco Teórico.....10

2.1. El switch en la red de campus..... 10

2.2. La configuración de un switch..... 11

2.3. La administración de los equipos..... 13

2.4. Automatización de redes..... 14

2.5. Programabilidad de redes y SDN..... 16

2.6. API REST en la automatización de redes..... 18

3. Planificación del Proyecto.....19

3.1. Descubrimiento de Requerimientos y Diseño del Sistema..... 19

3.2. Iteraciones del Proyecto..... 20

4. Ingeniería de Requerimientos.....22

4.1. Definición de la Problemática..... 22

4.2. Modelos de Solución..... 23

4.3. Elección del Modelo.....26

4.4. Elicitación de Requerimientos.....27

4.5. Limitaciones y Supuestos..... 28

4.6. Especificación de Requerimientos.....29

5. Arquitectura del Sistema.....41

5.1. Contexto del Sistema.....41

5.2. Modelo de Diseño.....41

5.3. El Núcleo: Recurso Gestionable y Ciclo de un Cambio.....42

5.4. Diseño de la Arquitectura..... 44

5.5. Diseño Modular..... 46

5.6. Diseño de Interfaces..... 64

5.7. Stack Tecnológico.....67

5.8. Entorno de Desarrollo....69
6. Primera Iteración: Conexión con los Equipos y Funcionalidades Base....70
6.1. Planificación y Diseño....70
6.2. Implementación....71
6.3. Pruebas y Validaciones....75
6.4. Resultados y Ajustes....76
7. Segunda Iteración: Extensión de los Recursos y Arquitectura Definitiva....80
7.1. Planificación y Diseño....80
7.2. Implementación....81
7.3. Pruebas y Validaciones....88
7.4. Resultados y Ajustes....89
8. Tercera Iteración: Arquitectura Multisitio, Lote y Cliente Web....93
8.1. Planificación y Diseño....93
8.2. Implementación....94
8.3. Pruebas y Validaciones....102
8.4. Resultados y Ajustes....103
9. Conclusiones....107
10. Trabajos Futuros....109
Bibliografía....111

Acrónimos

ACL	Access Control List (lista de control de acceso)
AP	Access Point (punto de acceso inalámbrico)
API	Application Programming Interface (interfaz de programación de aplicaciones)
CLI	Command-Line Interface (interfaz de línea de comandos)
DHCP	Dynamic Host Configuration Protocol (protocolo de configuración dinámica de hosts)
DNS	Domain Name System (sistema de nombres de dominio)
IOS	Internetwork Operating System (sistema operativo de los equipos Cisco)
JSON	JavaScript Object Notation (formato de intercambio de datos)
JWT	JSON Web Token (*token* web JSON)
NETCONF	Network Configuration Protocol (protocolo de configuración de red)
NTP	Network Time Protocol (protocolo de tiempo de red)
PoE	Power over Ethernet (alimentación a través de Ethernet)
RBAC	Role-Based Access Control (control de acceso basado en roles)
REST	Representational State Transfer (transferencia de estado representacional)
SDN	Software-Defined Networking (redes definidas por software)
SNMP	Simple Network Management Protocol (protocolo simple de administración de red)
SSH	Secure Shell (intérprete de comandos seguro)
SVI	Switched Virtual Interface (interfaz virtual conmutada)
TLS	Transport Layer Security (seguridad de la capa de transporte)
VLAN	Virtual Local Area Network (red de área local virtual)
VRP	Versatile Routing Platform (sistema operativo de los equipos Huawei)
YAML	YAML Ain't Markup Language (formato de serialización de datos legible por personas)

Índice de Figuras

Figura 2.1: Modelo jerárquico de una red de campus.....10

Figura 2.2: Vista simplificada de una arquitectura SDN, con las interfaces ‘northbound’ y ‘southbound’.....17

Figura 3.1: Modelo de proceso iterativo incremental adoptado en el proyecto.....19

Figura 4.1: Diagrama de casos de uso del sistema.....30

Figura 5.1: Contexto del sistema (nivel 1 del modelo C4).....41

Figura 5.2: Ciclo de un cambio de configuración.....43

Figura 5.3: Contenedores del sistema (nivel 2 del modelo C4).....45

Figura 5.4: Módulos del sistema y sus dependencias (nivel 3 del modelo C4).....46

Figura 5.5: Clases del módulo de organización.....48

Figura 5.6: Clases del módulo de autorización.....49

Figura 5.7: Estados de un trabajo.....50

Figura 5.8: Clases del módulo de orquestación.....53

Figura 5.9: Clases del modelo de dominio.....55

Figura 5.10: Acceso de un recurso al switch a través del equipo.....56

Figura 5.11: Controladores por fabricante.....57

Figura 5.12: Vocabulario de comandos, transporte y parsers.....58

Figura 5.13: Clases del módulo de auditoría y eventos.....60

Figura 5.14: Repositorios del módulo de persistencia.....62

Figura 5.15: Modelo de datos.....63

Figura 5.16: Mapa de navegación del cliente web.....65

Figura 6.1: Secuencia del alta de una VLAN en la primera iteración.....73

Figura 6.2: Interfaz web al cierre de la primera iteración.....75

Figura 6.3: Entradas de auditoría del alta de la VLAN 100.....77

Figura 7.1: Secuencia de edición de puerto en la segunda iteración.....85

Figura 7.2: Interfaz web al cierre de la segunda iteración.....88

Figura 8.1: Secuencia de una edición conjunta de puertos en la tercera iteración.....97

Figura 8.2: Cliente web al cierre de la tercera iteración.....99

Índice de Cuadros

Cuadro 2.1: Creación de una VLAN y asignación a un puerto en modo access en Cisco y Huawei.... 13
Cuadro 4.1: Comparación de modelos de solución.....26
Cuadro 4.2: Casos de uso principales y requerimientos que los implementan..... 36
Cuadro 4.3: Matriz de Riesgos.....39
Cuadro 4.4: Análisis de riesgo y priorización de los requerimientos funcionales..... 40
Cuadro 4.5: Análisis de riesgo y priorización de los requerimientos no funcionales..... 40
Cuadro 5.1: Recursos gestionables, categoría del switch y requerimiento asociado..... 43
Cuadro 5.2: Clasificación de los errores de los equipos..... 54
Cuadro 5.3: Requerimientos atendidos por cada módulo..... 64
Cuadro 5.4: Respuestas de la API REST..... 66
Cuadro 5.5: Recursos de la API REST.....67
Cuadro 5.6: Tecnologías empleadas por módulo..... 69
Cuadro 6.1: Requerimientos del sistema atendidos en la primera iteración..... 70
Cuadro 6.2: Pruebas automatizadas al cierre de la primera iteración..... 76
Cuadro 6.3: Tiempos de ejecución al cierre de la primera iteración.....78
Cuadro 7.1: Cambios estructurales de la segunda iteración y limitación que resuelve..... 81
Cuadro 7.2: Pruebas automatizadas incorporadas en la segunda iteración.....89
Cuadro 7.3: Requerimientos del sistema atendidos en la segunda iteración.....90
Cuadro 7.4: Sesiones SSH por operación en la primera y la segunda iteración..... 91
Cuadro 7.5: Tiempos de ejecución en la primera y la segunda iteración.....91
Cuadro 8.1: Cambios de la tercera iteración y limitación que resuelve.....93
Cuadro 8.2: Fases de la migración al modelo multisitio.....95
Cuadro 8.3: Pruebas automatizadas incorporadas en la tercera iteración..... 102
Cuadro 8.4: Requerimientos del sistema atendidos en la tercera iteración.....104
Cuadro 8.5: Fallas detectadas durante la validación en producción..... 104
Cuadro 8.6: Sesiones por operación en la segunda y la tercera iteración..... 105

1. Introducción

1.1. Motivación

La infraestructura de red de una organización da soporte a los sistemas y servicios que sus usuarios utilizan a diario. En este esquema, los *switches* cumplen un rol fundamental al proporcionar conectividad en la red de área local (LAN) y permitir la segmentación y gestión eficiente del tráfico [1]. Para asegurar un funcionamiento continuo y seguro, los dispositivos requieren intervenciones periódicas de configuración, administración y mantenimiento a lo largo de todo su ciclo de vida.

Tradicionalmente, la gestión se efectúa de manera individualizada, accediendo a cada *switch* mediante interfaces de línea de comandos (CLI), sobre el protocolo SSH, o mediante interfaces web propietarias. Este modelo operativo implica aplicar cambios por equipo, lo que incrementa la complejidad a medida que la infraestructura escala. Asimismo, exige conocer cada plataforma y coordinar los accesos, porque cada equipo admite pocas conexiones simultáneas, y los cambios quedan sin un registro común.

Esta problemática se agrava en entornos heterogéneos conformados por equipos de diversos fabricantes. Aunque los *switches* implementan funcionalidades equivalentes, cada proveedor utiliza sintaxis, nomenclaturas y estructuras de comandos distintas. En consecuencia, una misma acción lógica requiere procedimientos diferentes según el dispositivo de destino, lo que dificulta la estandarización y aumenta la dependencia del conocimiento especializado del personal.

De esta gestión manual e individualizada se derivan problemas de consistencia, porque las configuraciones de equipos que cumplen funciones equivalentes divergen con el tiempo (*configuration drift*) [16]; de escalabilidad, porque el esfuerzo crece con cada equipo que debe modificarse; de trazabilidad, porque resulta difícil reconstruir qué cambio se aplicó, en qué momento y quién lo ejecutó [30]; y de errores humanos, propios de la intervención manual directa. Estos problemas no son exclusivos de las redes, sino que forman parte de la gestión de configuración y del control de cambios en cualquier sistema informático, donde la centralización de las operaciones, el registro de auditoría y los controles de acceso permiten reducir el riesgo operativo y reconstruir las acciones realizadas ante un incidente [30].

En consecuencia, la falta de una herramienta que abstraiga las diferencias entre fabricantes y automatice las tareas recurrentes reduce la eficiencia operativa y aumenta el

riesgo de errores. Por ello, se propone una plataforma que centralice las operaciones, reduzca la intervención manual y estandarice los procedimientos sobre todos los equipos.

1.2. Objetivos

Desarrollar una plataforma de administración centralizada para la gestión de *switches multivendor* desde una interfaz única, que abstraiga la complejidad sintáctica de cada fabricante y reduzca la intervención manual, garantizando que las operaciones sobre la red sean idempotentes, trazables y reversibles.

1.2.1. Objetivos Específicos

- Analizar las estrategias de automatización de red y relevar las herramientas de gestión existentes, para fundamentar la selección del motor de automatización y del *stack* tecnológico.
- Diseñar una arquitectura modular que permita incorporar nuevos recursos sin modificar la lógica central de los cambios.
- Abstraer las diferencias de sintaxis entre fabricantes, de modo que incorporar uno nuevo no requiera modificar el resto del sistema.
- Implementar las operaciones sobre los recursos de red principales: VLAN, puertos físicos, interfaces virtuales (SVI) y configuración global del dispositivo.
- Incorporar una ejecución asíncrona de los cambios, con reintentos ante fallas transitorias y *rollback* ante fallas permanentes.
- Implementar un control de acceso basado en roles (RBAC) y un registro de auditoría que permita reconstruir qué cambio se aplicó, sobre qué equipo, en qué momento y por qué usuario [31].
- Organizar los equipos en sitios y grupos, con permisos por alcance y operaciones sobre conjuntos de equipos.
- Construir una interfaz web que permita operar el sistema y consultar el estado de la red sin recurrir a la CLI de cada equipo.
- Validar la plataforma de manera incremental en un laboratorio virtualizado, un entorno físico dedicado y equipos en producción, evaluando su desempeño e incorporando mitigaciones ante las fallas detectadas.

1.3. Estructura del Informe

En el **Marco Teórico** se describen los *switches* y su administración, las limitaciones de la gestión manual y las alternativas existentes para automatizarla.

A continuación, la **Planificación del Proyecto** describe la organización del trabajo, el diseño general de la solución y las etapas que recorre cada iteración. Siguiendo con la **Ingeniería de Requerimientos**, que define en detalle la problemática abordada, evalúa los modelos de solución considerados y especifica los requerimientos funcionales y no funcionales, junto con las limitaciones, los supuestos y el criterio de priorización adoptado.

En la **Arquitectura del Sistema** presenta el diseño de la plataforma siguiendo los niveles del modelo C4, desde su contexto y sus contenedores hasta el detalle de cada módulo. Antes de los módulos se describe el núcleo del diseño, formado por el recurso gestionable y el ciclo de un cambio. Los módulos abarcan, entre otros aspectos, la autorización, la ejecución asíncrona de trabajos, la coordinación de sesiones con los equipos y el modelo de datos. Concluye con el diseño de la interfaz web, y con la selección y justificación del *stack* tecnológico frente a las alternativas evaluadas.

El desarrollo se presenta luego en tres iteraciones, organizadas con la misma estructura: planificación y diseño, implementación, pruebas y validaciones, finalizando con resultados y ajustes. La **Primera Iteración** establece la conexión con los equipos, las funcionalidades base de autenticación, inventario, gestión de VLAN, trabajos y auditoría, y una primera interfaz web. La **Segunda Iteración** incorpora los puertos, las interfaces virtuales y la configuración global, y lleva la plataforma a la arquitectura de módulos del Capítulo 5. La **Tercera Iteración** introduce el modelo multisitio, las operaciones en lote y sobre grupos de equipos, y una interfaz web organizada por sitios y grupos, y valida la plataforma sobre los equipos en producción.

Cada iteración se validó de forma incremental, primero en un laboratorio virtualizado, luego sobre equipos físicos y, en la última, sobre los equipos en producción. El informe finaliza con las **Conclusiones** y **Trabajos Futuros**, donde se evalúa el cumplimiento de los objetivos, se repasan los desafíos técnicos resueltos y las lecciones aprendidas, y se proponen líneas de trabajo futuro.

2. Marco Teórico

En este capítulo se presentan los conceptos en los que se basa el proyecto. Primero se describe el componente central de la red: qué es un *switch*, qué parámetros se configuran en él y cómo se administra. Luego se exponen las limitaciones de la gestión manual que motivan la automatización. A continuación se recorren las herramientas para implementarla, desde las bibliotecas de programación y Ansible hasta la programabilidad de redes con SDN. El capítulo cierra con el rol de las API REST en este contexto.

2.1. El *switch* en la red de campus

Una red de campus, como las que se encuentran en instituciones académicas, organismos públicos y empresas, suele diseñarse según un modelo jerárquico de tres capas: un núcleo (*core*) que enlaza los distintos edificios, una capa de distribución que concentra los enlaces de los *switches* de acceso y los conecta con los servicios de la red, y una capa de acceso donde se conectan los dispositivos finales, como computadoras, impresoras, teléfonos IP y puntos de acceso inalámbrico (AP) [1]. En la Figura 2.1 se ilustra este modelo con enlaces redundantes entre capas. La capa de acceso puede combinar *switches* de distinto tipo, incluyendo equipos modulares, apilables (*stacks*) y de configuración fija.

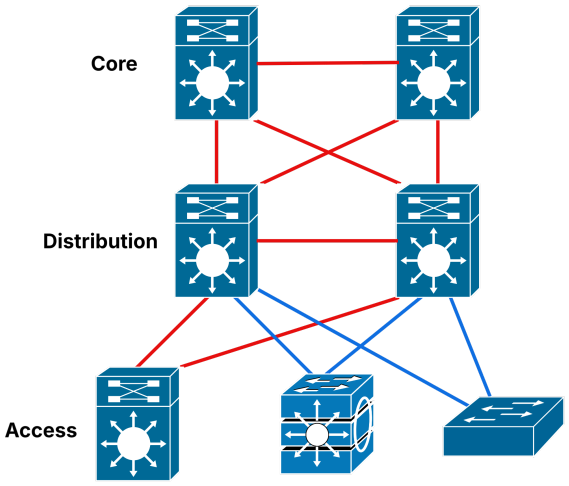


Figura 2.1: Modelo jerárquico de una red de campus.

Fuente: elaboración propia basada en Cisco Systems [1, Figura 1].

El elemento que articula esta arquitectura es el *switch*. En su concepción original, definida por el estándar IEEE 802.1D, un *switch* opera en la capa de enlace del modelo OSI y reenvía tramas en función de sus direcciones MAC [2]. Sin embargo, los diseños modernos de campus contemplan también *switches* multicapa, que además de conmutar en capa 2 realizan reenvío de capa 3 y enrutan el tráfico entre redes lógicas, una función que

tradicionalmente se asignaba a los *routers* [1]. Familias de productos como Cisco *Catalyst* o Huawei *CloudEngine* incluyen modelos con estas capacidades.

Los *switches* administrables cuentan con un sistema operativo de red que permite configurar redes lógicas, puertos y parámetros de gestión [3][4]. En una organización de mediana o gran escala, la red de campus reúne un número elevado de estos equipos, frecuentemente de distintos fabricantes y generaciones. Buena parte de la carga operativa cotidiana recae sobre la capa de acceso: reubicar usuarios, conectar AP, proveer telefonía IP o aislar impresoras. Los centros de datos utilizan también *switches* administrables para interconectar servidores físicos, aunque con modelos y diseños que pueden diferir de los de campus.

2.2. La configuración de un *switch*

2.2.1. Sistema operativo y configuración

Cada fabricante desarrolla su propio sistema operativo de red, como IOS en Cisco y VRP (*Versatile Routing Platform*) en Huawei. Aunque cumplen funciones equivalentes, difieren en la jerarquía de modos de ejecución y en la sintaxis de configuración [3][4].

El comportamiento del *switch* se define mediante su configuración, un conjunto de comandos en formato de texto. Los cambios tienen efecto inmediato sobre la configuración en ejecución (*running-config*), pero sólo se conservan tras un reinicio si se guardan en la de arranque (*startup-config*), con '*write'* en Cisco o '*save'* en Huawei. Por eso un cambio sin guardar puede perderse semanas después, ante un corte de energía [5].

Para modificarla, el administrador recorre niveles con distintos privilegios. En IOS pasa del modo de usuario al privilegiado (*enable*) y de allí a la configuración global (*configure terminal*), desde la que accede a cada interfaz o VLAN. En VRP, la vista de sistema (*system-view*) cumple el mismo papel.

2.2.2. Qué se configura

Las tareas que un administrador realiza habitualmente sobre un *switch*, y que constituyen el alcance de este trabajo, se agrupan en cuatro categorías:

- **Redes lógicas (VLAN)**: Una VLAN divide una misma infraestructura física en varios dominios de difusión independientes. El estándar IEEE 802.1Q define la etiqueta que permite transportar varias VLAN por un mismo enlace, con un identificador de 12 bits

cuyos valores utilizables van de 1 a 4094 [6]. Los fabricantes permiten, además, asignarle un nombre. De este modo, el tráfico de usuarios, telefonía o gestión puede separarse sin cableado dedicado.

- **Puertos físicos**: un puerto en modo *access* pertenece a una única VLAN y conecta dispositivos finales, mientras que uno en modo *trunk* transporta varias VLAN etiquetadas entre *switches*, salvo la nativa, que viaja sin etiqueta [7]. Cada puerto tiene además descripción, estado administrativo, control de tormentas (*storm-control*) y alimentación por Ethernet (*PoE*).
- **Interfaces virtuales (SVI)**: son interfaces lógicas de Capa 3 asociadas a una VLAN. Con una dirección IP actúan como puerta de enlace de su subred o como dirección de gestión del *switch* [8]. Sobre ellas se aplican listas de control de acceso (*ACL*) y el reenvío de solicitudes DHCP hacia servidores centralizados (*DHCP relay*).
- **Configuración global**: parámetros que afectan al equipo en su conjunto, como el nombre, los servidores DNS, NTP y *syslog*, la supervisión por SNMP [9], las rutas estáticas y el guardado de la configuración.

Un *switch* admite muchas otras funciones que quedan fuera del alcance de este trabajo. Entre ellas se encuentran el protocolo de árbol de expansión (STP), que evita bucles en topologías redundantes [2], la agregación de enlaces (LACP), que agrupa varios enlaces físicos en un único enlace lógico, la duplicación de tráfico (*port mirroring*), la calidad de servicio (*QoS*) y los protocolos de enrutamiento dinámico, como OSPF. Se trata de funciones que suelen definirse al diseñar la red y que, en el contexto de este trabajo, se modifican con menor frecuencia que las descritas anteriormente, propias de la operación cotidiana.

2.2.3. La heterogeneidad entre fabricantes

Si bien los conceptos anteriores son comunes a todos los fabricantes, su expresión en comandos no lo es. En el Cuadro 2.1 se compara la misma operación, crear una VLAN y asignarla a un puerto en modo *access*, en ambos sistemas.

Las diferencias no se limitan a la sintaxis. Los puertos se configuran de diferente manera, los mensajes de error tienen formatos dispares, algunos comandos piden confirmación interactiva y otros no, e incluso entre versiones de un mismo sistema operativo un comando puede cambiar o dejar de existir. Por lo tanto, un administrador que gestiona una red con equipos de ambos fabricantes debe dominar las dos sintaxis y sus particularidades, y

toda herramienta que pretenda automatizar estas tareas tiene que resolver esta misma heterogeneidad.

Paso	Cisco IOS	Huawei VRP
Modo	configure terminal	system-view
Crear la	vlan 10	vlan 10
Asignarle	name USUARIOS	name USUARIOS
Seleccionar	interface Gi1/0/5	interface GigabitEthernet0/0/5
Definir el	switchport mode access	port link-type access
Asignar la	switchport access vlan 10	port default vlan 10
Confirmar	no requiere	commit (serie CloudEngine)
Salir	end	return
Guardar	write memory	save

Cuadro 2.1: Creación de una VLAN y asignación a un puerto en modo access en Cisco IOS y Huawei VRP

2.3. La administración de los equipos

2.3.1. Canales de administración

Una de las formas más habituales de administrar un *switch* es a través de su línea de comandos (*CLI*). Localmente se accede a ella mediante la consola, una conexión serie que no depende de la red y resulta útil cuando el equipo aún no está configurado. En forma remota se utilizan Telnet o SSH. Telnet no protege la sesión [10], por lo que en la práctica fue reemplazado por SSH, que ofrece la misma experiencia sobre un canal cifrado [11]. Muchos *switches* integran también una interfaz web, práctica para tareas esporádicas pero poco apta para la automatización.

SSH establece un canal cifrado entre el cliente y el equipo, autentica a ambos extremos y transporta sobre ese canal la sesión de la CLI. Para autenticar al usuario, los *switches* admiten principalmente la contraseña y la clave pública. Con clave pública, el cliente firma la solicitud con su clave privada y el equipo la verifica con la clave pública registrada, de modo que la clave privada nunca se transmite. Este método no depende de una contraseña que deba memorizarse o compartirse. Además admite la autenticación no interactiva, lo que lo hace adecuado para procesos automatizados [14].

Existen además protocolos de gestión que intercambian información estructurada en lugar de texto. SNMP se utiliza principalmente para el monitoreo del estado de los equipos [9], mientras que NETCONF y RESTCONF permiten consultar y modificar la configuración

como datos descritos por modelos YANG [12][13]. Aunque resultan más adecuados para la automatización, su disponibilidad depende del modelo y la versión de cada equipo.

2.3.2. Configuración inicial

En la documentación de los fabricantes y en las prácticas de automatización es habitual distinguir entre el día 0, en el que el equipo recibe la configuración mínima necesaria para su gestión, el día 1, en el que se prepara para la función que cumplirá, y el día 2, que abarca los cambios a lo largo de su vida útil. En este trabajo se adopta esta terminología.

Los mecanismos de aprovisionamiento automático (*Zero Touch Provisioning*, ZTP) permiten resolver el día 0 con poca o ninguna intervención manual [15]. Este trabajo se enfoca en los días 1 y 2, y parte del supuesto de que el día 0 ya ocurrió. Para que un sistema pueda administrar un *switch* de forma remota, el equipo debe haber sido configurado previamente, ya sea por consola o mediante algún mecanismo de aprovisionamiento, con al menos:

- Una dirección IP de gestión y una ruta predeterminada, para que sea alcanzable.
- Un par de claves propio del equipo (por ejemplo, RSA), con el que el servidor SSH se autentica ante los clientes [11].
- El servicio SSH habilitado en su versión 2, como único protocolo de acceso remoto.
- Un usuario local con privilegios de administración, autenticado por contraseña o por clave pública. En este último caso, el equipo debe tener registrada la clave pública de la plataforma.

2.4. Automatización de redes

Las características de los equipos presentadas en este capítulo, desde la configuración comando a comando hasta la sintaxis propia de cada fabricante, dificultan su administración manual a escala. Necesidades similares de consistencia, escala, trazabilidad y repetibilidad impulsaron, en la administración de servidores, el desarrollo de las herramientas de gestión de configuración. Su aplicación al dominio de las redes es lo que se conoce como automatización de redes.

La automatización de redes consiste en delegar en *software* la ejecución de las tareas de configuración y consulta, de modo que un cambio se describa una única vez y pueda aplicarse de forma repetible sobre uno o varios dispositivos [16]. Se vincula con la programabilidad de la red, entendida como la capacidad de inicializar, controlar, modificar y

gestionar su comportamiento mediante interfaces programáticas con una especificación pública [17]. Entre sus beneficios se destacan una mayor eficiencia y confiabilidad, y la reducción del riesgo de errores asociado a las tareas rutinarias.

2.4.1. De los scripts a las herramientas

El primer paso hacia la automatización suele ser un *script* que reproduce lo que haría una persona, abriendo una sesión SSH, enviando comandos y leyendo las respuestas, por ejemplo con Paramiko, una implementación de SSHv2 en Python [18]. Rápidamente aparecen, sin embargo, las dificultades propias de la CLI, como detectar cuándo el equipo terminó de responder, manejar la paginación, reconocer los errores de cada fabricante o interpretar un texto pensado para ser leído por una persona.

Estas dificultades dieron lugar a herramientas de mayor nivel. Netmiko y Scrapli ofrecen controladores para múltiples plataformas que simplifican el manejo de la sesión con equipos de distintos fabricantes, aunque los comandos y la interpretación de las respuestas siguen siendo propios de cada uno [19][20]. NAPALM, por su parte, ofrece una API unificada que devuelve los datos de los equipos en un formato común [21]. Nornir es un *framework* en Python que ejecuta tareas de forma concurrente sobre un inventario de equipos, delegando la comunicación en complementos (*plugins*) basados en esas bibliotecas [22].

Todas estas herramientas están orientadas a quien programa, ya que cada tarea se escribe como código Python. En paralelo, y provenientes de la administración de servidores, se consolidaron las herramientas de gestión de configuración, que describen las tareas en archivos de texto en lugar de programarlas. Entre ellas se encuentra Ansible.

2.4.2. Ansible

Ansible es una herramienta de código abierto mantenida por Red Hat junto con su comunidad, que se utiliza tanto en servidores como en redes. No usa agentes (*agentless*), ya que no requiere instalar *software* en los equipos gestionados, sino que se conecta a ellos desde un nodo de control. Las tareas se describen en archivos YAML denominados *playbooks* y se ejecutan mediante módulos, componentes encargados de una acción concreta. En los equipos de red, los módulos corren en el propio nodo de control y envían los comandos al equipo por SSH. Los equipos a gestionar y sus variables se definen en un inventario [16].

En el caso de los servidores, Ansible copia y ejecuta código Python en el equipo remoto. Los dispositivos de red, en cambio, normalmente no ejecutan ese código, por lo que

Ansible utiliza tipos de conexión específicos: *'**network_cli**'* para la CLI sobre SSH, *'**netconf**'* para NETCONF y *'**httpapi**'* para API sobre HTTP [23]. Con *'**network_cli**'*, los módulos se ejecutan en el propio nodo de control y envían comandos al equipo a través de una sesión SSH persistente, y el propio complemento de conexión gestiona aspectos de la interacción con la CLI, como la detección del *prompt* y los tiempos de espera [24]. La plataforma de cada equipo se indica mediante variables como *'**ansible_network_os**'*, y el soporte para cada sistema operativo se distribuye en colecciones, como *'**cisco.ios**'* para Cisco IOS o *'**community.network**'*, que en su versión 5.1.0 incluye los módulos '**ce**' para los equipos Huawei *CloudEngine* [23].

Ansible resuelve así la ejecución de los cambios y parte de la heterogeneidad entre fabricantes, aunque presenta limitaciones en este contexto. El grado de soporte y el mantenimiento varían entre las colecciones, los paquetes en que se distribuyen los módulos de cada fabricante, y entre sus versiones. Además, según el módulo, el resultado de una consulta puede entregarse como datos estructurados o como el texto de salida de la CLI, que debe interpretarse. Por último, su interfaz natural es la línea de comandos del operador y no un servicio que otras aplicaciones puedan consumir [16].

Tanto las bibliotecas de Python como Ansible están pensadas, en su forma base, para un ingeniero que conoce la red, escribe código o *playbooks* y los ejecuta. Sobre ellas se construyeron plataformas de ejecución de automatizaciones, como AWX, base del *Automation Controller* de Red Hat Ansible Automation Platform, Rundeck o Semaphore, que agregan una interfaz web, un historial de ejecuciones y control de acceso por usuario [47][48][49].

2.5. Programabilidad de redes y SDN

La automatización descrita en la sección anterior se construye sobre la CLI, una interfaz pensada para el operador. La programabilidad de redes, en cambio, propone que los equipos expongan interfaces diseñadas para programas. Además de tratar la configuración como datos (sección 2.3.1), protocolos como NETCONF permiten validar un cambio antes de aplicarlo y confirmarlo como una unidad [12].

En torno a esta idea se difundió una forma de trabajo, conocida como NetDevOps, que aplica a la red prácticas del desarrollo de *software*, como el control de versiones de la configuración, las pruebas automatizadas, la integración continua y el uso de API en lugar de

comandos interactivos. Los fabricantes la impulsan con programas propios, como DevNet de Cisco, que ofrece documentación, laboratorios y formación en Python, API REST, NETCONF y Ansible [25].

Llevada a su expresión más amplia, la idea de gestionar la red mediante *software* da lugar a las redes definidas por *software* (SDN). SDN propone separar el plano de control, que decide cómo se reenvía el tráfico, del plano de datos, que efectivamente lo reenvía [17]. El plano de control se centraliza en un *software* denominado controlador, que programa a los equipos a través de una interfaz estándar, como OpenFlow [26].

Como se muestra en la Figura 2.2, el controlador expone API hacia las aplicaciones (interfaz *northbound*) y se comunica con los equipos mediante protocolos *southbound* [26]. Entre los controladores de código abierto se encuentran OpenDaylight y ONOS.

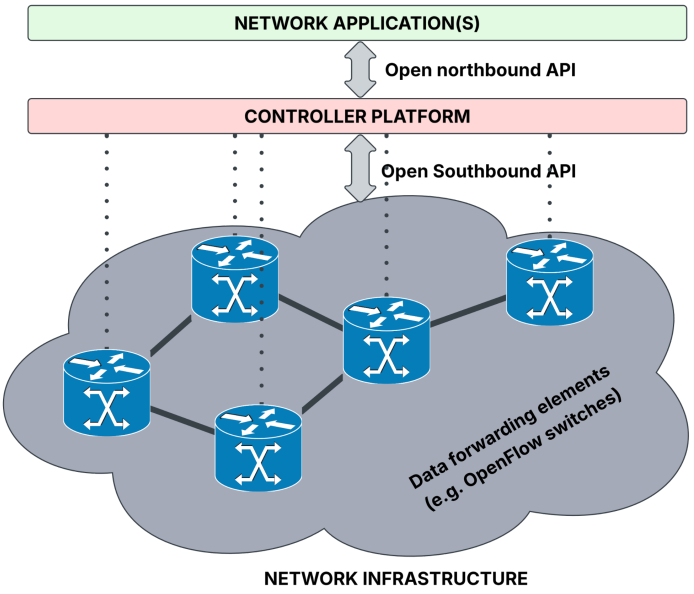


Figura 2.2: Vista simplificada de una arquitectura SDN, con las interfaces 'northbound' y 'southbound'. Fuente: elaboración propia basada en Software-Defined Networking [26, Figura 1].

La adopción de SDN puede requerir controladores y equipos compatibles y, según la arquitectura elegida, reemplazar o actualizar parte de la infraestructura, aunque también existen estrategias de adopción gradual o híbrida. Por su parte, los fabricantes ofrecen plataformas de gestión centralizada, como Cisco Catalyst Center o Huawei iMaster NCE, que aplican la idea de un control centralizado a la configuración y el monitoreo de equipos tradicionales, sin separar los planos de control y de datos, y están orientadas principalmente a los productos de su propio fabricante.

Para una organización con un parque heterogéneo de *switches* tradicionales, este camino no siempre resulta viable. Existe, entonces, un espacio intermedio: tomar de SDN la idea de una gestión centralizada expuesta mediante una API, pero aplicarla sobre los equipos existentes, administrados por su CLI, sin modificar el funcionamiento de sus planos de control y de datos. Es en ese espacio donde se ubica el presente trabajo.

2.6. API REST en la automatización de redes

Una API es un conjunto de definiciones y protocolos que permite que distintos sistemas interactúen de forma estandarizada, sin conocer los detalles de su implementación. Entre los estilos existentes, REST, propuesto por Fielding como estilo arquitectónico para sistemas distribuidos, es ampliamente utilizado en servicios web. Una API REST expone recursos identificados mediante URI, y cada solicitud contiene la información necesaria para interpretarla, sin depender de un estado de sesión conservado en el servidor [27]. Los recursos se manipulan con los métodos estándar de HTTP: *GET* para obtener una representación, *POST* para crear o procesar contenido, *PUT* para reemplazarlo, *PATCH* para modificarlo parcialmente y *DELETE* para eliminarlo [28]. Su descripción suele formalizarse con la especificación OpenAPI, que documenta las API HTTP de forma independiente del lenguaje y legible tanto por personas como por programas [29].

En la automatización de redes, una API REST puede actuar como capa de abstracción entre quien solicita un cambio y los equipos que lo ejecutan. Es el mismo principio que sigue la interfaz *northbound* de los controladores SDN [26], así como RESTCONF, que expone mediante HTTP datos modelados con YANG [13]. Para expresar una operación como asignar un puerto a una VLAN de forma independiente del fabricante, la API necesita un modelo común de los recursos. También necesita una capa que traduzca cada solicitud a los comandos de cada fabricante. Así, la automatización deja de depender de que el operador escriba código o *playbooks*, y puede ser consumida tanto por una interfaz gráfica como por otros sistemas de la organización.

3. Planificación del Proyecto

Para estructurar el desarrollo del proyecto se adoptó un **Modelo Iterativo Incremental** (Figura 3.1), en el que el sistema se construyó en incrementos sucesivos y cada uno produjo una versión funcional que ampliaba la anterior.

El modelo se eligió porque se adaptaba a un equipo de trabajo acotado que necesitaba validar primero las capacidades de mayor riesgo, dejando las de mayor complejidad para etapas posteriores. Cada incremento correspondió a una iteración que combinó la rigurosidad de la ingeniería de *software* con la flexibilidad de los principios ágiles, ya que el diseño se ajustó según los hallazgos de las validaciones de la iteración anterior [32].

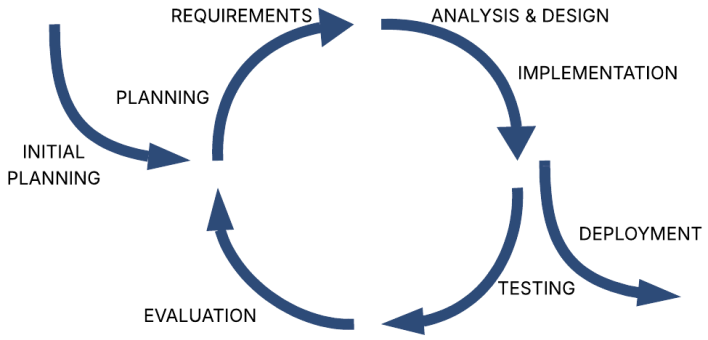


Figura 3.1: Modelo de proceso iterativo incremental adoptado en el proyecto.
Fuente: elaboración propia basada en ISO/IEC/IEEE 12207:2026 [32]

Dado que el proyecto tenía un alcance acotado y metas bien definidas, el desarrollo comenzó con una fase de descubrimiento de la problemática y de elicitación de los requerimientos de todo el sistema. Posteriormente se realizó el diseño general de la arquitectura. Una vez finalizadas estas etapas, las funcionalidades se agruparon en tres iteraciones, priorizando las de mayor criticidad.

3.1. Descubrimiento de Requerimientos y Diseño del Sistema

En esta fase se delimitó el alcance del sistema y se identificaron los problemas de la administración manual de redes *multivendor*, mediante entrevistas a los administradores y la observación de sus procedimientos. Las actividades clave fueron:

1. **Definición de la problemática:** se describieron las dificultades de la administración manual que el sistema debía resolver.

2. **Selección del modelo de solución:** se compararon las alternativas existentes para elegir la que mejor se ajustara a los equipos y al proyecto.
3. **Elicitación y especificación de requerimientos:** las necesidades relevadas se expresaron como requerimientos funcionales y no funcionales, junto con los casos de uso.
4. **Análisis de riesgos y priorización:** cada requerimiento se evaluó según su probabilidad de falla y su impacto.
5. **Asignación a iteraciones:** los requerimientos se distribuyeron en tres iteraciones según su prioridad y sus dependencias.

Sobre esa base, la fase de diseño general tradujo los requerimientos elicitados en un diseño modular del sistema antes de iniciar su implementación. Las tareas clave fueron:

1. **Diseño de la arquitectura:** se adoptó una arquitectura hexagonal, documentada con el modelo C4.
2. **Definición de los módulos:** el sistema se dividió en módulos con responsabilidades acotadas, en torno a un núcleo común.
3. **Modelado de datos:** se diseñó el esquema relacional del sistema.
4. **Diagramas de clases y de secuencia:** se modelaron la estructura de cada módulo y la interacción entre sus componentes.
5. **Selección del *stack* tecnológico:** se eligieron las tecnologías de cada módulo frente a las alternativas evaluadas.
6. **Configuración del entorno de desarrollo:** se prepararon los entornos reproducibles y el modo simulado.

3.2. Iteraciones del Proyecto

Finalizadas las fases de descubrimiento y diseño general, el trabajo siguió el modelo iterativo incremental, agrupando los requerimientos en iteraciones y priorizando los de mayor criticidad. Cada iteración recorrió las etapas de la Figura 3.1. Como el planeamiento, los requerimientos y el diseño general ya se habían abordado, las primeras etapas de cada iteración consistieron en revisar lo definido y adaptarlo a los resultados de las iteraciones anteriores, incorporando las nuevas necesidades que surgieron durante el desarrollo.

Etapas de cada iteración

1. **Planificación y diseño:** se fijó el objetivo de la iteración y se seleccionaron los requerimientos que abordaría. A partir de ellos se ajustó el diseño general, detallando los componentes que se incorporaban o modificaban antes de su implementación.
2. **Implementación:** se desarrollaron las funcionalidades previstas sobre la arquitectura definida. Cada funcionalidad se integró con las existentes, de modo que al cierre de la iteración el sistema se mantuviera operativo.
3. **Pruebas y validaciones:** se ejecutaron las pruebas automatizadas, que verificaban cada funcionalidad sin depender de los equipos. Luego se validó el comportamiento sobre equipos reales, primero en el laboratorio y, en la última iteración, sobre los equipos en producción.
4. **Resultados y ajustes:** se analizaron los resultados obtenidos, incluido el desempeño de las operaciones. Las fallas detectadas se corrigieron. Los problemas pendientes orientaron los objetivos de la iteración siguiente.

4. Ingeniería de Requerimientos

En este capítulo se plantea el dominio del problema y se desarrollan los requerimientos del sistema destinado a resolverlo. Se evalúan los modelos de solución considerados, se describen las funciones que la plataforma debe ofrecer y se especifican sus requerimientos funcionales y no funcionales, junto con el criterio utilizado para ponderar el riesgo de cada uno y ordenar su implementación.

4.1. Definición de la Problemática

En la red considerada, la administración de los *switches* se realiza de forma manual, equipo por equipo, sobre un parque que combina modelos Cisco *Catalyst* y Huawei de las series S y *CloudEngine*. Sobre esta base, las características descritas en el Capítulo 2 se traducen en seis dificultades concretas.

En primer lugar, conocer el estado de la red exige consultar cada equipo por separado. Determinar la configuración de un puerto, las VLAN presentes en un conjunto de *switches* o el puerto al que se conecta un dispositivo a partir de su dirección MAC implica recorrer varios equipos con los comandos de cada fabricante.

En segundo lugar, cada operación debe expresarse con la sintaxis del fabricante (Cuadro 2.1), que además puede variar entre familias y versiones de un mismo fabricante, por lo que su ejecución depende del conocimiento de quien la realiza.

En tercer lugar, los cambios que involucran a varios equipos deben repetirse en cada uno. Agregar una VLAN en todos los *switches* de un edificio implica repetir la operación tantas veces como equipos existan. Si falla en alguno, hay que identificar en cuáles se aplicó para que la configuración no quede inconsistente.

En cuarto lugar, los cambios no se validan antes de aplicarse, ya que cada comando tiene efecto apenas se ingresa. Asignar un puerto a una VLAN que no existe en el equipo deja sin conectividad a los dispositivos conectados. Escribir ‘***`switchport trunk allowed vlan 20`***’ en lugar de ‘***`switchport trunk allowed vlan add 20`***’ reemplaza la lista completa de VLAN permitidas del troncal. Si así se excluye la VLAN de gestión, los *switches* que dependen de ese enlace quedan inaccesibles y recuperarlos requiere acceder físicamente a su consola.

En quinto lugar, falta trazabilidad. Si bien los equipos pueden registrar los comandos ejecutados, en la práctica esos registros no se centralizan y, al utilizarse credenciales compartidas, identifican a la cuenta y no a la persona. Ante un incidente, reconstruir qué cambio se aplicó, sobre qué equipo, en qué momento y por quién resulta lento e incompleto.

Por último, los niveles de privilegio que ofrecen los equipos autorizan comandos y no recursos, y deben configurarse en cada equipo con la sintaxis de su fabricante, por lo que no permiten delegar tareas acotadas sin conceder un acceso más amplio del necesario.

De estas dificultades surge la necesidad de una plataforma que abstraiga las diferencias entre fabricantes, aplique los cambios verificando el estado previo y con capacidad de *rollback*, registre cada intervención y restrinja qué puede hacer cada usuario y sobre qué equipos.

4.2. Modelos de Solución

En esta sección se analizan los enfoques existentes para automatizar la administración de equipos de red, evaluados según su capacidad de resolver las dificultades descritas en la sección anterior. Los criterios de evaluación son los siguientes:

1. **Abstracción *multivendor*:** una misma operación lógica se solicita de igual forma, independientemente del fabricante del equipo de destino.
2. **Viabilidad sobre el parque existente:** el enfoque es aplicable sobre los equipos en servicio, sin reemplazo de *hardware* ni costos de licenciamiento adicionales.
3. **Confiabilidad de la operación:** el cambio se valida antes de aplicarse y se ejecuta verificando el estado previo, con reintentos ante fallas transitorias y *rollback* ante fallas permanentes. Cuando involucra varios equipos, la falla de uno no interrumpe a los demás.
4. **Trazabilidad y control de acceso:** cada cambio queda registrado de forma centralizada, con permisos que pueden acotarse por equipo y por tipo de operación.
5. **Interfaz para operadores:** el estado de los equipos puede consultarse y modificarse desde una interfaz web centralizada, sin requerir conocimiento de los comandos de cada fabricante.

Los controladores SDN presentados en la sección 2.5, como OpenDaylight u ONOS, se descartan antes de la comparación. Centralizan el plano de control y modifican el modelo de operación de la red, mientras que el problema planteado consiste en administrar la

configuración de *switches* tradicionales ya en servicio, cuyo plano de control permanece distribuido en cada equipo. A continuación se evalúan tres modelos: la automatización mediante *scripts* y *playbooks*, las plataformas genéricas de ejecución de automatizaciones y los controladores de gestión propietarios.

4.2.1. Automatización mediante Scripts y Playbooks

Consiste en automatizar las tareas repetitivas mediante *scripts* o *playbooks* que el operador ejecuta, habitualmente, desde su propia terminal, con bibliotecas de acceso SSH como Netmiko o con herramientas como Ansible (sección 2.4).

Entre sus **ventajas** se destacan:

1. Bajo costo de adopción: son herramientas de código abierto que operan sobre SSH y pueden aplicarse sobre los equipos en servicio.
2. Operación sobre múltiples equipos: un mismo *playbook* puede ejecutarse sobre un conjunto de equipos de un mismo fabricante, y la falla de uno no interrumpe la ejecución sobre los demás.

Entre sus **desventajas** se encuentran:

1. La abstracción es por plataforma y no por operación: cada fabricante requiere su propio *playbook* o módulo, y el operador debe elegir cuál ejecutar.
2. La confiabilidad depende de cada *script*: si bien Ansible ofrece mecanismos genéricos de reintento y manejo de fallas, qué estado leer antes del cambio y cómo realizar el *rollback* debe programarse para cada operación.
3. La ejecución está descentralizada: sin una infraestructura adicional, no existe un registro centralizado ni un control sobre quién ejecuta qué.

Este enfoque resuelve la repetición de tareas, pero no la heterogeneidad ni la trazabilidad, ya que traslada el problema de conocer la sintaxis de cada equipo al de mantener un repertorio de *scripts* por fabricante.

4.2.2. Plataformas Genéricas de Ejecución de Automatizaciones

Consiste en centralizar la ejecución de las automatizaciones en un servidor que agrega al enfoque anterior una interfaz web, un historial de ejecuciones y control de acceso por usuario. Herramientas como AWX, el proyecto de código abierto sobre el que se basa el Automation Controller de Red Hat Ansible Automation Platform, Rundeck o Semaphore siguen este enfoque. El operador lanza plantillas de trabajo predefinidas, que asocian un

playbook con el inventario y las credenciales necesarias, completando sus parámetros mediante formularios y sin acceder a una terminal.

Entre sus **ventajas** se destacan:

1. Centralización: conservan el historial de cada trabajo, con su salida y su resultado, y almacenan las credenciales de los equipos sin exponerlas a los usuarios.
2. Control de acceso: permiten definir qué usuarios pueden ejecutar qué plantillas y sobre qué inventarios.

Entre sus **desventajas** se encuentran:

1. No modelan los recursos de red: son herramientas de propósito general, por lo que la abstracción multivendor sigue dependiendo de los *playbooks*.
2. La confiabilidad sigue a cargo de los *playbooks*: la lectura del estado previo y el *rollback* deben programarse en cada uno.
3. Interfaz orientada a lanzar trabajos: no permite consultar el estado de los equipos ni operar sobre sus recursos.
4. Costo o complejidad de operación: las ediciones comerciales requieren licencia, y las comunitarias carecen de soporte. En el caso de AWX, su instalación requiere además un *clúster* de Kubernetes.

Estas plataformas resuelven la centralización y el control de acceso que le faltaban al enfoque anterior, pero no la heterogeneidad ni la confiabilidad, que siguen dependiendo de los *playbooks* que ejecutan.

4.2.3. Controladores de Gestión Propietarios

Consiste en adoptar la plataforma de gestión centralizada que cada fabricante ofrece para sus propios equipos, como Cisco Catalyst Center o Huawei iMaster NCE. Estos controladores modelan la red en términos de recursos, permiten aprovisionar configuraciones mediante plantillas e integran interfaz web, control de acceso y registro de operaciones.

Entre sus **ventajas** se destacan:

1. Cobertura funcional: abarcan ampliamente los equipos del propio fabricante, con modelado de recursos y validación de configuraciones.
2. Integración: interfaz web, control de acceso y registro de operaciones en una misma plataforma.

Entre sus **desventajas** se encuentran:

1. Orientación a un único fabricante: en un parque heterogéneo, cada fabricante requiere su propio controlador, lo que reproduce la fragmentación que se busca eliminar.
2. Costo y requisitos de *hardware*: requieren licenciamiento y, en general, modelos y versiones de *firmware* específicos para su soporte completo.

En conclusión, estos controladores resuelven el problema dentro del ecosistema de un fabricante, pero su adopción en un parque multivendor implica duplicar plataformas y costos, o bien homogeneizar el *hardware*.

4.3. Elección del Modelo

Las ventajas y desventajas de los modelos analizados se resumen en el Cuadro 4.1.

Modelo	Abstracción multivendor	Viabilidad sobre el parque existente	Confiabilidad de la operación	Trazabilidad y control de acceso	Interfaz para operadores
Scripts y playbooks	~	✓	✗	✗	✗
Plataformas genéricas de ejecución	~	✓	✗	✓	✓
Controladores propietarios	✗	✗	~	✓	✓

Cuadro 4.1: Comparación de modelos de solución (✓ cumple, ~ cumple parcialmente, ✗ no cumple)

Ninguno de los modelos analizados satisface simultáneamente los cinco criterios. Las plataformas genéricas y los controladores propietarios aportan la centralización, el control de acceso y la interfaz web que los *scripts* no ofrecen. Los controladores, además, modelan la red en términos de recursos. Sin embargo, ninguno combina la abstracción *multivendor* con la viabilidad sobre el parque existente y la confiabilidad de la operación.

Por lo tanto, se opta por desarrollar una plataforma propia que toma un elemento de cada enfoque:

1. **De la automatización mediante *playbooks*, el motor de ejecución**: los cambios se aplican sobre SSH, sin agentes, mediante Ansible, lo que garantiza la viabilidad sobre el parque existente.
2. **De las plataformas genéricas, el modelo operativo**: la ejecución se centraliza en un servidor que conserva el historial de cada trabajo y controla quién puede lanzarlo.
3. **De los controladores propietarios, el modelado de la red en términos de recursos**: el operador solicita operaciones sobre VLAN, puertos o interfaces, y no ejecuta comandos.

Sobre esa base, la plataforma incorpora lo que ninguno de los modelos ofrece en conjunto. Traduce cada operación lógica a la sintaxis del fabricante del equipo de destino. Antes de aplicar un cambio lee el estado previo del recurso, lo reintenta ante fallas transitorias y lo revierte ante fallas permanentes. Además, mantiene el estado de los equipos disponible para su consulta desde una interfaz centralizada.

4.4. Elicitación de Requerimientos

Los requerimientos de la plataforma se elicitaron mediante entrevistas con el personal de administración de red, el relevamiento de los procedimientos vigentes y pruebas de comandos sobre equipos de cada fabricante. Luego se transformaron en requerimientos verificables, trazables y priorizados, siguiendo la norma ISO/IEC/IEEE 29148:2018 [35]. De este proceso surgieron tres perfiles de uso: quien consulta el estado de la red, quien opera cambios de configuración y quien administra el inventario y los permisos. La plataforma debe reflejar esa separación y permitir acotar el alcance de cada usuario a una parte de la red.

El sistema debe mantener un inventario de equipos organizados en sitios y grupos, con sus credenciales cifradas. El estado de cada equipo se lee al registrarlo y se actualiza tras cada cambio, a pedido del usuario o al consultarlo cuando su información está desactualizada, de modo que pueda consultarse sin volver a conectarse al equipo.

Sobre cada equipo, el sistema debe permitir operar las VLAN, los puertos físicos, las interfaces virtuales y la configuración global. Cada operación se solicita en términos lógicos y el sistema la traduce a la sintaxis del fabricante. El sistema debe permitir aplicar las operaciones sobre VLAN a varios equipos a la vez y agrupar en una sola operación los cambios sobre puertos e interfaces virtuales de un mismo equipo.

Toda operación de escritura se valida antes de ejecutarse y se procesa de forma asíncrona como un trabajo, cuyo estado el usuario puede seguir. El sistema lee el estado previo del recurso antes de aplicar el cambio, reintenta la aplicación ante fallas transitorias y restaura el estado previo si no puede completarla. Cada operación queda registrada en un historial de auditoría con el usuario, el equipo, la operación, el resultado y el momento.

El acceso requiere autenticación, y cada usuario opera con un rol de observador, operador o administrador sobre un sitio o un grupo de equipos. Un administrador del sistema, con alcance sobre toda la plataforma, gestiona los sitios y los demás administradores. El

sistema debe impedir cualquier operación que exceda el rol del usuario sobre el recurso involucrado.

4.5. Limitaciones y Supuestos

4.5.1. Limitaciones

- **Tiempo de desarrollo**: al tratarse de un proyecto integrador de fin de carrera, el plazo de desarrollo es acotado y no extensible, lo que restringe el alcance funcional a los recursos de red de uso más frecuente.
- **Equipamiento de prueba**: se dispone de un número reducido de equipos físicos por fabricante. Las pruebas sobre equipos en producción sólo pueden realizarse dentro de ventanas de mantenimiento y sobre puertos o interfaces sin uso, lo que limita la cantidad de modelos y versiones de *firmware* verificados.
- **Fabricantes soportados**: el desarrollo se circunscribe a Cisco IOS y Huawei VRP, los fabricantes presentes en el parque relevado.
- **Mecanismo de acceso**: la gestión se realiza por SSH sobre la interfaz de línea de comandos. La disponibilidad de interfaces programáticas como NETCONF o RESTCONF depende del modelo y la versión de cada equipo. En los equipos relevados, estas interfaces no estaban disponibles o, en el caso de NETCONF, sólo permitían transportar comandos de CLI, sin los datos estructurados modelados con YANG que constituyen su principal ventaja. Por este motivo, la CLI es el único mecanismo de acceso común a todo el parque.

4.5.2. Supuestos

- **Configuración inicial de los equipos**: se asume que cada equipo ya cuenta con su configuración de día 0 (sección 2.3.2), es decir, una dirección IP de gestión alcanzable por SSH desde el servidor de la plataforma y una cuenta local con privilegios suficientes para leer y modificar su configuración.
- **Canal único de cambios**: se asume que los cambios sobre los recursos gestionados se realizan a través de la plataforma. Los cambios aplicados directamente sobre la CLI de un equipo se reflejan en la siguiente sincronización del estado, pero no quedan registrados en la auditoría.
- **Estabilidad del alcance**: se asume que los requerimientos no cambian durante el desarrollo, y que la validación con los usuarios sólo da lugar a ajustes menores sobre las funcionalidades previstas.

4.6. Especificación de Requerimientos

En esta sección se presentan los actores y los casos de uso del sistema, que sirven de base para los requerimientos funcionales y no funcionales elicitados. Al final se describe el análisis de riesgo con el que se ponderó la prioridad de cada requerimiento dentro del desarrollo.

4.6.1. Actores y Casos de Uso

El sistema contempla cuatro actores humanos, definidos por el rol que cada usuario tiene sobre un sitio o un grupo de equipos. A ellos se suma un actor externo, el equipo de red, sobre el que se aplican las operaciones. Los roles son acumulativos, de modo que cada uno incluye las capacidades del anterior:

- **Observador**: consulta el estado de los equipos, los trabajos y el registro de auditoría dentro de su alcance.
- **Operador**: crea y renombra VLAN, configura puertos e interfaces virtuales, guarda la configuración de los equipos, cancela trabajos pendientes y reintenta *rollbacks* fallidos.
- **Administrador**: registra, modifica y elimina equipos, gestiona grupos, elimina VLAN y modifica la configuración global de los equipos. Si su alcance es un sitio completo, gestiona además los usuarios y asigna permisos.
- **Administrador del sistema**: posee acceso total, crea y elimina sitios, designa a otros administradores del sistema, desbloquea cuentas y depura el registro de auditoría.

La Figura 4.1 presenta los casos de uso principales del sistema. Las flechas entre actores indican que cada uno hereda los casos de uso del actor al que apunta, de modo que un operador puede realizar todo lo que hace un observador, y un administrador todo lo que hace un operador. El equipo de red participa sólo en los casos que requieren conectarse a él, como registrar un equipo o aplicar un cambio, mientras que las consultas se resuelven sin intervenir sobre él. Cada caso de uso se desarrolla en la secuencia de acciones de los requerimientos funcionales que siguen.

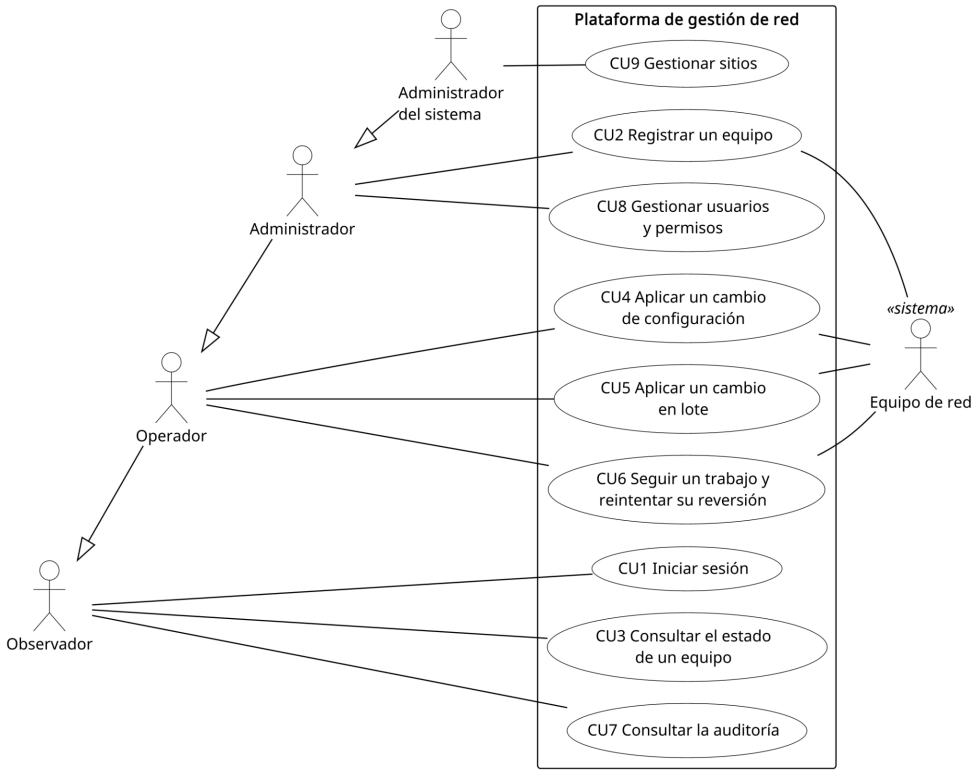


Figura 4.1: Diagrama de casos de uso del sistema.

Fuente: elaboración propia.

4.6.2. Requerimientos Funcionales

Los requerimientos funcionales se agrupan en catorce, numerados de RF1 a RF14. Cada uno se presenta con una descripción general, la secuencia de acciones del caso de uso cuando corresponda, y los requerimientos del sistema que lo componen, numerados de S1 a S56 para su trazabilidad en los capítulos siguientes.

RF1: Autenticación de Usuarios

Para acceder al sistema, el usuario debe autenticarse con su nombre de usuario y contraseña.

S1 El sistema debe autenticar a los usuarios mediante nombre de usuario y contraseña, emitiendo un *token* de acceso de vigencia corta y un *token* de renovación.

S2 El sistema debe bloquear temporalmente una cuenta tras reiterados intentos fallidos de inicio de sesión, y permitir que un administrador del sistema la desbloquee.

S3 El sistema debe permitir al usuario listar sus sesiones activas, cerrar la sesión actual y revocar las demás.

S4 El sistema debe solicitar nuevamente la contraseña antes de ejecutar operaciones administrativas sensibles: eliminación de sitios, desactivación de usuarios, revocación de permisos y cambios en la condición de administrador del sistema.

RF2: Gestión de Usuarios

El sistema debe permitir a un administrador gestionar los usuarios y sus permisos iniciales.

S5 El sistema debe permitir crear, consultar, modificar y desactivar usuarios.

S6 Las contraseñas deben almacenarse mediante una función de derivación de claves con sal, nunca en texto plano.

S7 Sólo un administrador del sistema puede otorgar o retirar a otro usuario la condición de administrador del sistema.

RF3: Gestión del Inventario de Dispositivos

El sistema debe permitir a un administrador registrar y administrar los equipos del inventario, cuyo estado queda disponible para su consulta. También debe permitir a un operador solicitar que un equipo guarde su configuración de forma persistente.

Secuencia de Acciones

1. El administrador accede al inventario y selecciona la opción de registrar un equipo.
2. Ingresa el nombre, la dirección de gestión, el fabricante, el método de autenticación y las credenciales.
3. El sistema almacena las credenciales cifradas e inicia la lectura del estado del equipo.
4. El equipo se muestra en el inventario con sus VLAN, puertos e interfaces virtuales.

S8 El sistema debe permitir registrar, consultar, modificar y eliminar equipos.

S9 Las credenciales de acceso a los equipos deben almacenarse cifradas y no deben devolverse en ninguna respuesta de la API.

S10 El sistema debe soportar autenticación SSH por contraseña y por par de claves.

S11 Al registrar un *switch*, el sistema debe leer y almacenar su estado, que queda disponible para consulta sin abrir una nueva sesión con el equipo.

S12 El sistema debe permitir al usuario solicitar la actualización manual del estado almacenado de un equipo.

S13 El sistema debe permitir guardar la configuración en ejecución como configuración de arranque.

RF4: Gestión de VLAN

El sistema debe permitir al operador consultar, crear y renombrar las VLAN de un equipo. La eliminación queda reservada al administrador. Cada modificación se ejecuta como un trabajo asíncrono.

S14 El sistema debe listar las VLAN de un equipo a partir de su estado almacenado.

S15 El sistema debe permitir crear, renombrar y eliminar VLAN, traduciendo cada operación a la sintaxis del fabricante.

S16 Al crear o renombrar una VLAN, el sistema debe comparar el estado deseado con el estado leído del *switch* y no aplicar cambios si ya coinciden.

RF5: Configuración de Puertos Físicos

El sistema debe permitir al operador consultar y modificar la configuración de los puertos de un equipo. Cada modificación se ejecuta como un trabajo asíncrono.

S17 El sistema debe listar los puertos de un equipo con su modo, VLAN, descripción, estado administrativo y operativo.

S18 El sistema debe permitir configurar un puerto en modo acceso, asignando su VLAN, o en modo troncal, definiendo su VLAN nativa y sus VLAN permitidas.

S19 El sistema debe permitir agregar o quitar VLAN permitidas en un puerto troncal sin redefinir la lista completa.

S20 El sistema debe permitir modificar la descripción, el estado administrativo, la alimentación *PoE* y los umbrales de *storm-control* de un puerto, así como restablecer su configuración.

S21 La interfaz debe advertir al usuario cuando una modificación afecte a un puerto que transporta la VLAN de gestión del equipo.

RF6: Configuración de Interfaces Virtuales

El sistema debe permitir al operador crear, eliminar y configurar las interfaces virtuales de un equipo. Cada modificación se ejecuta como un trabajo asíncrono.

S22 El sistema debe permitir crear y eliminar interfaces virtuales asociadas a una VLAN.

S23 El sistema debe permitir asignar y quitar una dirección IPv4 primaria, direcciones IPv4 secundarias y una dirección IPv6.

S24 El sistema debe permitir asignar y quitar ACLs en sentido entrante y saliente.

S25 El sistema debe permitir agregar y quitar servidores de reenvío DHCP, y modificar la descripción y el estado administrativo de la interfaz.

RF7: Configuración Global del Dispositivo

El sistema debe permitir a un administrador modificar los parámetros globales de un equipo. Su consulta, junto con información de diagnóstico, debe estar disponible para cualquier usuario con acceso al equipo. Cada modificación se ejecuta como un trabajo asíncrono.

S26 El sistema debe permitir modificar el nombre del equipo, las rutas estáticas, los servidores NTP, DNS y syslog, la configuración SNMP y las listas de control de acceso.

S27 El sistema debe permitir consultar la versión de *firmware*, la configuración en ejecución, las tablas ARP y MAC, y el registro local del equipo.

S28 El sistema debe permitir buscar una dirección MAC en la tabla del equipo.

RF8: Ejecución Asíncrona y Seguimiento de Trabajos

Toda operación de escritura debe ejecutarse como un trabajo asíncrono, de modo que el usuario no quede bloqueado mientras el cambio se aplica. El sistema debe permitir al usuario consultar el estado y el resultado de cada trabajo, cancelar un trabajo pendiente y reintentar un rollback que no se haya completado.

Secuencia de Acciones

1. El usuario solicita una operación de escritura.
2. El sistema crea un trabajo en estado pendiente y responde inmediatamente con su identificador.
3. El trabajo pasa a ejecución: se lee el estado previo del recurso y se aplica el cambio.
4. Si la aplicación falla por una causa transitoria, se reintenta. Si no puede completarse, se restaura el estado previo.
5. El trabajo finaliza como completado, fallido o cancelado, y el usuario visualiza su resultado.

S29 El sistema debe validar los parámetros y la existencia de los recursos requeridos antes de crear el trabajo. Si un recurso sólo puede verificarse durante la ejecución, el trabajo debe fallar indicando cuál falta.

S30 Toda operación de escritura debe responder con el identificador del trabajo creado, sin esperar su ejecución sobre el equipo.

S31 Los trabajos deben seguir el ciclo de estados: pendiente, en ejecución y uno de los estados finales (completado, fallido o cancelado).

S32 Antes de aplicar un cambio, el sistema debe leer y conservar el estado previo del recurso.

S33 El sistema debe categorizar los errores detectados para determinar su estrategia de recuperación: **Transitorios**, **Desconocidos** y **Permanentes**.

S34 Si el cambio no puede completarse, el sistema debe restaurar el estado previo del recurso y registrar el resultado del *rollback*. Este se omite cuando el cambio no llegó a aplicarse.

S35 Tras completar una operación de escritura, el sistema debe actualizar el estado almacenado del recurso modificado.

S36 El sistema debe permitir al usuario cancelar un trabajo pendiente y reintentar manualmente

un rollback fallido de un cambio sobre puertos o interfaces virtuales.

RF9: Registro de Auditoría

Cada operación que modifica un equipo o la configuración del sistema debe quedar registrada. El sistema debe permitir a cada usuario consultar el registro dentro de su alcance y filtrarlo [39].

S37 El sistema debe registrar cada operación con su usuario, recurso, operación, resultado y fecha.

S38 El sistema debe permitir filtrar el registro por usuario, operación, recurso, resultado, equipo, sitio y rango de fechas.

S39 Las entradas del registro deben ser inmutables una vez escritas.

S40 Cada usuario debe ver sólo la auditoría de su alcance y sus propios eventos de autenticación.

S41 Sólo un administrador del sistema puede depurar el registro, eliminando las entradas anteriores al período de retención indicado.

RF10: Interfaz Web de Operación

El sistema debe ofrecer una interfaz web desde la cual los usuarios accedan a las funciones disponibles según su rol, sin necesidad de utilizar la línea de comandos de los equipos.

S42 La interfaz debe permitir realizar las operaciones de consulta y configuración ofrecidas por el sistema.

S43 La interfaz debe ocultar o deshabilitar las acciones que excedan el rol efectivo del usuario sobre el recurso.

S44 La interfaz debe notificar al usuario la finalización de los trabajos que lanzó.

RF11: Organización Multisitio y Permisos por Alcance

Los equipos deben organizarse en una jerarquía de sitios y grupos. Cada permiso de un usuario debe asignarse sobre un sitio completo o sobre un grupo de equipos.

Secuencia de Acciones

1. El administrador del sistema crea un sitio.
2. Un administrador del sitio crea grupos dentro de él y asigna equipos a cada grupo.
3. El administrador asigna a un usuario un rol sobre el sitio o sobre uno de sus grupos.
4. El usuario, al iniciar sesión, visualiza y opera únicamente los recursos dentro de su alcance.

S45 El sistema debe permitir crear, modificar y eliminar sitios, y crear y eliminar grupos de equipos dentro de un sitio.

S46 El sistema debe permitir mover un equipo entre grupos, incluso de sitios distintos, verificando los permisos del usuario sobre el origen y el destino.

S47 El sistema debe permitir asignar y revocar los roles de observador, operador o administrador sobre un sitio o sobre un grupo. Sólo un administrador con alcance sobre el sitio completo puede asignar permisos a otros usuarios.

S48 El rol efectivo de un usuario sobre un grupo debe ser el mayor entre el asignado sobre el sitio y el asignado sobre el grupo.

S49 Las consultas de equipos, grupos y sitios fuera del alcance del usuario deben responder como inexistentes, y las operaciones sobre ellos deben rechazarse.

RF12: Operaciones en Lote

El sistema debe permitir al operador agrupar en una sola operación, denominada lote, varios cambios sobre un mismo equipo. También debe permitirle aplicar una misma operación sobre un conjunto de equipos de un grupo o de un sitio, con seguimiento del resultado individual de cada equipo.

Secuencia de Acciones

1. El operador accede a la vista de un grupo o de un sitio.
2. Selecciona la operación a aplicar, por ejemplo, crear una VLAN.
3. El sistema crea un trabajo por equipo, agrupados bajo una misma operación de grupo, y responde con su identificador.
4. Los trabajos se ejecutan respetando los límites de concurrencia, y el operador visualiza el resultado de cada equipo.

S50 El sistema debe permitir aplicar operaciones sobre VLAN, puertos, interfaces virtuales y parámetros de configuración global (servidores NTP, DNS y syslog, y SNMP) a un conjunto de equipos de un grupo o sitio.

S51 Los cambios de un lote sobre las VLAN, los puertos o las interfaces virtuales de un mismo equipo deben aplicarse en un único trabajo, con una sola sesión de aplicación. Una operación sobre varios equipos debe generar un trabajo independiente por equipo, de modo que la falla en uno no interrumpa ni revierta los demás.

S52 El sistema debe presentar el resultado agregado de la operación y el detalle por equipo.

RF13: Sincronización del Estado

El sistema debe mantener actualizado el estado almacenado de los equipos que se consultan, sin abrir sesiones con los que nadie consulta.

Secuencia de Acciones

1. El usuario abre la vista de un sitio, un grupo o un equipo.
2. El sistema selecciona los equipos de esa vista cuyo estado almacenado supera la antigüedad máxima.
3. Lee el estado de esos equipos y actualiza la información almacenada, mientras la vista muestra el estado anterior.

S53 Al abrir la vista de un sitio, un grupo o un equipo, el sistema debe actualizar el estado almacenado de los equipos cuya información supere una antigüedad configurable, de 15 minutos por defecto.

S54 El sistema debe admitir una actualización periódica de todos los equipos, deshabilitada por defecto y distribuida en el tiempo para no abrir simultáneamente sesiones con todos ellos.

S55 Si ya hay una sincronización en espera para el mismo equipo, una nueva solicitud debe unirse a ella en lugar de generar otra.

RF14: Tablero de Estado

El sistema debe presentar un resumen del estado de los equipos dentro del alcance del usuario.

S56 El sistema debe presentar, para la organización, un sitio, un grupo o un equipo, los totales de equipos, VLAN, puertos, interfaces virtuales y trabajos.

El Cuadro 4.2 relaciona cada caso de uso de la Figura 4.1 con los requerimientos que lo implementan.

Caso de uso	Descripción	Actor principal	Requerimientos
CU1	Iniciar sesión	Todos	RF1
CU2	Registrar un equipo	Administrador	RF3
CU3	Consultar el estado de un equipo	Observador	RF3, RF4, RF5, RF6, RF7
CU4	Aplicar un cambio de configuración	Operador (administrador en la configuración global)	RF4, RF5, RF6, RF7, RF8
CU5	Aplicar un cambio en lote	Operador	RF12
CU6	Seguir un trabajo y reintentar su reversión	Operador	RF8
CU7	Consultar la auditoría	Observador	RF9
CU8	Gestionar usuarios y permisos	Administrador	RF2, RF11
CU9	Gestionar sitios	Administrador del sistema	RF11

Cuadro 4.2: Casos de uso principales y requerimientos que los implementan.

4.6.3. Requerimientos No Funcionales

Los requerimientos no funcionales se clasificaron según las características de calidad de producto de ISO/IEC 25010:2023 [36]. A continuación se enumeran agrupados en seguridad, confiabilidad, eficiencia del desempeño, mantenibilidad y usabilidad.

Requerimientos de Seguridad

RNF1: Protección del Acceso

El sistema debe proteger el acceso a la plataforma y a las credenciales de los equipos, dado que concentra la capacidad de modificar la configuración de toda la red [38] [39].

SN1 El *token* de acceso debe tener una vigencia de 20 minutos. La sesión debe cerrarse tras 15 minutos de inactividad y, en cualquier caso, a las 2 horas de iniciada.

SN2 Una cuenta debe bloquearse durante 15 minutos tras 5 intentos fallidos de inicio de sesión, y una dirección IP durante 1 hora tras 20 intentos fallidos.

SN3 El sistema debe limitar, por minuto, los intentos de inicio de sesión a 5 por dirección IP, las solicitudes autenticadas a 200 por usuario y las no autenticadas a 20 por dirección IP.

SN4 La re-autenticación exigida por las operaciones sensibles debe tener una validez máxima de 5 minutos.

SN5 Las contraseñas de los usuarios deben almacenarse mediante PBKDF2-SHA256, y las credenciales de los equipos cifradas con Fernet (AES-128-CBC con autenticación HMAC-SHA256).

SN6 La comunicación entre los usuarios y el sistema debe cifrarse con TLS.

RNF2: Aislamiento entre Alcances

La restricción de visibilidad por alcance no debe depender únicamente de que cada consulta de la aplicación aplique el filtro correcto [40].

SN7 El aislamiento debe reforzarse en el motor de base de datos mediante políticas de seguridad a nivel de fila sobre las tablas de sitios, grupos, equipos y auditoría, que por defecto rechacen el acceso a filas fuera del alcance del usuario.

SN8 Los procesos en segundo plano, que no actúan en nombre de un usuario, deben identificarse ante la base de datos como procesos del sistema, para que las políticas de aislamiento no les oculten los datos que necesitan.

Requerimientos de Confiabilidad

RNF3: Tolerancia a Fallas en la Ejecución

La aplicación de un cambio debe tolerar las fallas propias de la comunicación con los equipos, sin dejar recursos en estados intermedios ni saturar su capacidad de sesiones.

SN9 Sobre un mismo equipo sólo puede ejecutarse una operación de escritura a la vez. El bloqueo debe liberarse automáticamente si el proceso que lo retiene deja de responder.

SN10 El sistema no debe ejecutar más de 30 trabajos por minuto sobre un mismo equipo. Los que excedan el límite deben esperar su turno.

SN11 La cantidad de sesiones SSH simultáneas abiertas por la plataforma no debe superar un límite global configurable, de 20 por defecto.

SN12 El *rollback* de un cambio debe intentarse con un presupuesto de reintentos acotado, para no retener el bloqueo del equipo indefinidamente.

RNF4: Trazabilidad

El sistema debe permitir reconstruir a posteriori qué se solicitó en cada operación, cómo se ejecutó y cuál fue su resultado.

SN13 Cada trabajo debe conservar los parámetros de la operación, el estado previo leído, el error devuelto por el equipo y el resultado del *rollback*, si lo hubo.

SN14 Todo intento de modificar una entrada de auditoría existente debe ser rechazado por la capa de persistencia.

SN15 El período de retención por defecto del registro de auditoría debe ser de 90 días.

Requerimientos de Eficiencia

RNF5: Uso de Sesiones y Tiempo de Respuesta

El sistema debe minimizar las conexiones a los equipos y responder al usuario sin esperar la ejecución de los cambios.

SN16 Las operaciones de consultas deben resolverse a partir del estado almacenado, sin abrir sesiones con los equipos, e indicar el momento de su última actualización. La interfaz no debe repetirlas periódicamente mientras no esté visible para el usuario.

SN17 La lectura completa del estado de un equipo debe realizarse con a lo sumo dos sesiones SSH, sin abrir una por cada tipo de recurso.

SN18 Las operaciones de cambio deben responder al ser aceptadas, sin esperar a que se ejecuten sobre los equipos.

Requerimientos de Mantenibilidad

RNF6: Extensibilidad y Verificabilidad

El sistema debe admitir la incorporación de nuevos fabricantes, y su verificación no debe depender de equipos reales.

SN19 El sistema debe permitir incorporar un nuevo fabricante de equipos sin modificar las funcionalidades existentes ni afectar la operación sobre los fabricantes ya soportados.

SN20 El sistema debe contar con un modo de ejecución simulado que permita ejecutar la suite de pruebas sin equipos.

SN21 La API debe documentarse automáticamente según la especificación OpenAPI.

Requerimientos de Usabilidad

RNF7: Operación sin Conocimiento de la CLI

El sistema debe permitir que un usuario sin conocimiento de la sintaxis de cada fabricante realice las operaciones habituales.

SN22 Ninguna operación de la interfaz debe requerir que el usuario ingrese comandos de la CLI del equipo.

SN23 Cada trabajo debe presentarse con una descripción legible de la operación que representa y, en caso de falla, con el error devuelto por el equipo.

4.6.4. Análisis de Riesgo y Priorización

Un requerimiento es riesgoso en la medida en que su implementación puede presentar dificultades o incertidumbres que pongan en peligro los objetivos del proyecto. Para estimarlo se valoran los siguientes criterios entre 0 y 1, donde 1 indica la mayor probabilidad de que surjan problemas:

1. **Recursos Disponibles**: cuánto le faltan al equipo las habilidades, el tiempo o las herramientas que exige.
2. **Complejidad de Implementación**: la dificultad técnica de construirlo.
3. **Dependencias Externas**: cuánto depende de sistemas ajenos al proyecto. En este dominio, la principal es el propio equipamiento de red, cuyo comportamiento varía entre fabricantes, modelos y versiones de firmware.
4. **Integración**: cuántos componentes del sistema involucra y qué tan acoplado está a ellos.

La probabilidad total se obtiene como el promedio de los cuatro criterios. El impacto se valora en una escala de 1 (insignificante) a 5 (severo), según el efecto que tendría sobre los objetivos del proyecto no cumplir el requerimiento. El nivel de riesgo resulta de cruzar ambos valores en la matriz del Cuadro 4.3.

Probabilidad/Impacto	**Insignificante (1)**	**Menor (2)**	**Significante (3)**	**Mayor (4)**	**Severo (5)**
Muy Probable (0.81 - 1.0)	Medio	Alto	Muy Alto	Extremo	Extremo
Probable (0.61 - 0.8)	Bajo	Medio	Alto	Muy Alto	Extremo
Moderado (0.41 - 0.6)	Muy Bajo	Bajo	Medio	Alto	Muy Alto
Improbable (0.21 - 0.4)	Despreciable	Muy Bajo	Bajo	Medio	Alto
Raro (0.0 - 0.2)	Despreciable	Despreciable	Muy Bajo	Bajo	Medio

Cuadro 4.3: Matriz de Riesgos

Los Cuadros 4.4 y 4.5 presentan los requerimientos, funcionales y no funcionales respectivamente, ordenados según su criticidad, con los valores de su análisis de riesgo: recursos disponibles (Rec.), complejidad de implementación (Compl.), dependencias externas

(Dep.), integración (Integr.), probabilidad total (Prob.) e impacto (Imp.). Se indica además el factor que determina el nivel de riesgo y la iteración (Iter.) en la que se implementa cada requerimiento.

Requerimiento Funcional	Rec.	Compl.	Dep.	Integr.	Prob.	Imp.	Nivel	Factor determinante	Iter.
RF3: Gestión del Inventario de Dispositivos	0.3	0.4	0.6	0.4	0.425	5	Muy alto	Dependencia del equipamiento	1
RF4: Gestión de VLAN	0.3	0.5	0.7	0.5	0.5	5	Muy alto	Dependencia del equipamiento	1
RF8: Ejecución Asíncrona y Seguimiento de Trabajos	0.3	0.8	0.4	0.7	0.55	5	Muy alto	Complejidad e integración	1
RF1: Autenticación de Usuarios	0.2	0.4	0.1	0.3	0.25	5	Alto	Rotación y revocación de tokens	1
RF5: Configuración de Puertos Físicos	0.3	0.6	0.8	0.4	0.525	4	Alto	Dependencia del equipamiento	2
RF6: Configuración de Interfaces Virtuales	0.3	0.6	0.7	0.4	0.5	4	Alto	Dependencia del equipamiento	2
RF11: Organización Multi-sitio y Permisos por Alcance	0.4	0.7	0.2	0.8	0.525	4	Alto	Integración con la autorización	3
RF7: Configuración Global del Dispositivo	0.4	0.5	0.6	0.3	0.45	3	Medio	Cantidad de subrecursos	2
RF9: Registro de Auditoría	0.2	0.3	0.1	0.4	0.25	4	Medio	Integración con el orquestador	1
RF12: Operaciones en Lote	0.4	0.6	0.5	0.6	0.525	3	Medio	Saturación de sesiones SSH	3
RF10: Interfaz Web de Operación	0.5	0.4	0.3	0.3	0.375	3	Bajo	Recursos disponibles	1
RF13: Sincronización Programada del Estado	0.3	0.4	0.5	0.4	0.4	3	Bajo	Disponibilidad de los equipos	3
RF2: Gestión de Usuarios	0.2	0.2	0.1	0.2	0.175	3	Muy bajo	Baja complejidad	1
RF14: Tablero de Estado	0.4	0.3	0.1	0.3	0.275	2	Muy bajo	Bajo impacto	3

Cuadro 4.4: Análisis de riesgo y priorización de los requerimientos funcionales

Requerimiento No Funcional	Rec.	Compl.	Dep.	Integr.	Prob.	Imp.	Nivel	Factor determinante	Iter.
RNF3: Tolerancia a Fallas en la Ejecución	0.3	0.8	0.7	0.6	0.6	5	Muy alto	Complejidad y dependencia del	Todas
RNF1: Protección del Acceso	0.3	0.5	0.2	0.4	0.35	5	Alto	Impacto de un acceso indebido	Todas
RNF2: Aislamiento entre Alcances	0.4	0.7	0.3	0.7	0.525	4	Alto	Integración con consultas y procesos	3
RNF5: Uso de Sesiones y Tiempo de Respuesta	0.3	0.6	0.6	0.5	0.5	4	Alto	Dependencia del equipamiento	Todas
RNF4: Trazabilidad	0.2	0.4	0.1	0.5	0.3	4	Medio	Integración en el flujo de ejecución	Todas
RNF6: Extensibilidad y Verificabilidad	0.3	0.5	0.2	0.5	0.375	3	Bajo	Separación orquestador-controladores	Todas
RNF7: Operación sin Conocimiento de la CLI	0.5	0.4	0.2	0.3	0.35	2	Muy bajo	Bajo impacto	Todas

Cuadro 4.5: Análisis de riesgo y priorización de los requerimientos no funcionales

Los requerimientos se asignaron a las iteraciones según su criticidad y sus dependencias técnicas, de modo que cada uno se implementó recién cuando aquello sobre lo que se apoya ya estaba construido y validado. Por eso la gestión de usuarios, la auditoría y la interfaz básica integraron la primera iteración junto con la autenticación, pese a su menor criticidad, ya que todas las operaciones posteriores dependen de ellas. A la inversa, la organización multisitio, de criticidad alta, se postergó a la tercera, porque modifica la autorización de operaciones que debían existir antes. Dentro de cada iteración, la criticidad fijó el orden de implementación y el esfuerzo de prueba.

Los requerimientos no funcionales son transversales y se atienden desde la primera iteración, aunque algunos se completan en las siguientes, a medida que se incorporan las funcionalidades de las que dependen, como el aislamiento entre alcances junto con el modelo multisitio.

5. Arquitectura del Sistema

A partir de los requerimientos especificados en el capítulo anterior, en este capítulo se expone el diseño del sistema, de lo general a lo particular. Primero se lo ubica en su contexto y se presenta el modelo de diseño. Luego se describe cómo el sistema representa la configuración de un *switch* y cómo aplica un cambio. A continuación se presentan la arquitectura del sistema y el diseño de cada módulo. Por último, se detallan las tecnologías empleadas y el entorno de desarrollo.

5.1. Contexto del Sistema

La Figura 5.1 muestra las entidades externas con las que interactúa el sistema, según el primer nivel del modelo C4 [33], que representa al sistema como una única caja rodeada de las personas que lo usan y los sistemas con los que se comunica.

- **Personas:** los cuatro roles definidos en la sección 4.6.1 (Observador, Operador, Administrador y Administrador del sistema), que acceden al sistema desde un navegador web.
- **Infraestructura de red gestionada:** los *switches* Cisco IOS y Huawei VRP, que el sistema consulta y modifica mediante sesiones SSH sobre su línea de comandos, el único mecanismo de acceso común a todo el parque, sin instalar agentes.
- **Consumidores externos de la plataforma:** herramientas de la organización (como sistemas de monitoreo o scripts de automatización) que interactúan mediante la API REST documentada.



Figura 5.1: Contexto del sistema (nivel 1 del modelo C4).

Fuente: elaboración propia.

5.2. Modelo de Diseño

El sistema se desarrolla con orientación a objetos. Los elementos configurables de los *switches*, los fabricantes y los mecanismos externos se representan mediante clases que cumplen contratos comunes. Sobre esa base, el diseño adopta los principios de la arquitectura hexagonal [34], que aísla la lógica central del sistema, o núcleo, de los mecanismos externos.

El núcleo define las interfaces de las que depende. Los adaptadores de entrada, como la API REST, lo invocan, y los de salida, como el acceso a los equipos o la persistencia, son invocados por él. Así, incorporar un fabricante o cambiar el almacenamiento no altera la lógica del núcleo. La lógica que aplica los cambios de forma segura puede verificarse, además, en modo simulado.

5.3. El Núcleo: Recurso Gestionable y Ciclo de un Cambio

5.3.1. El Recurso Gestionable

El concepto central del núcleo es el recurso gestionable. En el modelo de solución adoptado, el operador solicita operaciones sobre VLAN, puertos, interfaces virtuales o la configuración global, una capa de controladores las traduce a cada fabricante y un orquestador las aplica con capacidad de *rollback*. El recurso gestionable es la pieza que une estos tres elementos. En este informe, un controlador es el componente que traduce las operaciones a la sintaxis de un fabricante.

Esta abstracción representa, de forma independiente del fabricante, la modificación deseada sobre un elemento configurable del *switch*. Todo recurso gestionable cumple un mismo contrato, que el orquestador ejecuta de manera polimórfica, sin conocer su tipo concreto. El contrato define cuatro responsabilidades principales.

- **Validarse:** comprobar las reglas que no dependen del equipo, como que el identificador de una VLAN se encuentre entre 1 y 4094 y no sea uno de los reservados, o que un puerto en modo acceso tenga una VLAN asignada.
- **Reconciliarse:** leer del equipo el estado actual del elemento.
- **Aplicarse:** determinar qué pasos llevan del estado actual al deseado y ejecutarlos a través del controlador del fabricante. Cada paso se expresa por su nombre, como "pasar un puerto a modo acceso", y no por la sintaxis de un fabricante. Si el estado actual ya coincide con el deseado, no hay pasos y la operación no genera cambios.
- **Hacer *rollback*:** restaurar el estado previo y verificar sobre el equipo que la restauración efectivamente se produjo.

El recurso no conoce la interfaz web, la cola de trabajos ni el protocolo SSH, sino sólo el contrato del controlador. El sistema cuenta con cinco tipos de recurso gestionable, los cuatro elementos configurables y el guardado de la configuración (Cuadro 5.1), cada uno con sus propias reglas, en particular las de *rollback*.

Recurso Gestionable	Categoria del Switch	Operaciones	Requerimiento
VLAN	Redes Logicas	Crear, Renombrar y Eliminar	RF4
Puerto	Puertos Fisicos	Modo access o trunk, VLAN asignadas, descripción, habilitación, PoE, storm-control y restablecimiento	RF5
Interfaz Virtual	Interfaces Virtuales (SVI)	Crear y eliminar, direccionamiento IPv4 e IPv6, ACL, DHCP relay, descripción y habilitación	RF6
Configuración Global	Configuración Global	Nombre del equipo, rutas estáticas, servidores NTP, DNS y de registro, SNMP y ACL	RF7
Guardado de Configuración	Configuración Global	Guardar la configuración en ejecución como configuración de arranque	RF3

Cuadro 5.1: Recursos gestionables, categoría del switch y requerimiento asociado.

5.3.2. Ciclo de un Cambio

Todo cambio de configuración se ejecuta como un trabajo, que recorre el ciclo representado en la Figura 5.2. Un trabajo cancelado mientras esperaba su turno no se ejecuta. Al comenzar, se toma el bloqueo del equipo, de modo que ninguna otra operación, sea un cambio o una sincronización, puede ejecutarse sobre él hasta que se libere.

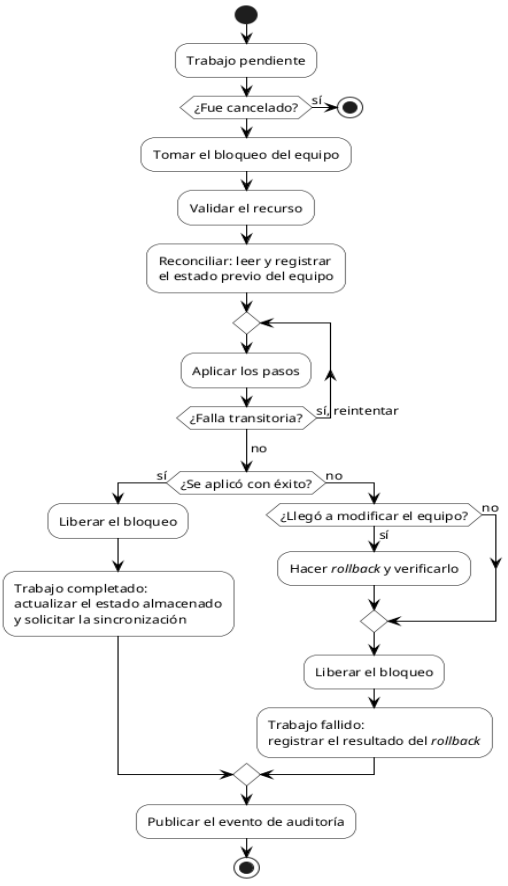


Figura 5.2: Ciclo de un cambio de configuración.

Fuente: elaboración propia.

A continuación el recurso se valida y se reconcilia, es decir, se lee del equipo su estado previo, reintentando ante fallas transitorias. Ese estado se registra en el trabajo, porque sin él no es posible hacer el *rollback*. Luego el recurso se aplica. Ante una falla, el error se clasifica para decidir si se reintenta. Si la aplicación falla definitivamente, el recurso hace el *rollback* y una nueva lectura del equipo lo verifica.

Por último se libera el bloqueo y se registra el resultado. Un trabajo exitoso se marca como completado, actualiza el estado almacenado y solicita la sincronización del equipo. Uno fallido indica si el *rollback* se realizó y si tuvo éxito. En ambos casos se publica un evento de auditoría.

El estado previo se lee del equipo en el momento del cambio y no del estado almacenado, de modo que el *rollback* restaura lo que el equipo efectivamente tenía, incluso si fue modificado por fuera de la plataforma.

5.4. Diseño de la Arquitectura

Esta sección ubica el núcleo descrito en la sección 5.3 dentro de las unidades que componen el sistema. Sigue los niveles de detalle creciente del modelo C4 [33], que son el contexto (sección 5.1), los contenedores, los módulos y el diseño de cada módulo (sección 5.5). Esta sección cubre los dos intermedios.

5.4.1. Contenedores

La Figura 5.3 muestra los contenedores del sistema, es decir, las unidades que se ejecutan de forma independiente.

- **Cliente web:** aplicación web a la que el usuario accede desde su navegador y que se comunica con la API mediante HTTP sobre TLS.
- **API:** recibe las solicitudes, autentica al usuario, verifica sus permisos, responde las consultas a partir del estado almacenado y encola las operaciones de escritura.
- **Workers:** procesos que toman los trabajos de la cola, aplican los cambios sobre los equipos y actualizan su estado almacenado. Un planificador encola además las tareas periódicas.
- **Base de datos relacional:** almacena las entidades de negocio, el estado de los equipos, los trabajos y el registro de auditoría.
- **Servicio en memoria:** aloja la cola de trabajos y la información que los workers comparten para no operar a la vez sobre un mismo equipo.

Separar la API de los workers permite que la primera responda de inmediato, aun cuando un *switch* tarda varios segundos en contestar, y que la cantidad de workers se ajuste de forma independiente.

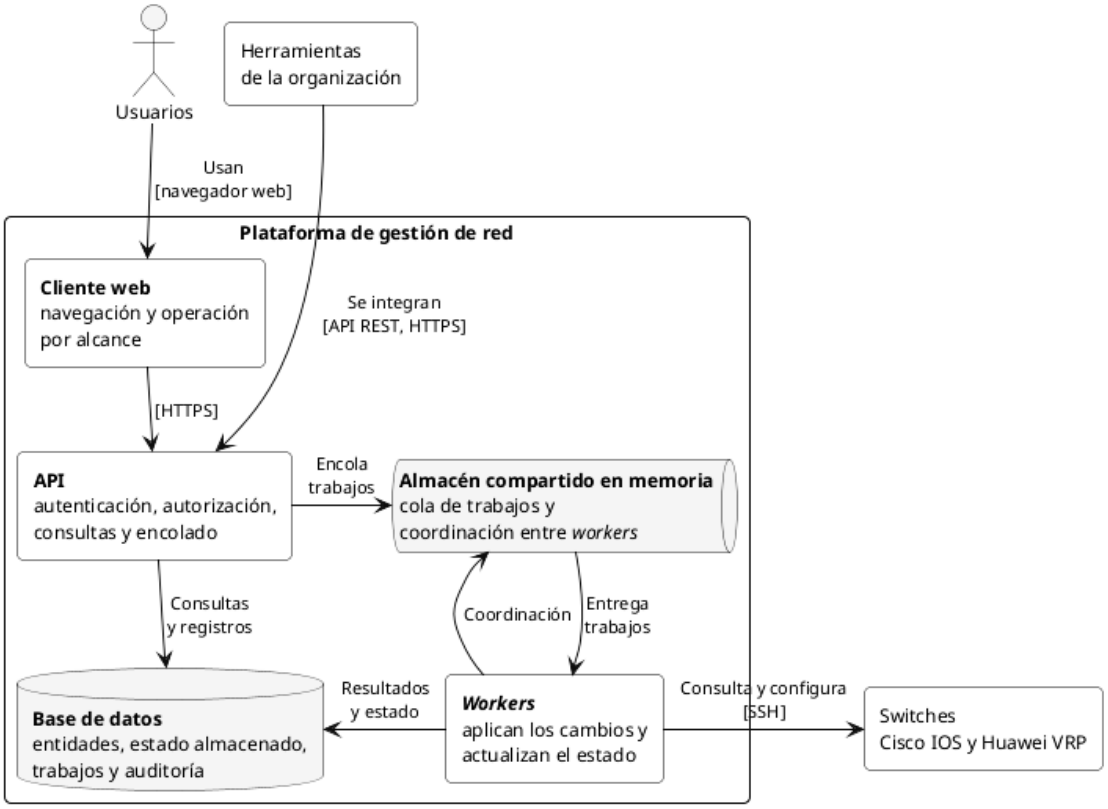


Figura 5.3: Contenedores del sistema (nivel 2 del modelo C4).

Fuente: elaboración propia.

5.4.2. Módulos

Un módulo agrupa los componentes del sistema dedicados a una misma responsabilidad, como el acceso a los equipos o el registro de auditoría. En el modelo C4, este nivel describe los módulos de cada contenedor por separado.

De los cinco contenedores, sólo la API y los *workers* se detallan en este nivel, porque la base de datos y el servicio en memoria son productos de terceros y el cliente web se trata en el diseño de interfaces. Como ambos procesos se inician a partir del mismo programa y comparten la mayoría de los módulos, la Figura 5.4 los presenta en un único diagrama.

Los módulos se organizan según el modelo hexagonal. Los adaptadores de entrada, la API REST y la Ejecución Asíncrona, invocan al núcleo, que reúne Dominio, Orquestación, Sincronización, y Organización y Acceso. Los adaptadores de salida, Acceso a Equipos, Persistencia, Coordinación y Auditoría, son invocados por él. Como casi todos los módulos

del núcleo recurren a los tres últimos, la Figura 5.4 resume esas dependencias en una única flecha desde el núcleo.

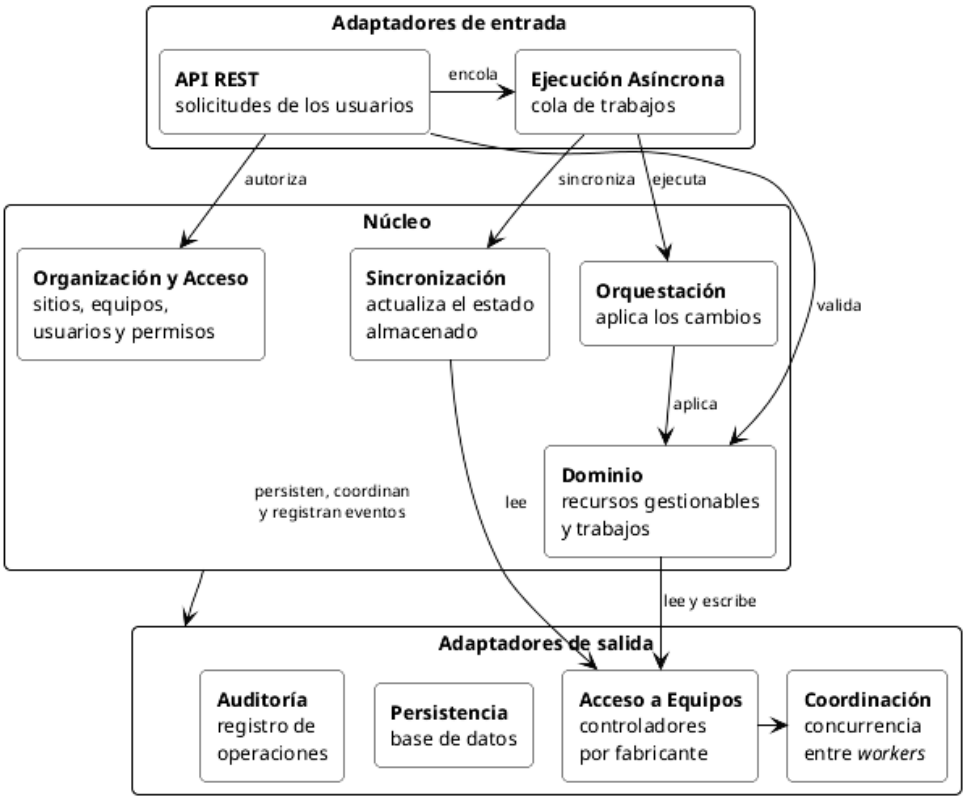


Figura 5.4: Módulos del sistema y sus dependencias (nivel 3 del modelo C4).

Fuente: elaboración propia.

5.5. Diseño Modular

En esta sección se describe cada módulo de la Figura 5.4, en el orden en que los atraviesa una solicitud de cambio. Se acompaña con diagramas de clases donde ayudan a entender la estructura. Cuando un módulo aplica un patrón de diseño, se explica también el problema que resuelve.

5.5.1. Módulo de API REST

La API traduce las solicitudes HTTP en operaciones del dominio. Cada solicitud es atendida por una ruta, que es una función asociada a un método HTTP y una URL. Es un adaptador de entrada, que no contiene las reglas del dominio, sino que recibe, valida, autoriza y delega. Toda solicitud atraviesa primero un componente común a todas las rutas, que identifica al usuario a partir de su *token*, fija su identidad para las políticas de la base de datos y aplica los límites de frecuencia de solicitudes, más estrictos para el inicio de sesión.

Las solicitudes de lectura autorizan el acceso al recurso y devuelven el dato ya almacenado. Las de escritura sobre la organización, como el alta de un sitio o de un usuario, se validan, autorizan y aplican en la misma solicitud, porque no dependen de ningún equipo. Las de configuración de red, en cambio, siguen tres etapas.

1. **Validación**
    a. **Validación de forma:** el cuerpo se valida contra un esquema declarado para la ruta, y una solicitud mal formada se rechaza sin llegar al dominio.
    b. **Validación del recurso:** la ruta construye el recurso gestionable y le pide que se valide, porque las reglas que combinan campos son reglas de negocio y pertenecen al recurso.
2. **Autorización:** la ruta verifica que el usuario tenga el rol mínimo que exige la operación sobre el equipo, el grupo o el sitio involucrado.
3. **Encolado:** el recurso se entrega a la ejecución asíncrona, que crea un trabajo por cada equipo involucrado. La API responde de inmediato, sin esperar a que el cambio se aplique.

5.5.2. Módulo de Organización y Acceso

Este módulo define sobre qué se opera y quién puede hacerlo. Reúne la organización de los equipos, la autorización y la identidad de los usuarios.

Los equipos se organizan en una jerarquía estricta de sitios y grupos. Cada equipo pertenece a un único grupo y cada grupo a un único sitio. Cada sitio se crea con un grupo por defecto, de modo que ningún equipo queda fuera de la jerarquía. Existe además un sitio de infraestructura base, único y no eliminable, visible sólo para el administrador del sistema, que reúne los equipos que no pertenecen a ningún otro sitio (Figura 5.5).

Un servicio de inventario concentra cinco operaciones sobre los equipos, que son consultarlos, listarlos, registrarlos, trasladarlos y darlos de baja. Cada una de ellas coordina varios componentes, ya que registrar un equipo implica validar su ubicación, cifrar sus credenciales, almacenarlo, publicar el evento de auditoría y solicitar su primera sincronización.

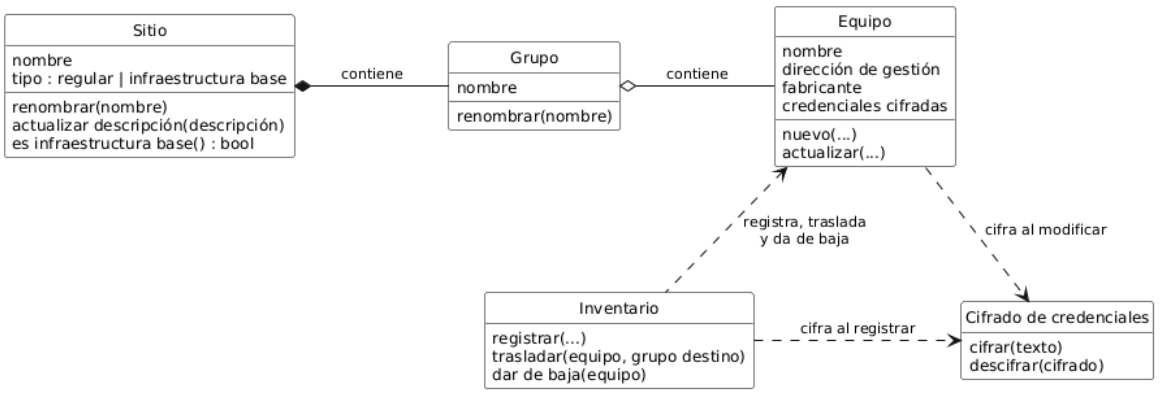


Figura 5.5: Clases del módulo de organización.

Fuente: elaboración propia.

La autorización sigue un modelo de control de acceso basado en roles [40], asignado por sitio o por grupo (Figura 5.6). Un usuario puede tener el rol de observador, operador o administrador en cada uno. El administrador del sistema es la excepción, porque no depende de ninguna asignación y tiene acceso a toda la plataforma.

Al empezar cada solicitud, el sistema reúne una sola vez los sitios y grupos en los que el usuario tiene un rol, y los usa en todas las verificaciones de esa solicitud sin volver a consultar la base.

Cada operación exige un rol mínimo sobre el equipo, el grupo o el sitio que afecta. Modificar un puerto, por ejemplo, exige el rol de operador sobre ese equipo, y si el usuario tiene un rol menor o ninguno, la operación se rechaza. Si tiene un rol sobre el sitio y otro sobre uno de sus grupos, vale el más alto.

En las lecturas puntuales, como buscar un equipo por su nombre, un usuario sin ningún permiso sobre él recibe la misma respuesta que si no existiera, para no confirmarle su existencia. El resto de las operaciones no necesita ocultarlo, porque para modificar un equipo primero hay que encontrarlo.

Las operaciones sobre dos alcances exigen permiso en ambos. Trasladar un equipo dentro de un sitio requiere operador sobre el grupo de origen y el de destino, y entre sitios requiere administrador sobre los dos. La delegación de permisos queda reservada a los administradores de un sitio completo, para concentrarla en un punto fácil de auditar.

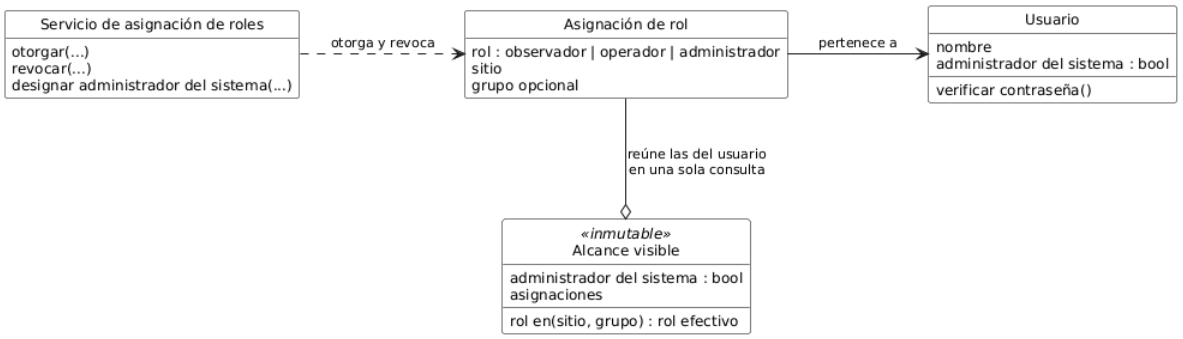


Figura 5.6: Clases del módulo de autorización.

Fuente: elaboración propia.

La identidad se resuelve mediante dos *tokens*. El de acceso es firmado, de corta duración y acompaña cada solicitud. El de renovación es un valor aleatorio, guardado sólo como resumen criptográfico y reemplazado en cada uso. Si uno ya usado vuelve a presentarse, se interpreta como robo y se revocan todas las sesiones del usuario. Las sesiones vencen tras un período de inactividad o, aunque sigan en uso, al cumplir una duración máxima.

Las operaciones irreversibles, como eliminar un sitio, exigen reingresar la contraseña, que emite un *token* elevado de corta duración. Los intentos fallidos de inicio de sesión bloquean temporalmente la cuenta y la dirección IP. Las contraseñas se almacenan con una función de derivación de claves con sal, que hace que dos contraseñas iguales se guarden de forma distinta [37].

5.5.3. Módulo de Ejecución Asíncrona

Este módulo desacopla la recepción de una solicitud de su ejecución sobre el equipo. La API crea el trabajo en estado pendiente, lo coloca en la cola y devuelve su identificador. Un worker toma el trabajo, reconstruye el recurso gestionable y lo entrega al orquestador (sección 5.5.5). Mientras tanto, el cliente web sigue el trabajo y notifica al usuario cuando finaliza.

El desacople responde a que la duración de un cambio no es predecible, porque el trabajo puede esperar a que el equipo termine otra operación o reintentar ante una falla de conexión. Con la cola, el usuario sigue operando mientras el cambio se aplica. La misma cola ejecuta las sincronizaciones del estado de los equipos, que no se registran como trabajos.

El ciclo de vida de un trabajo es una máquina de estados (Figura 5.7). Un trabajo pendiente puede pasar a en ejecución o a cancelado, y uno en ejecución, a completado o a

fallido, que son estados finales. El propio trabajo rechaza cualquier otra transición, de modo que su consistencia no depende de quién lo modifique. Un trabajo en ejecución no puede cancelarse, porque una interrupción podría dejar el equipo a mitad de un cambio.

La cancelación y el inicio de la ejecución compiten por el mismo trabajo. Por eso ambas se implementan como una actualización condicional en la base de datos, que modifica el trabajo sólo si sigue pendiente, de modo que únicamente una se concreta. Si gana el worker, la cancelación responde que el trabajo ya comenzó. Si gana la cancelación, el worker lo descarta sin tocar el equipo.

Una operación sobre varios equipos, como crear una VLAN en todo un grupo, genera un trabajo independiente para cada uno, agrupados bajo un mismo identificador de operación. Así, la falla en uno no interrumpe ni revierte a los demás, que pueden ejecutarse en paralelo en distintos workers. El resultado se presenta como un resumen agregado con el detalle de cada equipo, de modo que el usuario identifica cuáles fallaron sin revisarlos uno por uno.

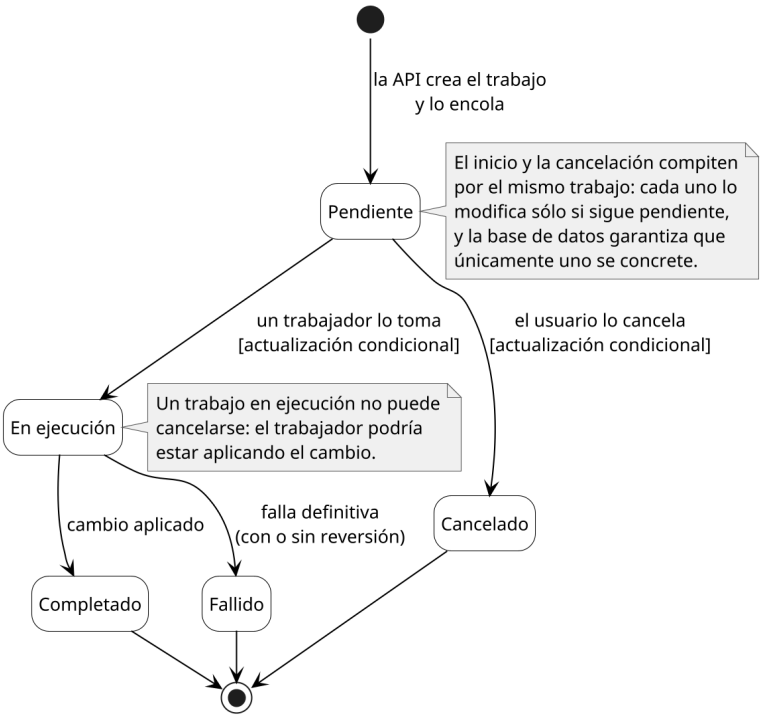


Figura 5.7: Estados de un trabajo.

Fuente: elaboración propia.

Varias modificaciones sobre un mismo equipo, como la edición de varios puertos, forman en cambio un único trabajo, denominado lote, que el orquestador aplica en una sola

sesión y, ante una falla, revierte en conjunto. Abrir una sesión por puerto multiplicaría las conexiones al equipo y su consumo de procesador.

Un *rollback* también puede fallar, por ejemplo por una pérdida de conexión, y dejar el equipo en un estado intermedio. Para no reconstruir a mano la configuración anterior, el operador puede reintentarlo, con lo que el sistema la restaura a partir del estado previo registrado en el trabajo original. El reintento crea un trabajo nuevo vinculado al original, en lugar de reabrirlo, para conservar intacto el registro de la falla. No se acepta un segundo reintento mientras otro está en curso, para que un doble clic no abra sesiones repetidas.

El reintento se ofrece para puertos e interfaces virtuales, cuyo *rollback* restaura atributos de un elemento existente. En las VLAN y en la configuración global implica crear o eliminar elementos, y queda fuera del alcance actual.

5.5.4. Módulo de Coordinación

La coordinación protege a los equipos y al propio servidor de los efectos de la concurrencia mediante cuatro mecanismos, que son el bloqueo por equipo, el límite de frecuencia por equipo, el límite global de ejecuciones y la unión de sincronizaciones. Como varios workers operan en paralelo, su estado se guarda en un servicio en memoria compartido por todos, que ejecuta de forma atómica las operaciones que no pueden quedar a medias, como tomar un bloqueo o reservar una sincronización, de modo que dos workers nunca obtienen lo mismo a la vez.

El bloqueo abarca el equipo completo y no cada recurso por separado, aunque crear una VLAN y editar un puerto parezcan operaciones independientes. Ambas modifican la misma configuración a través de un número limitado de sesiones, y el estado previo que se lee para el *rollback* sólo es válido si nadie toca la configuración mientras tanto. Un único bloqueo para todo el sistema, en el otro extremo, impediría trabajar en paralelo sobre equipos distintos.

Si un worker se detiene sin liberar el bloqueo, por ejemplo porque su proceso se reinicia, el equipo no debe quedar bloqueado hasta que alguien intervenga. Por eso el bloqueo tiene un vencimiento (*lease*) que el worker renueva mientras sigue activo. Si deja de renovarlo, el equipo queda libre en menos de un minuto.

Un trabajo que encuentra el equipo ocupado espera su turno, hasta 10 minutos, en lugar de rechazarse, porque el usuario ya recibió la confirmación de que su cambio fue

aceptado. Las sincronizaciones esperan menos, porque una lectura que no se concreta se repite en la próxima consulta.

No se ejecutan más de 30 trabajos de cambio por minuto sobre un mismo equipo. Los que exceden el límite esperan su turno. En la operación habitual no se alcanza, porque el bloqueo ya ejecuta los trabajos de a uno. Es una segunda barrera para los trabajos que fallan casi de inmediato, como los de credenciales incorrectas que un script podría repetir sin pausa, obligando al *switch* a negociar una sesión cifrada en cada intento.

Las ejecuciones sobre los equipos consumen mucha memoria en el servidor, por lo que demasiadas a la vez podrían saturarlo. Cada ejecución de Ansible ocupa alrededor de 125 MB, de modo que el máximo de 20 sesiones simultáneas en todo el sistema equivale a unos 2,5 GB. Las que excedan ese número esperan su turno. Si un worker falla sin liberar su lugar, ese lugar vuelve a quedar disponible a los 10 minutos.

Cuando varios usuarios abren la misma vista, o uno pide refrescar varias veces seguidas, cada pedido generaría una lectura del equipo. Para evitarlo, cada solicitud reserva de forma atómica el lugar de la sincronización pendiente de ese equipo y ese tipo de información. Si ya había una esperando, se une a ella. Si ya comenzó, en cambio, se encola una nueva, porque una lectura en curso podría no incluir los cambios posteriores a su inicio.

5.5.5. Módulo de Orquestación

La orquestación ejecuta el ciclo de un cambio descrito en la sección 5.3. Es el centro del núcleo, porque concentra la política de confiabilidad del sistema, que define cuándo se reintenta una falla, cómo se hace el *rollback* y cómo se registra el resultado.

El ciclo de un cambio tiene una secuencia fija, pero algunos pasos dependen del tipo de recurso. Leer un puerto y crear una VLAN requieren comandos y reglas distintas, mientras que tomar el bloqueo, reintentar o registrar el resultado es igual para todos. El patrón Template Method [41] responde a ese problema fijando el esqueleto de un algoritmo y delegando los pasos variables. En su forma clásica los delega en subclases, lo que obligaría a un orquestador por tipo de recurso. Aquí se aplica por composición, con el esqueleto en el orquestador y los pasos variables en el recurso gestionable, del que el orquestador sólo conoce su contrato.

El orquestador ejecuta los pasos comunes y delega en el recurso las cuatro responsabilidades de su contrato (Figura 5.8). Como la política de confiabilidad queda en el

esqueleto, un tipo de recurso nuevo la recibe completa sin modificarla ni repetirla. El contrato permite además resolver un paso sin ejecutarlo, lo que se usa para aplicar todos los pasos de un lote en una sola sesión.

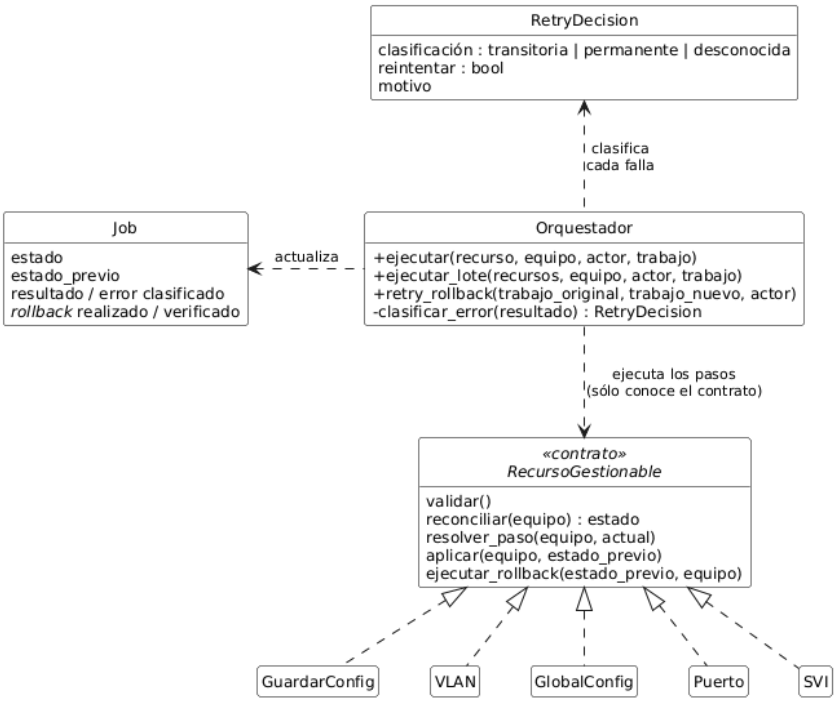


Figura 5.8: Clases del módulo de orquestación.
Fuente: elaboración propia.

Parte de esa política es decidir qué fallas se reintentan. Los equipos no devuelven códigos de error estandarizados, sino que informan cada falla como un texto en la salida de la conexión SSH. El orquestador la clasifica como transitoria, permanente o desconocida comparando este texto con una tabla de patrones agrupados por categoría (Cuadro 5.2). La categoría decide si se reintenta.

Reintentar toda falla sería más simple, pero repetir un comando rechazado sólo suma sesiones y retiene el bloqueo. Por eso los patrones permanentes se evalúan antes que los transitorios, de modo que un mensaje con señales de ambos tipos no se reintenta.

Las fallas transitorias se reintentan hasta tres veces, con esperas crecientes de 1, 2 y 4 segundos que dan tiempo a la red o al equipo para recuperarse. Las desconocidas reciben un único reintento, porque muchas fallas de red llegan con mensajes poco descriptivos y reintentar una vez es menos costoso que fallar por prudencia.

Clasificación	Categoría	Ejemplo de mensaje	Tratamiento
Permanente	Sintaxis	*Invalid input detected*, *Unrecognized command*	No se reintenta
Permanente	Conflicto	*overlaps with*, *already in use*	No se reintenta
Permanente	Credenciales	*Authentication failed*, *Permission denied*	No se reintenta
Permanente	Objeto inexistente	*The VLAN does not exist*	No se reintenta
Transitoria	Conectividad	*Connection timed out*, *closed by remote host*	Reintento con espera creciente
Transitoria	Límite de sesiones	*Channel closed*, *too many sessions*	Reintento con espera creciente
Desconocida	—	Mensaje sin coincidencia	Un único reintento

Cuadro 5.2: Clasificación de los errores de los equipos.

La clasificación se registra en el trabajo con un resumen legible del motivo, como que el equipo rechazó el comando por inválido, en lugar del texto crudo de la sesión. Como los criterios están en una tabla, incorporar un mensaje nuevo no requiere modificar el algoritmo de reintentos.

El *rollback* se resuelve por compensación. Como se describió en el Capítulo 2, cada comando tiene efecto apenas el equipo lo acepta y la CLI no permite deshacer un cambio completo, por lo que el *rollback* aplica pasos que llevan el elemento de vuelta a su estado previo. Se omite si ningún intento llegó a conectarse con el equipo, porque no hay nada que revertir, y si no se pudo leer el estado previo, porque no hay a qué volver.

Sigue cuatro pasos. Lee el equipo para saber cómo quedó el elemento, calcula los pasos de compensación, los aplica y vuelve a leer el equipo para verificar la restauración. Sólo se revierte lo que todavía conserva el valor que puso el cambio, para no escribir sin necesidad ni deshacer una modificación posterior hecha por otra vía. Si la primera lectura falla, en los puertos, las interfaces virtuales y la configuración global se revierten todos los atributos, porque ante una pérdida de acceso es preferible restaurar de más. En las VLAN, en cambio, sólo se recrea una VLAN eliminada, porque eliminarla o renombrarla a ciegas podría destruir una VLAN ajena.

El *rollback* se ejecuta antes de liberar el bloqueo, para que nadie modifique el equipo entre la falla y la restauración. Para no retener el bloqueo indefinidamente, se reintenta una sola vez. Si aun así falla, el trabajo queda disponible para el reintento manual.

En un lote, esos pasos se realizan una sola vez para todos los recursos, con una lectura, una única sesión de aplicación y una lectura de verificación, de modo que el *rollback* requiere tres sesiones sin importar cuántos recursos tenga.

Después de un cambio exitoso, el estado almacenado se actualiza en dos pasos. Cuando la solicitud indica el valor final de un atributo, el orquestador lo guarda de inmediato, para que el usuario vea el resultado sin esperar una lectura. Después pide una sincronización, que corrige el estado con lo que realmente quedó en el equipo. Si llegan varios cambios seguidos sobre el mismo equipo y tipo de recurso, sólo el último la pide, porque una lectura al final los refleja a todos.

5.5.6. Módulo de Dominio

El dominio contiene las entidades que representan el problema, como los recursos gestionables, el trabajo, los eventos y las entidades organizativas. Mientras los demás módulos describen acciones, el dominio define sobre qué se realizan.

La Figura 5.9 muestra sus entidades principales y cómo se relacionan. El equipo es el centro del modelo. Pertenece a un grupo, los recursos gestionables lo modifican y los trabajos se ejecutan sobre él, registrando los recursos que se aplican. La API, la cola y la persistencia usan estas entidades, pero el dominio no las conoce.

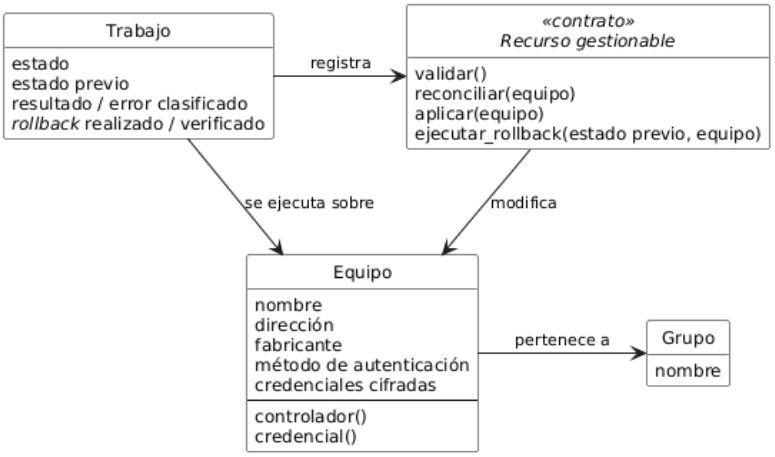


Figura 5.9: Clases del modelo de dominio.
Fuente: elaboración propia.

Cada recurso contiene sus propias reglas de validación. La API las comprueba antes de crear el trabajo y el orquestador antes de tocar el equipo. Como están en un solo lugar, una solicitud individual, un lote y una operación sobre un grupo aplican las mismas, sin repetirlas en cada punto de entrada.

El equipo guarda su fabricante y sus credenciales cifradas. Cuando un recurso necesita conectarse, le pide su controlador y su credencial (Figura 5.10). El controlador se obtiene de un registro según el fabricante. La credencial se descifra recién en ese momento y, como cada trabajo obtiene el equipo de nuevo, sólo se conserva mientras dura ese trabajo. Así, los recursos no necesitan saber de qué fabricante es el equipo ni si se autentica con contraseña o con clave. El dominio depende del contrato de los controladores y no de la implementación de cada fabricante, por lo que agregar un fabricante no requiere modificar ningún recurso.

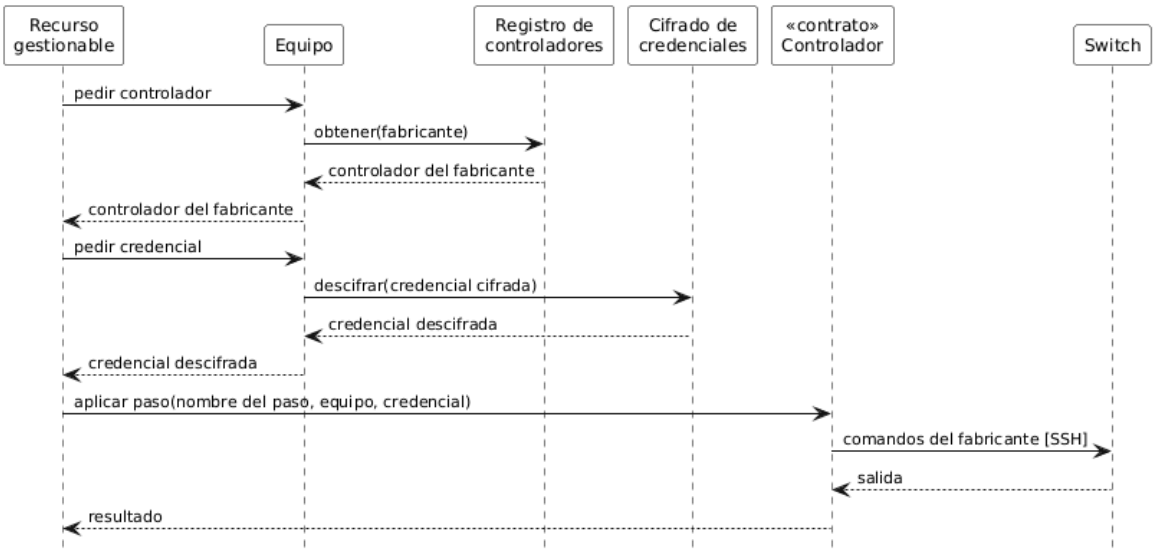


Figura 5.10: Acceso de un recurso al switch a través del equipo.

Fuente: elaboración propia.

5.5.7. Módulo de Acceso a Equipos

Este módulo traduce los pasos lógicos que resuelve un recurso en comandos de la CLI de cada fabricante, los envía al equipo e interpreta sus respuestas. Se compone de controladores, un vocabulario de comandos, dos vías de transporte y parsers.

Los controladores resuelven las diferencias entre fabricantes. Una misma operación, como pasar un puerto a modo acceso, se escribe distinto en cada uno. Algunas diferencias, además, no son sólo de sintaxis. Huawei, por ejemplo, limita la cantidad de VLAN por comando y pide una confirmación que depende del estado previo del puerto, por lo que cada fabricante necesita su propia lógica.

El patrón Strategy [41] encapsula cada variante de un algoritmo en una clase intercambiable detrás de una interfaz común. Aquí, cada fabricante tiene un controlador que

cumple un mismo contrato, de modo que el recurso invoca siempre las mismas operaciones (Figura 5.11). Un controlador simulado responde como un equipo sin abrir conexiones, lo que permite ejecutar el sistema completo en desarrollo y en las pruebas.

El contrato no es sólo una interfaz, ya que su clase base concentra la mecánica común, como leer el vocabulario de comandos, probar las alternativas ante un rechazo, detectar los errores y elegir la vía de transporte. Así, cada controlador implementa sólo lo propio de su fabricante, mientras que lo que sostiene la confiabilidad se escribe una única vez.

El contrato exige sólo las operaciones básicas, como crear, renombrar y eliminar VLAN, guardar la configuración y leer las VLAN y los puertos. Las demás indican por defecto que no están soportadas, lo que la API comprueba antes de crear el trabajo. Así, un fabricante nuevo puede incorporarse de a poco sin que el sistema falle con lo que todavía no ofrece.

Para elegir el controlador de cada equipo sin una cadena de condiciones sobre el fabricante, se aplica el patrón Registry [42], que mantiene un registro de implementaciones identificadas por nombre, en este caso el fabricante. El punto de composición registra los controladores al iniciar el sistema, o el simulado bajo los mismos nombres, sin que el resto del sistema note la diferencia. Así, incorporar un fabricante consiste en escribir su controlador y registrarlo.

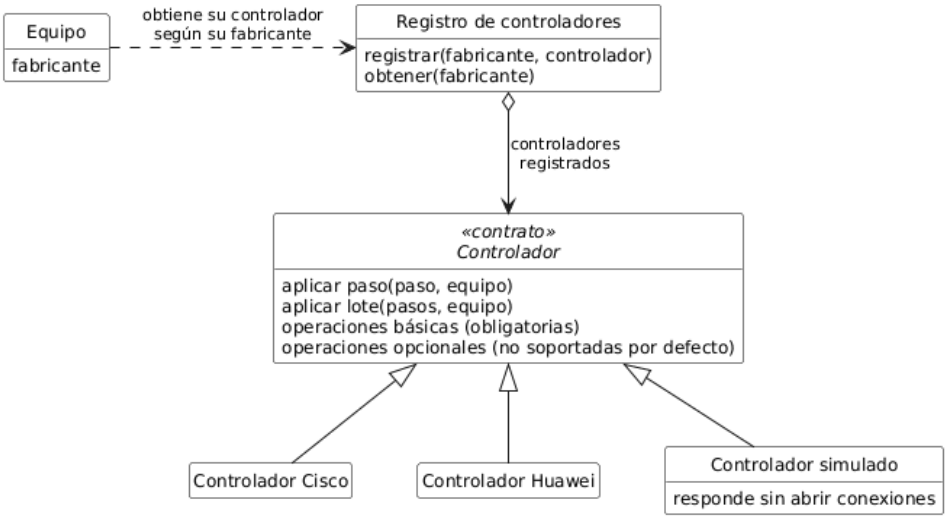


Figura 5.11: Controladores por fabricante.
Fuente: elaboración propia.

Los comandos no se escriben en el código de los controladores, sino en un vocabulario declarativo por fabricante (Figura 5.12). Como entre modelos y versiones de *firmware* la mayoría de los comandos coinciden, cada operación se escribe una vez y sus variantes se agregan como alternativas asociadas a un patrón de error, que se prueban si el equipo rechaza la secuencia principal. Así, no hace falta un vocabulario por modelo ni conocer de antemano el *firmware* de cada equipo. Una variante nueva se incorpora editando el vocabulario, sin modificar el controlador.

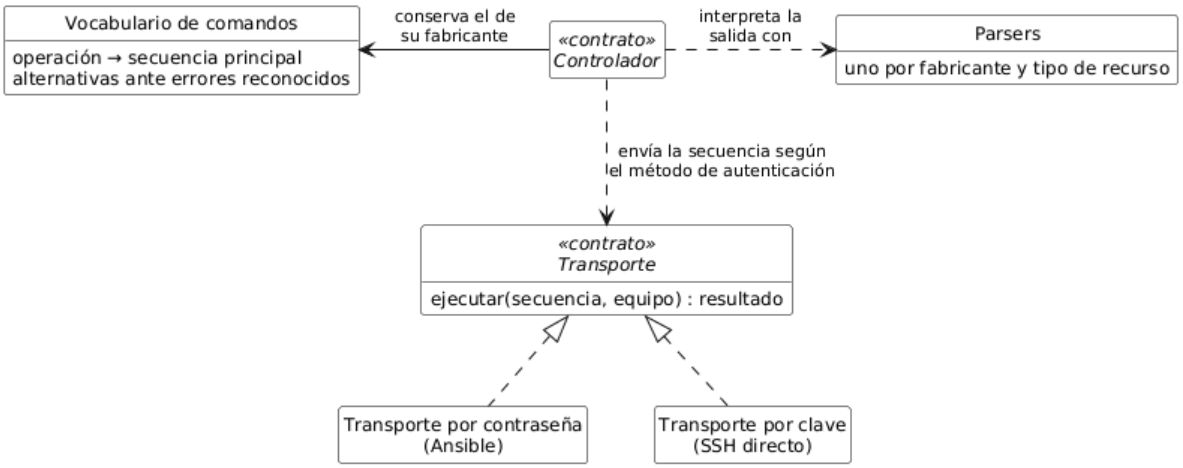


Figura 5.12: Vocabulario de comandos, transporte y parsers.

Fuente: elaboración propia.

El Listado 5.1 muestra la operación que pasa un puerto a modo acceso en ambos fabricantes. En Cisco hace falta borrar la configuración que el puerto tenía como troncal. En Huawei, el cambio pide una confirmación sólo si el puerto venía de troncal con VLAN asignadas.

El controlador decide qué comandos enviar, pero no cómo llegan al equipo. Esa tarea se separa en un adaptador de transporte, siguiendo el patrón Adapter [41], que convierte una herramienta de conexión externa en un contrato común. Todo adaptador recibe la secuencia de comandos y devuelve el resultado de la misma forma.

```
# Cisco IOS
set_access_mode:
  primary:
    parents: "interface {interface}"
    lines:
      - "switchport mode access"
      - "switchport access vlan {vlan_id}"
      - "no switchport trunk native vlan"
      - "no switchport trunk allowed vlan"

# Huawei VRP (variante para un puerto que venía de troncal)
set_access_mode:
  con_confirmacion:
    primary:
      block: ["system-view", "interface {interface}", "port link-type access",
              "y", "port default vlan {vlan_id}", "commit", "quit", "quit"]
    alternatives:
      - triggered_by_error: 'commit ... Error: Unrecognized command'
        block: ["system-view", "interface {interface}", "port link-type access",
                "y", "port default vlan {vlan_id}", "quit", "quit"]
```

Listado 5.1: Operación "pasar a modo acceso" en el vocabulario de cada fabricante (simplificado).

Depender de una única herramienta de conexión es un riesgo. De hecho, algunos equipos debían autenticarse por clave y las bibliotecas SSH de Python que usa Ansible no lograron hacerlo contra su firmware. Por eso hay dos adaptadores, uno basado en Ansible y otro que abre una sesión SSH directa. La vía se elige en un único punto según el método de autenticación del equipo, de modo que los equipos con contraseña usan Ansible y los que usan clave, la vía directa. Como esa regla está en un solo lugar, puede cambiarse sin afectar a los controladores, al vocabulario ni al orquestador.

Los parsers convierten la salida de las consultas en objetos del dominio, iguales para cualquier fabricante. Esa salida es texto pensado para una persona, con un formato que cambia entre fabricantes y versiones de *firmware*, por lo que los parsers aíslan esa parte frágil del resto del sistema. Así, la sincronización y el estado almacenado trabajan siempre con los mismos objetos. Un cambio en el formato de un equipo se corrige sólo en su parser.

Cuando un dato no puede leerse, el parser lo marca como desconocido en lugar de suponer un valor, para que el resto del sistema no tome decisiones con información que no tiene. Por ejemplo, el *rollback* no restaura un atributo cuyo valor previo se desconoce.

Existe un *parser* por fabricante y por tipo de recurso. Para los puertos, las interfaces virtuales y la configuración global, en los que los dos fabricantes comparten la estructura del resultado, se define una clase abstracta con una implementación por fabricante. Para las

VLAN, las tablas ARP y MAC y las listas de acceso se usan funciones por fabricante, porque no hay un comportamiento compartido que justifique una jerarquía de clases.

5.5.8. Módulo de Auditoría y Eventos

La auditoría registra quién hizo qué, sobre qué recurso, cuándo y con qué resultado. Muchas operaciones distintas deben quedar registradas, y si cada una escribiera en el registro por su cuenta, la auditoría dependería de que ninguna lo olvide. Por eso el módulo aplica el patrón Observer [41], que desacopla a quien produce un hecho de quienes reaccionan a él. La Figura 5.13 muestra su estructura.

Los emisores, como el orquestador, el inventario, el servicio de asignación de roles y las operaciones administrativas, representan cada hecho relevante como un evento de dominio, que indica qué ocurrió, sobre qué entidad, quién lo produjo, si fue exitoso y los datos asociados. La entidad puede ser un recurso aplicado sobre un equipo, pero también un sitio, un usuario o una asignación de rol.

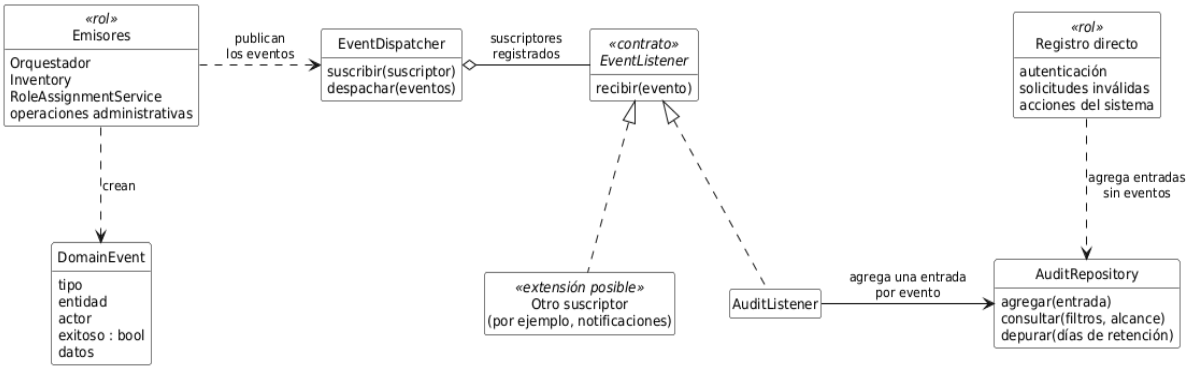


Figura 5.13: Clases del módulo de auditoría y eventos.

Fuente: elaboración propia.

Los emisores publican los eventos en un despachador, que los entrega a todos los suscriptores registrados. Como quien genera un evento desconoce quién lo consume, se puede agregar un suscriptor nuevo, como un servicio de notificaciones, sin modificar a ningún emisor.

El suscriptor de auditoría, hasta ahora el único, agrega una entrada al registro por cada evento, con el usuario, la operación, el recurso, el resultado, la fecha, el equipo, el trabajo y un resumen legible.

El registro ofrece tres operaciones: agregar una entrada, consultar y depurar. No existe una operación para modificar una entrada y la persistencia rechaza cualquier intento de

hacerlo [38], de modo que nadie, ni siquiera un administrador, puede alterar lo registrado. La única eliminación prevista es la depuración diaria de las entradas que superan los 90 días de retención, que también puede solicitar el administrador del sistema y que queda a su vez registrada.

La consulta recibe además el alcance del usuario, porque el registro revela información sobre los equipos. Cada usuario ve sólo las entradas de su alcance y sus propios eventos de autenticación, y el administrador del sistema, el registro completo. La restricción se aplica también en la base de datos, de modo que una consulta mal construida no expone entradas ajenas.

5.5.9. Módulo de Sincronización

La sincronización mantiene el estado almacenado de cada equipo, es decir, sus VLAN, puertos, interfaces virtuales y configuración global, además de sus tablas ARP y MAC y su registro local. Mientras la orquestación modifica los equipos, la sincronización los lee.

El estado almacenado funciona como un modelo de lectura separado del de escritura. Las consultas lo leen siempre, sin abrir sesiones con los equipos, y sólo lo actualizan la sincronización y el orquestador. Cada tipo de información registra su última sincronización y su último error, que la interfaz muestra junto con los datos. Así, se ve la antigüedad de lo mostrado sin que cada consulta pague el costo de actualizarlo.

Cada lectura consume procesador en el *switch* y, en los modelos de menor capacidad, provoca picos de uso. Por eso una sincronización se solicita sólo cuando hace falta, al registrar un equipo, después de cada cambio completado, a pedido del usuario y al entrar a la vista de un sitio, un grupo o un equipo, para los que tengan su estado desactualizado. Este último caso reemplaza a un barrido periódico de todo el inventario, que sigue disponible como opción pero está desactivado por defecto. Las solicitudes del usuario y las que se disparan al entrar a una vista pasan por la unión de sincronizaciones (sección 5.5.4).

Las VLAN, los puertos y las interfaces virtuales se leen de forma combinada, con una única toma del bloqueo del equipo. En Cisco, todas las consultas se envían en una sola sesión. En Huawei se requieren dos, porque el sistema operativo del equipo no permite consultar en un solo comando la configuración de todas sus interfaces virtuales. Primero es

necesario obtener cuáles existen para luego consultarlas una por una. La configuración global se sincroniza por separado, porque implica leer la configuración completa del equipo.

Si la lectura falla, se conserva el último estado conocido junto con el error. Si sólo falla el almacenamiento de un tipo de información, los demás se guardan igualmente. Una lectura incompleta nunca se almacena como un estado vacío.

5.5.10. Módulo de Persistencia y Modelo de Datos

La persistencia almacena la organización, los usuarios, el estado de los equipos, los trabajos y el registro de auditoría en una base de datos relacional.

El dominio y el orquestador trabajan con objetos, mientras que la base guarda filas con columnas fijas, y mezclar ambas formas haría que el dominio dependa del esquema. El patrón Repository [42] resuelve este problema ofreciendo una colección de objetos que oculta cómo se almacenan, a la que se le pide obtener, listar, agregar, actualizar o eliminar (Figura 5.14).

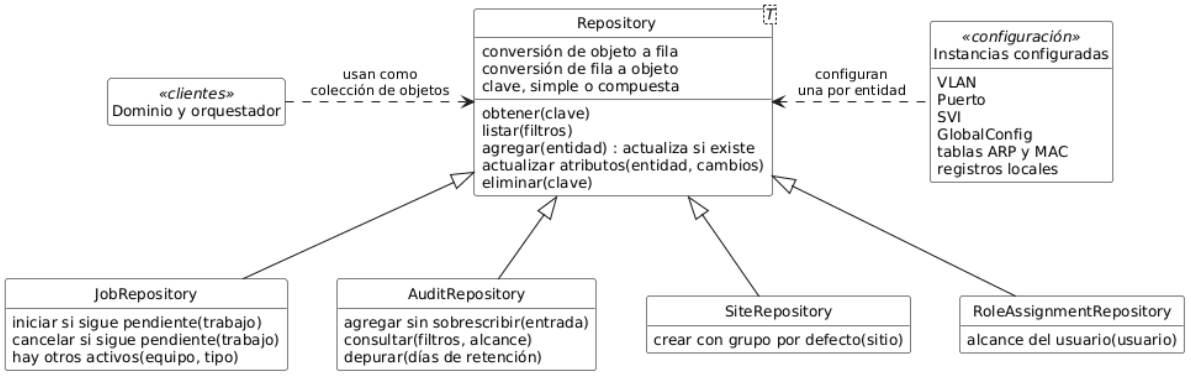


Figura 5.14: Repositorios del módulo de persistencia.

Fuente: elaboración propia.

Un único repositorio genérico implementa esas operaciones una sola vez, y para cada entidad se le indica cómo convertir entre objetos y filas y cuál es su clave, que puede combinar varias columnas, como el número y el equipo de una VLAN. Así, todo el estado almacenado usa la misma clase, en lugar de una por tipo de información.

Agregar un objeto que ya existe lo actualiza, de modo que una sincronización o un reintento pueden volver a escribirlo sin duplicarlo. Actualizar algunos atributos, por su parte, registra el cambio sin borrar los que no tocó.

Sólo las entidades con reglas propias tienen un repositorio específico, que suma a las operaciones comunes las que exige su módulo, como iniciar un trabajo sólo si sigue pendiente o agregar entradas de auditoría sin sobrescribirlas.

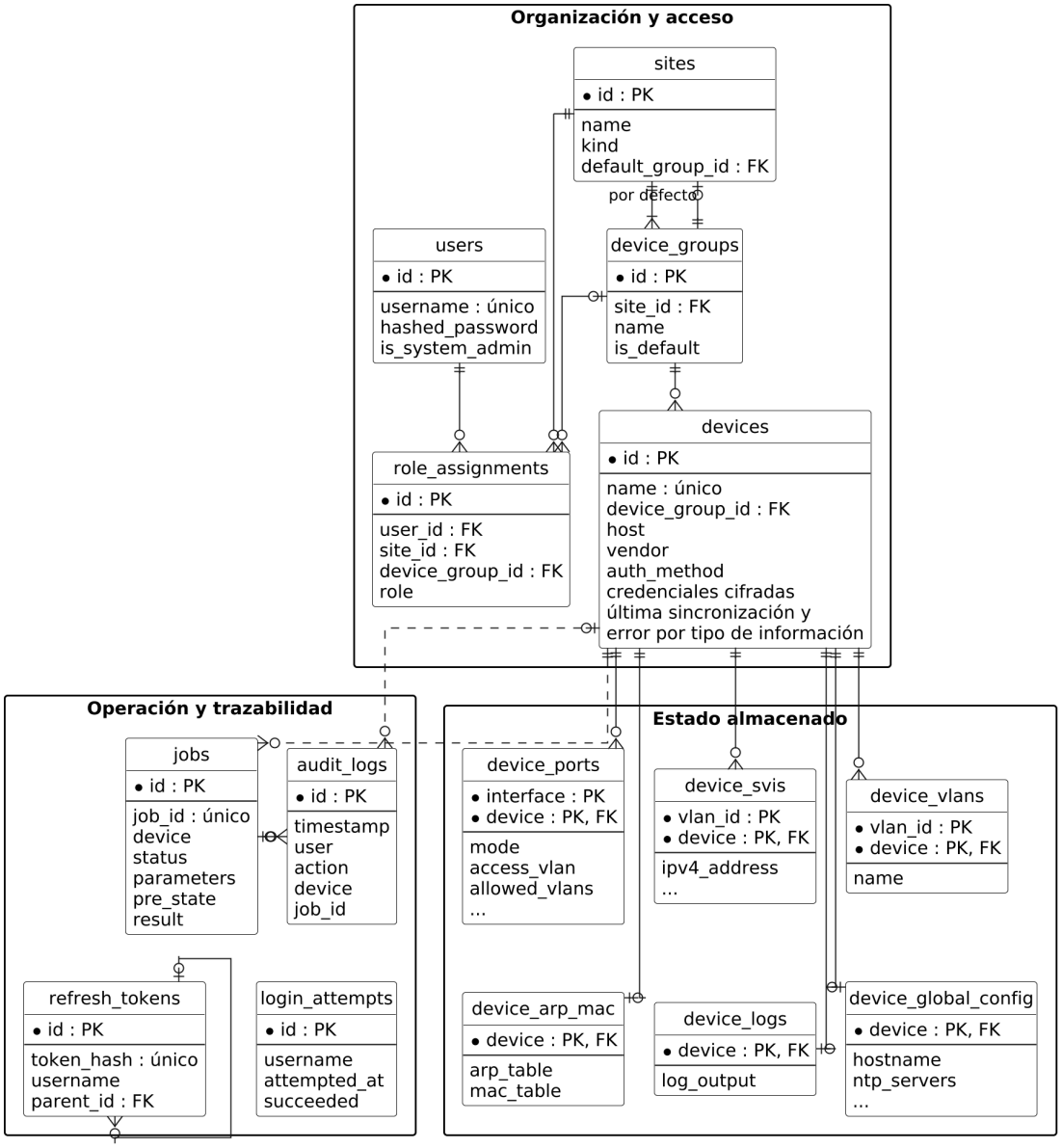


Figura 5.15: Modelo de datos.

Fuente: elaboración propia.

El modelo de datos organiza las tablas en tres grupos (Figura 5.15). El de organización y acceso refleja la jerarquía de sitios, grupos y equipos, y las asignaciones de rol de cada usuario. El de estado almacenado tiene una tabla por tipo de información, que se elimina junto con el equipo. El de operación y trazabilidad reúne los trabajos, la auditoría y dos tablas de apoyo a la autenticación.

Los trabajos y la auditoría guardan el nombre del equipo sin depender de su existencia, y por eso esas relaciones son punteadas en la figura, de modo que el historial se conserva aunque el equipo se dé de baja. El esquema se versiona con migraciones, que se aplican automáticamente al iniciar la API.

El aislamiento entre sitios se refuerza en la propia base de datos, mediante políticas de seguridad a nivel de fila sobre las tablas de sitios, grupos, equipos y auditoría. Al iniciar cada transacción, la API informa a la base qué usuario la realiza, y la base devuelve sólo las filas que ese usuario puede ver, o ninguna si no se informa un usuario, de modo que un acceso que omita identificarse falla de forma segura.

Las políticas mencionadas restringen sólo las lecturas, donde una falla expondría datos de otros sitios y la regla es simple. Las escrituras se autorizan en la API, con reglas más ricas, como exigir permisos en el grupo de origen y en el de destino de un traslado, y repetirlas en la base obligaría a mantenerlas en dos lugares.

5.5.11. Trazabilidad con los requerimientos

Módulo	Requerimientos funcionales	Requerimientos no funcionales
API REST (5.5.1)	Punto de entrada de RF1 a RF14	RNF1, RNF5
Organización y acceso (5.5.2)	RF1, RF2, RF3, RF11	RNF1, RNF2
Ejecución asíncrona (5.5.3)	RF8, RF12, RF13	RNF5
Coordinación (5.5.4)	RF13	RNF3, RNF5
Orquestación (5.5.5)	RF8, RF12	RNF3, RNF4, RNF6
Dominio (5.5.6)	RF4, RF5, RF6, RF7	RNF6
Acceso a equipos (5.5.7)	RF4, RF5, RF6, RF7	RNF6, RNF7
Auditoría y eventos (5.5.8)	RF9	RNF4
Sincronización (5.5.9)	RF3, RF13, RF14	RNF5
Persistencia y modelo de datos (5.5.10)	RF3, RF11, RF14	RNF2, RNF4
Cliente web (5.6.1)	RF10, RF14	RNF7

Cuadro 5.3: Requerimientos atendidos por cada módulo.

El Cuadro 5.3 resume qué requerimientos atiende cada módulo. Todos quedan cubiertos por al menos uno, y varios se reparten entre más de un módulo, como la sincronización.

5.6. Diseño de Interfaces

5.6.1. Interfaz de Usuario

El cliente web es el punto de contacto de los usuarios con la plataforma, y permite operar sin recurrir a la línea de comandos de los equipos. La Figura 5.16 muestra cómo se organiza su navegación.

El usuario parte de un tablero con el resumen de la organización y desciende de sitio a grupo y de grupo a equipo. Todos los niveles ofrecen las mismas vistas, un resumen y una vista por recurso, aplicadas al conjunto de equipos del nivel, de modo que la misma vista sirve para revisar un equipo o para crear una VLAN en todo un sitio. Las vistas generales, como trabajos, auditoría, usuarios, sesiones e inventario, están disponibles desde cualquier punto.

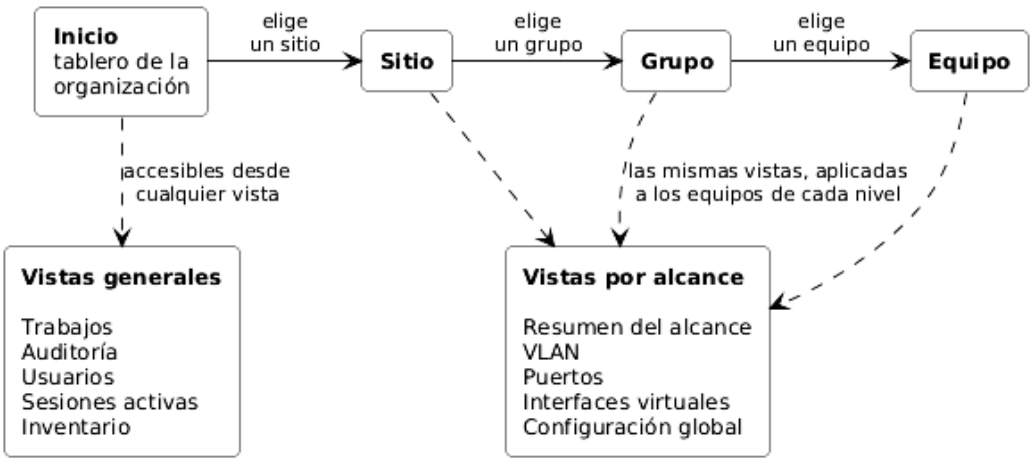


Figura 5.16: Mapa de navegación del cliente web.

Fuente: elaboración propia.

Las vistas por recurso muestran el estado almacenado con su antigüedad, y al entrar a un nivel se solicita la lectura de los equipos desactualizados. Los cambios se envían como trabajos, por lo que no bloquean la interfaz, y el cliente avisa al usuario cuando terminan.

Cada acción se habilita u oculta según el rol del usuario en el alcance actual, aunque la autorización efectiva ocurre en la API. La interfaz advierte, además, cuando un cambio puede cortar el camino por el que la plataforma accede al equipo, como apagar un puerto que transporta la VLAN de gestión, uno de los errores de mayor impacto de la sección 4.1, que ni el *rollback* puede reparar. El aviso no bloquea el cambio, porque la plataforma no puede saber qué puerto es el enlace real, y la decisión queda en manos del operador.

5.6.2. API REST

La API REST es la segunda interfaz del sistema, que usa el cliente web y que permite a otros sistemas de la organización integrarse con la plataforma. Esta sección describe el contrato que ve quien la consume, que responde a cuatro decisiones.

- **Rutas orientadas a recursos:** los recursos se identifican mediante rutas bajo un prefijo con número de versión, que permite modificar la API sin romper las integraciones existentes, y los de un equipo se anidan bajo su ruta. Cada operación usa el método HTTP acorde a su semántica [28].
- **Escrituras asíncronas:** toda operación que modifica un equipo responde con el código 202 (Accepted), que indica que la solicitud fue aceptada pero todavía no se aplicó, junto con el identificador del trabajo, con el que el cliente consulta su estado.
- **Formato uniforme:** todas las respuestas indican si la operación tuvo éxito y traen los datos o, en caso de error, un código, un mensaje y sus detalles. Así, quien consume la API puede tratar todos los errores de la misma forma, sin conocer cada ruta.
- **Documentación automática:** la API se documenta según la especificación OpenAPI [29] a partir de las propias definiciones de las rutas, por lo que la documentación no puede quedar desactualizada.

Código	Situación real en el sistema
200	Operación exitosa, como una consulta o un cambio en la organización que se aplica en el momento
201	Se creó un elemento de la aplicación sin pasar por un equipo, como una asignación de rol
202	La solicitud de cambio fue aceptada y todavía no se aplicó en el equipo
204	La operación no devuelve contenido, como revocar una asignación de rol
400	Falta un dato que la operación necesita y que el esquema no puede exigir
401	Falta el token o no es válido
403	El usuario está identificado pero no tiene el rol mínimo requerido
404	El equipo, sitio o grupo referenciado no existe
409	La operación entra en conflicto con el estado actual
422	La solicitud no cumple el esquema o viola una regla del recurso
429	Se superó el límite de solicitudes permitidas
500	Error interno no previsto. Las fallas al aplicar un cambio en un equipo se informan en el estado del trabajo
501	La operación no está soportada para el fabricante del equipo

Cuadro 5.4: Respuestas de la API REST.

El Cuadro 5.4 resume los códigos de respuesta que devuelve la API y el Cuadro 5.5, los recursos que expone. El detalle completo de cada ruta se consulta en la documentación que genera el propio sistema.

Recurso	Ruta (bajo /api/v1)	Operaciones principales	Rol mínimo (consulta / escritura)
Autenticación	/auth	Iniciar, renovar y cerrar sesión, reautenticación, sesiones activas, desbloqueo de cuentas	Cualquier usuario autenticado (desbloqueo: administrador del sistema)
Usuarios	/users	Alta, consulta, modificación y desactivación, asignación de roles	Administrador de sitio / Administrador de sitio
Sitios	/sites	Alta, consulta, modificación y baja	Observador / Administrador (alta y baja: administrador del sistema)
Grupos	/device-groups	Alta, consulta y baja, equipos del grupo	Observador / Administrador
Equipos	/devices	Registro, modificación, traslado y baja, actualización del estado, guardado de configuración	Observador / Administrador (guardado y traslado dentro del sitio: operador)
VLAN	/vlans	Consulta, creación, renombrado y eliminación, individual o en lote	Observador / Operador (eliminación: administrador)
Puertos	/devices/{equipo}/ports	Consulta, modo, VLAN, descripción, habilitación, PoE, control de tormentas, restablecimiento, lote	Observador / Operador
Interfaces virtuales	/devices/{equipo}/svis	Consulta, alta y baja, direccionamiento, ACL, relay DHCP, descripción, habilitación, lote	Observador / Operador
Configuración global	/devices/{equipo}/global-config	Consulta, versión, configuración en ejecución, tablas ARP y MAC, búsqueda de MAC, registro local, modificación de parámetros	Observador / Administrador
Trabajos	/jobs, /group-jobs	Consulta, cancelación, reintento de rollback	Observador / Operador
Auditoría	/audit	Consulta y depuración	Observador / Administrador del sistema
Tablero	/dashboard	Resumen por alcance, actualización del estado	Observador / Observador

Cuadro 5.5: Recursos de la API REST.

5.7. Stack Tecnológico

Las tecnologías de cada módulo se seleccionaron a partir del diseño y de las limitaciones del proyecto (sección 4.5.1). El Cuadro 5.6 resume esa asignación, y a continuación se justifican las decisiones principales.

- **Lenguaje del servidor:** se eligió Python porque Ansible ofrece en ese lenguaje la biblioteca *ansible-runner*, que lanza cada ejecución desde el servidor y devuelve su resultado como datos. Con otro lenguaje, como Go o Java, habría que invocar Ansible como un programa externo e interpretar su salida. Además, las herramientas de la sección 2.4, como Netmiko, Nornir y Paramiko, también son de Python.
- **API REST:** FastAPI genera la documentación de la API a partir de sus rutas, valida los datos de cada solicitud con Pydantic y permite que cada ruta declare los permisos que exige. Django REST Framework se descartó porque asocia cada objeto a una tabla, mientras que muchos objetos del dominio, como una solicitud de cambio, no se guardan tal cual en la base, y Flask, porque habría obligado a agregar por separado la validación y la documentación.

- **Persistencia:** se eligió PostgreSQL porque ofrece de forma nativa las políticas de seguridad a nivel de fila [43] en las que se apoya el aislamiento entre alcances y aplica las migraciones dentro de una transacción, de modo que una que falla no deja la base a medio modificar. MySQL y MariaDB no ofrecen ninguna de las dos. SQLAlchemy convierte entre objetos y filas, y Alembic versiona las migraciones.
- **Ejecución asíncrona y coordinación:** Celery reúne en una sola herramienta los workers, el reparto de tareas y un planificador periódico, que corre aparte y encola el barrido de trabajos huérfanos y, si se habilita, la sincronización periódica. Las tareas de mantenimiento de la API, como la depuración diaria de la auditoría, las programa APScheduler dentro de su propio proceso. Redis sirve a la vez como cola de Celery y como servicio de coordinación, porque ejecuta de forma atómica las operaciones de bloqueo y reserva. RabbitMQ habría servido como cola, pero la coordinación igualmente requería un servicio compartido.
- **Acceso a equipos:** Ansible se adoptó por las razones de la sección 2.4.2, entre ellas que no requiere agentes y que distribuye su soporte por fabricante en colecciones [23][24], y se ejecuta mediante ansible-runner [45], que aísla cada ejecución en un proceso propio, algo que con Netmiko habría que programar. Nornir se descartó porque su ejecución concurrente se superpondría con la de los *workers* [19][22]. La vía directa usa el cliente OpenSSH, el único que autenticó por clave de forma consistente en los equipos físicos de ambos fabricantes. El vocabulario de comandos se escribe en YAML, el formato de los *playbooks*.
- **Identidad y cifrado:** los *tokens* siguen el estándar JWT y se emiten con PyJWT, las contraseñas se almacenan con PBKDF2-SHA256, aceptada por NIST para este fin [37], y las credenciales de los equipos se cifran con Fernet, un esquema de cifrado autenticado que evita combinar primitivas criptográficas a mano.
- **Cliente web:** Next.js organiza la aplicación en rutas que corresponden a los niveles de alcance, con componentes React en TypeScript, TanStack Query para la caché de las consultas y Tailwind CSS para los estilos.
- **Despliegue:** Docker Compose levanta la API, los *workers* y el planificador de Celery, con una misma imagen, junto con la base de datos y el servicio en memoria. Caddy actúa como proxy inverso y punto de entrada HTTPS.

Módulo o contenedor	Tecnología	Alternativa considerada
Lenguaje del servidor	Python 3.12	Go, Java
API REST y validación	FastAPI, Uvicorn y Pydantic	Django REST Framework, Flask
Persistencia	PostgreSQL 16, SQLAlchemy 2 y Alembic	MySQL, MariaDB
Ejecución asíncrona	Celery (*workers* y planificador Beat)	Tareas en segundo plano dentro de la API
Cola y coordinación	Redis	RabbitMQ o bloqueos en la base de datos
Acceso a equipos por contraseña	Ansible, ejecutado mediante ansible-runner	Netmiko, Nornir
Acceso a equipos por clave pública	Cliente OpenSSH del sistema operativo	Paramiko
Vocabulario de comandos	YAML	Sin alternativa evaluada
Identidad y cifrado	PyJWT, passlib (PBKDF2-SHA256) y cryptography (Fernet)	Sin alternativa evaluada
Cliente web	Next.js 16, React 19, TypeScript, TanStack Query y Tailwind CSS 4	Sin alternativa evaluada
Despliegue	Docker Compose y Caddy	Sin alternativa evaluada

Cuadro 5.6: Tecnologías empleadas por módulo.

5.8. Entorno de Desarrollo

El entorno de desarrollo busca la reproducibilidad, la calidad y la seguridad del trabajo, considerando que el sistema opera sobre equipos en servicio.

Entorno virtual y contenedores: las dependencias del servidor se instalan en un entorno virtual de Python con versiones acotadas, y las del cliente web con el gestor de paquetes de Node.js. Docker Compose levanta todos los servicios del servidor.

Modo simulado y laboratorio: una variable de configuración elige entre los controladores reales y el simulado, con el que el sistema funciona sin equipos para desarrollar y ejecutar las pruebas sin afectar la red. Antes de llegar a producción, las funcionalidades se validan en un laboratorio con equipos físicos y virtuales de ambos fabricantes.

Calidad de código: las pruebas automatizadas del servidor usan pytest y se ejecutan contra PostgreSQL porque el esquema usa capacidades propias de ese motor. Las de seguridad a nivel de fila se ejecutan con un rol sin superusuario, para comprobar que las políticas se aplican. En el cliente web, TypeScript verifica los tipos y ESLint señala errores frecuentes y malas prácticas.

Control de versiones: el código se gestiona con Git y se aloja en GitHub. La rama principal contiene la versión estable del sistema, y cada tarea se desarrolla en una rama propia, identificada por un prefijo según su tipo (*feat*/ para funcionalidades, *fix*/ para correcciones y *refactor*/ para reestructuraciones), que se integra a la principal una vez finalizada.

6. Primera Iteración: Conexión con los Equipos y Funcionalidades Base

6.1. Planificación y Diseño

La primera iteración tuvo como objetivo establecer la comunicación entre la plataforma y los *switches* y construir sobre ella las funcionalidades base de las que dependen las demás. Se buscaba que una operación solicitada desde la API REST o el cliente web se tradujera en la ejecución asíncrona de un *playbook* de Ansible sobre los equipos Cisco IOS y Huawei VRP del laboratorio y quedara registrada como un trabajo, con su resultado y los mensajes devueltos por el equipo.

Conforme a la priorización de requerimientos de la sección 4.6.4, se abordaron el inventario de dispositivos, la gestión de VLAN y la ejecución asíncrona de trabajos, de criticidad muy alta. Se sumaron la autenticación, la gestión de usuarios, la auditoría y una interfaz web básica, que pese a su menor criticidad se incluyeron porque todas las operaciones posteriores dependen de ellas (Cuadro 6.1). La primera prioridad fue validar la conectividad SSH con el equipamiento, el principal factor de riesgo del proyecto (Cuadro 4.4). Sobre ella se construyó la gestión de VLAN y el sistema de trabajos, luego la autenticación, los usuarios y la auditoría y, por último, la interfaz web, sobre la API ya estabilizada.

Requerimiento	Atendido en esta iteración	Pendiente para las siguientes
RF1: Autenticación	Inicio de sesión con dos tokens, bloqueo por intentos fallidos y cierre de la sesión actual	Sesiones activas y reautenticación
RF2: Usuarios	Alta, modificación y desactivación de usuarios, contraseñas derivadas con sal y designación de administradores del sistema	—
RF3: Inventario	Registro y administración de equipos, credenciales cifradas, autenticación por contraseña y guardado de la configuración	Autenticación por clave, lectura del estado al registrar y actualización manual
RF4: VLAN	Alta, renombrado y baja sin cambios innecesarios, y consulta leyendo el equipo	Consulta desde el estado almacenado
RF8: Trabajos	Validación previa, trabajos asíncronos con estado previo, reintentos, rollback y cancelación de pendientes	Errores desconocidos, omisión del rollback si el cambio no llegó al equipo, estado actualizado tras un cambio y reintento manual del rollback
RF9: Auditoría	Registro inmutable, filtros por usuario, operación, recurso, equipo y fechas, y depuración	Filtro por resultado y por sitio, y auditoría por alcance
RF10: Interfaz web	Vistas de las funciones de esta iteración y aviso al terminar cada trabajo	Vistas de las iteraciones siguientes

Cuadro 6.1: Requerimientos del sistema atendidos en la primera iteración.

El desarrollo alternó entre dos modos. El modo simulado, que se habilita con una variable de configuración, reemplaza la ejecución sobre los equipos por respuestas simuladas. En él, un equipo con un nombre reservado falla siempre, para ejercitar los caminos de error y de *rollback*. Con este modo se construyó la lógica y se ejecutaron las pruebas automatizadas, incluidos los caminos de error y de *rollback*. El modo real se usó para la validación sobre un laboratorio emulado en GNS3 [44]. En cuanto al diseño, varios módulos se implementaron en esta iteración en una forma más simple que la del Capítulo 5, que las iteraciones siguientes llevaron a su forma definitiva (Cuadros 7.1 y 8.1).

6.2. Implementación

Funcionalidad "Gestión del Inventario de Dispositivos"

Registrar un equipo en el inventario supone que la plataforma pueda conectarse a él, por lo que lo primero fue verificar esa conexión, en dos etapas. Primero, una sesión con el cliente OpenSSH confirmó desde el servidor la alcanzabilidad de cada equipo, sus credenciales y sus privilegios. Después se automatizó la misma interacción con Ansible, mediante el tipo de conexión network_cli y las colecciones cisco.ios y community.network.

Cada operación sobre un equipo seguía el mismo recorrido. El controlador del fabricante elegía el *playbook* de la operación y sus variables, como el número y el nombre de la VLAN. La biblioteca ansible-runner lo lanzaba como un proceso separado, con esas variables y los tiempos máximos de espera de la conexión, configurables desde el entorno. Al terminar, la plataforma recorría los eventos que ansible-runner emitía por cada tarea. De la tarea de consulta tomaba la salida del equipo, que pasaba al parser, y de la tarea fallida, el mensaje de error, que registraba en el trabajo. El inventario que Ansible necesita, con la dirección del equipo y su credencial, se generaba en un archivo temporal propio de cada ejecución y se borraba al terminar.

Para aislar las diferencias entre fabricantes se implementó el contrato del controlador, según el patrón Strategy, con cinco operaciones: crear, renombrar, eliminar y consultar VLAN, y guardar la configuración. El controlador de cada equipo se elegía mediante una condición sobre su fabricante. Cada operación se resolvía con un *playbook* propio del fabricante, como los de alta del Listado 6.1, que usan el módulo de configuración de cada colección. Los *parsers* convierten las respuestas en objetos del dominio y omiten las VLAN

que el equipo trae de fábrica y no permite eliminar, como la 1 en ambos fabricantes y las 1002 a 1005 de Cisco, aunque la VLAN 1 sigue disponible para configurar los puertos.

```
# (a) Cisco IOS: vendors/cisco/create_vlan.yml
- name: Create VLAN on Cisco device
  hosts: all
  gather_facts: no
  tasks:
    - name: Ensure VLAN exists with name
      cisco.ios.ios_config:
        parents: "vlan {{ vlan_id }}"
        lines:
          - name {{ vlan_name }}
    - name: Save running config
      cisco.ios.ios_config:
        save_when: always
```

```
# (b) Huawei CloudEngine: vendors/huawei/create_vlan.yml
- name: Create VLAN on Huawei VRP device
  hosts: all
  gather_facts: no
  tasks:
    - name: Ensure VLAN exists with name
      community.network.ce_config:
        parents: vlan {{ vlan_id }}
        lines:
          - description {{ vlan_name }}
```

Listado 6.1: Playbooks de alta de una VLAN en la primera iteración: (a) Cisco IOS; (b) Huawei CloudEngine.

El inventario registra cada equipo con su nombre único, su dirección IP de gestión, su fabricante y sus credenciales. A diferencia de las contraseñas de los usuarios, que se guardan como un resumen irreversible, la de un equipo se necesita en texto claro para iniciar sesión en él. Por eso se cifra con Fernet, con una clave que se lee de una variable de entorno del servidor y no se guarda en la base. Así, una copia de la base no alcanza para recuperar las credenciales.

Si esa variable no está definida, la API no arranca. La contraseña nunca se devuelve en las respuestas de la API ni se incluye en el registro de auditoría. En esta iteración, los observadores podían consultar el inventario, los operadores guardar la configuración y los administradores dar de alta, modificar y dar de baja equipos.

La persistencia se apoyó en SQLAlchemy sobre SQLite. Junto a las primeras migraciones de Alembic, la API creaba al iniciar las tablas que faltaban y agregaba las columnas nuevas con instrucciones propias.

Funcionalidad "Gestión de VLAN y Ejecución Asíncrona de Trabajos"

La API ofrecía la consulta, el alta, el renombrado y la baja de VLAN, y una misma solicitud podía abarcar varios equipos. La consulta se resolvía dentro de la propia solicitud. La API tomaba el bloqueo del equipo, ejecutaba el playbook de consulta y devolvía las VLAN leídas, o respondía que el equipo estaba ocupado si otra operación lo retenía por más de unos segundos.

En los cambios (Figura 6.1), la API validaba primero los parámetros, como que el identificador estuviera entre 1 y 4094 y que el nombre no superara los 32 caracteres. También comprobaba que los equipos existieran, de modo que una solicitud inválida se rechazaba sin llegar a ningún equipo. Luego, por cada equipo, creaba un trabajo pendiente con su entrada de auditoría y leía el estado previo de la VLAN, también bajo el bloqueo del equipo. Por último, entregaba cada trabajo junto con ese estado a una tarea en segundo plano y respondía con los identificadores de los trabajos, sin esperar a que se aplicaran los cambios.

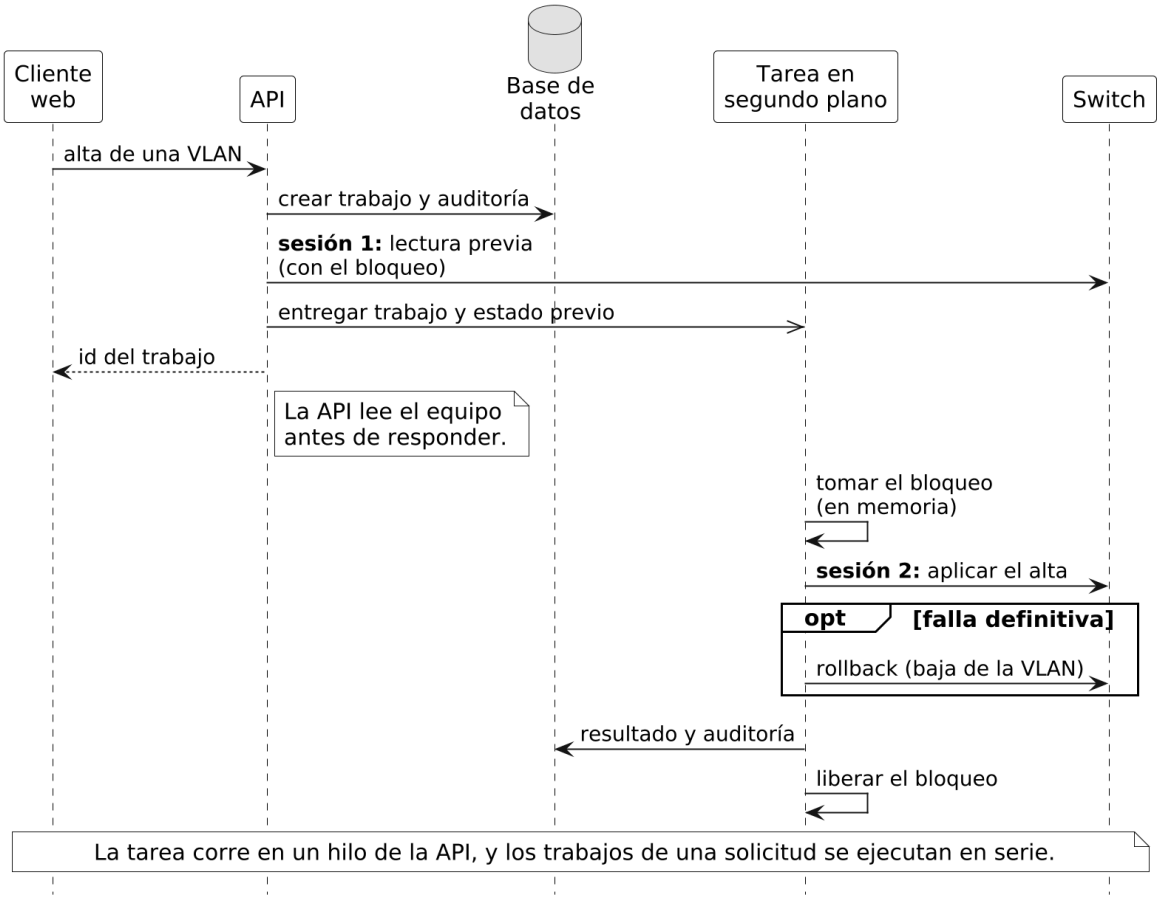


Figura 6.1: Secuencia del alta de una VLAN en la primera iteración.

Fuente: elaboración propia.

Las tareas en segundo plano eran las de FastAPI, que se ejecutaban en un hilo del mismo proceso de la API una vez enviada la respuesta. Por eso, el bloqueo de cada equipo y el límite de cinco trabajos por minuto se implementaron como estructuras en su memoria, protegidas por los cerrojos del módulo *threading* de Python. Cada tarea esperaba primero su turno según ese límite y luego tomaba el bloqueo. La contrapartida era que un reinicio de la API interrumpía los trabajos. Al arrancar, la API marcaba como fallidos los que quedaron en

ejecución, mientras que los que todavía esperaban su turno quedaban pendientes sin ejecutarse.

Con el estado previo, el trabajo decidía si hacía falta un cambio. Si la VLAN ya existía con el mismo nombre, terminaba como completado sin enviar comandos. Si existía con otro nombre, terminaba como fallido sin aplicar cambios. Si el estado previo no pudo leerse, porque el equipo no respondió o porque el bloqueo no se liberó a tiempo, el trabajo también fallaba sin tocar el equipo, ya que sin ese estado no era posible el *rollback*. Como el estado se leía en la solicitud y el cambio se aplicaba después, otra operación sobre el mismo equipo podía modificarlo en ese intervalo.

Si el cambio fallaba por un error transitorio, como un tiempo de espera agotado o una conexión rechazada, el trabajo lo reintentaba hasta tres veces, con esperas de uno, dos y cuatro segundos. Si aún así no se completaba, aplicaba el *rollback* con la operación inversa, es decir, borraba la VLAN creada, volvía a crear la eliminada o restauraba el nombre anterior. Luego registraba en el trabajo si la reversión se aplicó.

Funcionalidad "Autenticación, Gestión de Usuarios y Registro de Auditoría"

La autenticación usa dos *tokens*. El de acceso es un JWT firmado con una clave secreta del servidor. El de renovación es un valor aleatorio generado con el módulo *secrets* de Python, del que una tabla guarda sólo su resumen SHA-256, su vencimiento y si fue revocado. El cliente web lo recibe en una *cookie* que JavaScript no puede leer y que el navegador sólo envía a la propia aplicación, lo que la deja fuera del alcance de un script inyectado y de las solicitudes originadas en otros sitios. Cada intento de inicio de sesión se guarda con el usuario, la dirección IP, el resultado y el momento. Antes de verificar la contraseña, la API cuenta los intentos fallidos recientes de la cuenta y de la dirección IP, de modo que los bloqueos se conservan aunque el servidor se reinicie.

En esta iteración los roles se asignaban de forma global. La gestión de usuarios impide desactivar al último administrador activo, para que la plataforma nunca quede sin alguien capaz de administrarla.

Cada operación sobre los equipos dejaba dos entradas de auditoría. La API escribía la primera, en estado pendiente, al aceptar la solicitud. El trabajo agregaba la segunda al terminar, con el resultado y una referencia a la primera, de modo que la solicitud quedaba registrada aunque el trabajo no llegara a terminar. Un planificador dentro del proceso de la

API depuraba una vez por día las entradas que superan el período de retención. La API ofrecía además la consulta paginada del registro, filtrada por usuario, operación, recurso, equipo y fechas.

Funcionalidad "Interfaz Web"

El cliente web, construido con las tecnologías de la sección 5.7, ofrecía vistas de inicio de sesión, panel principal, VLAN, equipos, grupos de equipos, trabajos, auditoría y usuarios (Figura 6.2). La vista de VLAN permitía consultar y modificar las de cada equipo, y la de trabajos mostraba, al seleccionar uno, su estado, el estado previo, los reintentos, el error y el resultado del *rollback*.

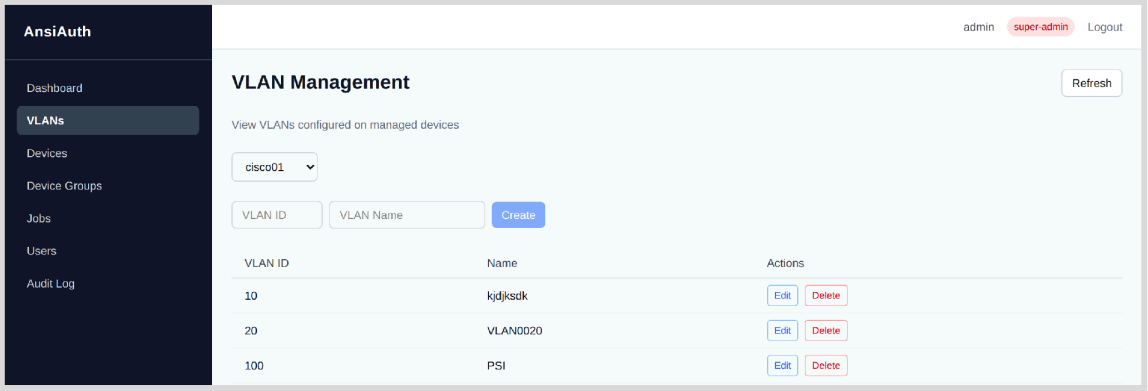


Figura 6.2: Interfaz web al cierre de la primera iteración: (a) Vista de VLAN

El *token* de acceso se guarda sólo en la memoria de la página, y al recargar la sesión se recupera con la *cookie* de renovación. Cuando el *token* vence, el cliente lo renueva y repite la petición, y si varias peticiones lo encuentran vencido a la vez, sólo una solicita la renovación, porque la API tomaría la segunda como una reutilización. Las acciones se ocultan según el rol del usuario. Al lanzar una operación, el cliente consulta cada pocos segundos el estado de sus trabajos mientras siguen activos y muestra una notificación al terminar, que distingue si el cambio se aplicó, si no hizo falta o si falló.

6.3. Pruebas y Validaciones

La estrategia de prueba combinó una suite automatizada en modo simulado con validaciones sobre el laboratorio emulado. La suite, escrita con pytest, ejercita la mayoría de las funcionalidades mediante peticiones HTTP autenticadas a la API sobre una base de datos aislada, y al cierre de la iteración reunía 409 pruebas en 27 archivos, que se ejecutaban en unos 35 segundos (Cuadro 6.2). Las pruebas de concurrencia envían varios trabajos sobre un mismo equipo y registran el orden en que la aplicación los ejecuta, para verificar que no

operen a la vez, y las de *rollback* reemplazan la ejecución sobre el equipo por dobles de prueba que fallan a propósito.

La validación en el laboratorio, emulado con GNS3, consistió en ejecutar cada operación sobre equipos Cisco IOS y Huawei VRP y contrastar la configuración resultante, leída en la CLI del equipo, con el estado registrado en el trabajo. Mediante guiones que invocan la API se ejercitaron además tres situaciones. La primera fueron fallas de acceso provocadas a mano, como una contraseña incorrecta, el SSH deshabilitado o el enlace del equipo desconectado, para comprobar que el trabajo terminara como fallido con el error registrado. La segunda fueron altas simultáneas sobre un mismo equipo, para confirmar que se ejecutaran de a una, y la tercera, altas de varias VLAN sobre dos equipos en una misma operación.

Área	Archivos	Pruebas	Aspectos verificados
Autenticación y protección del acceso	7	94	Emisión y vencimiento de tokens, rotación y detección de reutilización, bloqueo por cuenta y por dirección IP, límites de frecuencia, rol mínimo de cada ruta, TLS y creación del administrador inicial.
Gestión de usuarios	4	97	Alta, consulta, modificación y desactivación, almacenamiento de contraseñas y restricciones sobre el administrador del sistema.
Inventario de equipos	2	31	Registro, modificación con recifrado de la contraseña, baja y omisión de credenciales en las respuestas.
Gestión de VLAN	3	38	Validación de parámetros, idempotencia y operaciones sobre varios equipos.
Trabajos, reintentos y rollback	6	59	Ciclo de estados, filtrado y paginación, recuperación de trabajos interrumpidos, ejecución de a uno por equipo, clasificación de errores, reintentos y compensación de cada operación.
Auditoría	3	67	Registro de cada operación, filtros, rechazo de modificaciones, retención y depuración.
Aspectos transversales de la API	2	23	Formato uniforme de errores y verificación del estado del servicio.
Total	27	409	—

Cuadro 6.2: Pruebas automatizadas al cierre de la primera iteración, por área.

6.4. Resultados y Ajustes

Al cierre de la iteración quedaron operativas las funcionalidades comprometidas. Para comprobar su funcionamiento de punta a punta se dio de alta la VLAN 100, con nombre PSI, sobre un *switch* Cisco IOSvL2 y uno Huawei CE12800 del laboratorio, en una sola operación. La API generó un trabajo por equipo, y ambos terminaron completados, sin reintentos ni *rollback*, en 5,1 segundos el de Cisco y 2,2 segundos el de Huawei. En el registro de auditoría cada trabajo aparece primero como pendiente y luego como completado, junto con el estado previo leído del equipo y la duración de la ejecución (Figura 6.3), y la lectura posterior de cada equipo confirma que la VLAN quedó creada con el nombre indicado (Listado 6.2).

La prueba mostró también una limitación. Aunque los dos trabajos correspondían a equipos distintos, el de Huawei permaneció pendiente hasta que terminó el de Cisco, porque FastAPI ejecuta una tras otra las tareas en segundo plano de una misma petición, aunque las de peticiones distintas corren en paralelo.

```
# (a) Cisco IOSvL2: show vlan brief
Switch#show vlan brief
VLAN Name                             Status    Ports
---- -------------------------------- --------- ---------------------
1    default                          active    Gi0/0, Gi0/1, Gi1/2, ...
10   kjdjksdk                         active    Gi0/2, Gi1/0
20   VLAN0020                         active    Gi0/3, Gi1/1
100  PSI                              active
# (b) Huawei CloudEngine: display vlan
<HUAWEI>display vlan
VID  Type     Status  Property  MAC-LRN STAT    BC  MC  UC  Description
--------------------------------------------------------------------------------
   1 common   enable  default   enable  disable FWD FWD FWD VLAN 0001
  10 common   enable  default   enable  disable FWD FWD FWD VLAN 0010
 100 common   enable  default   enable  disable FWD FWD FWD PSI
```

Listado 6.2: Estado de las VLAN en cada equipo después del alta: (a) Cisco IOSvL2; (b) Huawei CloudEngine.

Timestamp	User	Action	Resource Type	Status	Details
5/10/2026, 21:12:21	admin	create_vlan	vlan	completed	{ "vlan_id": 100, "name": "PSI", "retries": 0, "rollback_performed": false, "duration_seconds": 2.16, "pre_state": {
5/10/2026, 21:12:19	admin	create_vlan	vlan	completed	{ "vlan_id": 100, "name": "PSI", "retries": 0, "rollback_performed": false, "duration_seconds": 5.13, "pre_state": {
5/10/2026, 21:12:12	admin	create_vlan	vlan	pending	{ "vlan_id": 100, "name": "PSI" }
5/10/2026, 21:12:09	admin	create_vlan	vlan	pending	{ "vlan_id": 100, "name": "PSI"

Figura 6.3: Entradas de auditoría del alta de la VLAN 100.

Fuente: elaboración propia.

Para estimar los tiempos de respuesta se ejecutó la versión de cierre de la iteración sobre el mismo laboratorio, en un equipo con un procesador Intel Core i5-1135G7, 16 GB de memoria y Ubuntu 24.04, con Python 3.12, ansible-core 2.20 y ansible-runner 2.4. Se realizaron 20 consultas de VLAN por equipo y 15 ciclos de alta, renombrado y baja sobre los dos equipos en una misma operación (Cuadro 6.3).

Operación	Respuesta de la API	Ejecución en Cisco	Ejecución en Huawei	Total de la operación
Consulta de VLAN (20)	—	2,6	2,3	—
Alta (15)	4,8	5,3	2,2	13,0
Renombrado (15)	4,9	5,9	2,1	13,3
Baja (14)	4,9	10,4	6,8	22,7

Cuadro 6.3: Tiempos de ejecución al cierre de la primera iteración, en segundos (mediana). Entre paréntesis, la cantidad de repeticiones.

Cada ejecución de ansible-runner demoraba entre 2 y 3 segundos aun para una lectura, porque iniciaba Ansible y abría una sesión SSH nueva. Ese costo se repetía en cada paso de una operación. La API tardaba cerca de 5 segundos en responder porque, antes de hacerlo, leía el estado previo de cada equipo. La baja era la operación más lenta, porque su trabajo volvía a leer el equipo dos veces, una para comprobar que la VLAN exista y otra para verificar que se haya eliminado. Las modificaciones en Cisco tardaban unos 3 segundos más que en Huawei, en buena parte porque su playbook era el único que guardaba la configuración al terminar.

Como los trabajos de una misma petición se ejecutaban uno tras otro, el tiempo total de una operación era la suma de las lecturas previas y de las ejecuciones en cada equipo. Sobre dos equipos, un alta tardaba unos 13 segundos y una baja unos 23, de los que cerca de dos tercios correspondían a lecturas. Estos tiempos crecían en proporción a la cantidad de equipos.

La ejecución sobre el laboratorio obligó, además, a corregir dentro de la iteración los siguientes problemas:

- **Inventarios aislados**: dos trabajos simultáneos sobre equipos distintos terminaban conectándose al mismo equipo, por lo que cada ejecución pasó a escribir su inventario en un archivo temporal propio.
- **Módulos de Huawei**: el módulo ce_vlan de community.network no funcionó con los equipos y se reemplazó por ce_config (Listado 6.1). Al cierre de la iteración, los *playbooks* de Huawei pasaron a cli_command de ansible.netcommon, porque el módulo de consulta esperaba una respuesta estructurada y el equipo devolvía texto plano.

- **Verificación del *rollback***: un error del *playbook* de compensación no garantiza que la reversión haya fallado, por lo que en ese caso el trabajo vuelve a leer el equipo y registra si la VLAN quedó en su estado previo.

En síntesis, la primera iteración estableció la comunicación con los equipos Cisco y Huawei del laboratorio y construyó sobre ella las funcionalidades base: el inventario, la gestión de VLAN con *rollback*, la autenticación, los usuarios, la auditoría y la interfaz web. La suite automatizada y el alta de punta a punta confirmaron su funcionamiento. Las mediciones mostraron también sus límites: cada ejecución de ansible-runner demoraba entre 2 y 3 segundos, las consultas abrían sesiones con el equipo y los trabajos de una operación se ejecutaban en serie. Por eso la segunda iteración incorporó un estado almacenado para las consultas, *workers* que ejecutan en paralelo los trabajos de equipos distintos y un orquestador único para no repetir el ciclo de un cambio.

7. Segunda Iteración: Extensión de los Recursos y Arquitectura Definitiva

7.1. Planificación y Diseño

La iteración tuvo dos objetivos. El primero fue sumar la configuración de los puertos físicos, las interfaces virtuales y los parámetros globales del equipo. El segundo fue llevar la plataforma a la estructura definitiva del Capítulo 5. En la primera iteración, cada consulta abría una sesión con el equipo, los trabajos de una solicitud se ejecutaban en serie dentro del proceso de la API y cada operación de VLAN repetía su propia copia del ciclo de un cambio. Sumar tres tipos de recurso sobre esta estructura habría multiplicado estos problemas.

Conforme a la priorización de requerimientos, se abordaron los puertos físicos y las interfaces virtuales, de riesgo alto por su dependencia del equipamiento, y la configuración global, de riesgo medio, junto con sus vistas en la interfaz web. Se retomaron además los requerimientos pendientes del inventario, la gestión de VLAN y la ejecución asíncrona. La reautenticación y la restricción de la auditoría por sitio se postergaron a la tercera iteración, porque dependen del modelo multisitio, y con ellas la gestión de las sesiones.

La iteración se organizó en tres etapas. En la primera se implementó la configuración de puertos sobre la estructura heredada, para validar cuanto antes el requerimiento de mayor riesgo. Esa etapa mostró que cada operación nueva exigía su propio playbook por fabricante y su propia copia del ciclo de un cambio, de modo que los playbooks, casi idénticos entre sí, pasaron de 9 al cierre de la primera iteración a 26. Por eso, la segunda etapa llevó los módulos de la plataforma a su forma definitiva (Cuadro 7.1) antes de abordar los recursos restantes, que se incorporaron en la tercera junto con la reducción de sesiones y la autenticación por clave.

El laboratorio sumó a los nodos Cisco IOSvL2 y Huawei CE12800 de GNS3 dos equipos físicos, un Cisco Catalyst WS-C3650 con IOS-XE y un Huawei S5731 con VRP V200R022. Fueron necesarios porque las imágenes emuladas no implementan algunas funciones de los puertos, como la alimentación PoE y el storm-control, y porque la sintaxis de VRP difiere entre familias de equipos.

Módulo (sección)	Cambio en esta iteración	Limitación de la primera iteración que resuelve
Ejecución asíncrona (5.5.3)	Workers independientes que toman los trabajos de una cola.	La ejecución compartía el proceso de la API, por lo que los trabajos de una misma solicitud corrían en serie y un reinicio de la API los interrumpía.
Coordinación (5.5.4)	Bloqueo por equipo, límite por equipo y semáforo global en un servicio compartido.	El bloqueo y el límite en memoria no se comparten entre procesos.
Orquestación (5.5.5)	Orquestador único sobre el contrato del recurso gestionable, con tres categorías de error.	El ciclo de un cambio se repetía en cada operación, el estado previo se leía en la solicitud y los errores se clasificaban solo en transitorios y permanentes.
Acceso a equipos (5.5.7)	Registro de controladores, vocabulario de comandos, un playbook genérico por fabricante y vía SSH directa para la autenticación por clave.	Un playbook por operación y fabricante, que con los puertos llegó a 26 casi idénticos. Cada variante de sintaxis exigía modificar el controlador y solo se admitía la contraseña.
Auditoría y eventos (5.5.8)	Eventos de dominio publicados en un despachador.	Cada servicio escribía directamente en el registro.
Sincronización (5.5.9)	Estado almacenado con lectura combinada.	Cada consulta abría una sesión con el equipo, y las lecturas eran cerca de dos tercios de la duración de una baja.
Persistencia (5.5.10)	PostgreSQL, con la inmutabilidad de la auditoría en la capa de persistencia.	SQLite admite un único escritor a la vez, y la inmutabilidad dependía de un disparador propio de ese motor.

Cuadro 7.1: Cambios estructurales de la segunda iteración y limitación que resuelve cada uno.

7.2. Implementación

Funcionalidad "Ejecución Asíncrona y Seguimiento de Trabajos"

Las tareas en segundo plano de la API se reemplazaron por workers de Celery con una cola en Redis, con una tarea por tipo de trabajo, como aplicar un cambio, sincronizar un equipo o reintentar un *rollback*. Como la cola sólo acepta mensajes en JSON, la API encola el recurso convertido en un diccionario junto con el identificador del trabajo, y el worker lo reconstruye según su tipo antes de entregarlo al orquestador. Cada contenedor de workers ejecuta cuatro procesos de Celery, cada uno con su propia conexión a la base, porque una conexión no puede compartirse entre procesos. Al escribir en la base desde varios procesos, la persistencia pasó de SQLite, que admitía un único escritor a la vez, a PostgreSQL.

La coordinación se trasladó de la memoria de la API a Redis. En esta iteración, el bloqueo por equipo era un lock de redis-py que vencía a los cinco minutos. El límite por equipo es un contador que admite 30 trabajos por minuto. El límite global es un semáforo de 20 lugares compartido por todos los workers, implementado como un conjunto ordenado con un elemento por ejecución en curso. Cada worker toma su lugar mediante un breve script en Lua, que Redis ejecuta como una sola operación, de modo que dos workers no pueden contar a la vez y ocupar el mismo lugar.

El ciclo de un cambio, que en la primera iteración se repetía en cada operación de VLAN, se extrajo a un orquestador único, sobre cuyo contrato se implementaron primero la

VLAN y luego los puertos, las interfaces virtuales y la configuración global. El orquestador pasó a publicar un evento al terminar cada trabajo en lugar de escribir en la auditoría, de modo que cada trabajo deja una única entrada y no dos, ya que la solicitud queda registrada en el propio trabajo. La inmutabilidad de la auditoría, que dependía de un disparador de SQLite, pasó a la capa de persistencia, que rechaza toda modificación de una entrada antes de enviarla a la base.

La clasificación de errores sumó la categoría de los desconocidos. Los códigos de salida con los que Ansible informa un equipo inalcanzable o una falla de conexión se tratan siempre como transitorios, antes de revisar el texto del error. La lectura del estado previo pasó a reintentarse con la misma política que la aplicación. Cada reintento vuelve a leer el equipo antes de aplicar, porque tras un tiempo de espera agotado no se sabe si el cambio llegó a aplicarse. El reintento manual de un *rollback* fallido crea un trabajo nuevo vinculado al original.

Funcionalidad "Playbook Genérico y Vocabulario de Comandos"

En el acceso a equipos, la condición sobre el fabricante se reemplazó por un registro de controladores, y los 26 playbooks se redujeron a uno genérico por fabricante (Listado 7.1), cuyas tareas el controlador activa según la variable que le pasa. Un cambio se envía como un único bloque de comandos, con la entrada al modo de configuración, la confirmación y la salida, de modo que se aplica completo en una sola sesión. Una lectura, en cambio, se envía como una lista de comandos que se ejecutan uno por uno sobre la misma conexión. La lectura tiene una versión estricta, en la que un comando rechazado hace fallar la tarea y se pierden también las respuestas de los demás, y una tolerante, que devuelve cada error como texto para que el controlador descarte sólo el comando no soportado, como el storm-control en las imágenes emuladas.

Los comandos pasaron a un vocabulario en YAML por fabricante (Listado 5.1), donde cada operación tiene su secuencia principal y sus alternativas, cada una asociada a una expresión regular sobre la respuesta del equipo. Con los tres recursos incorporados, Cisco reunía 48 operaciones y una sola alternativa, mientras que Huawei, con 47 operaciones, requería 76 alternativas repartidas en 41 de ellas. Además de las diferencias entre fabricantes, apareció una variabilidad dentro de VRP, entre sus familias de equipos. Los CloudEngine confirman los cambios con commit, que la serie S rechaza, lo que explicaba 51 alternativas. Otras 15 se debían a que una familia abrevia los nombres de interfaz y la otra los escribe

completos, y el resto, a comandos de la sesión y a funciones que no todos los modelos admiten. Todas se incorporaron editando el vocabulario, sin modificar los controladores.

```
# Huawei VRP: vendors/huawei/run.yml
- name: Huawei VRP device operation
  hosts: all
  gather_facts: no
  tasks:
    - name: Apply command block
      when: command_block is defined
      ansible.netcommon.cli_command:
        command: "{{ command_block }}"
    - name: Run read commands (strict)
      when: commands is defined and not (tolerate_command_errors | default(false) | bool)
      ansible.netcommon.cli_command:
        command: "{{ item }}"
      loop: "{{ commands }}"
    - name: Run read commands (tolerant)
      when: commands is defined and (tolerate_command_errors | default(false) | bool)
      ansible.netcommon.cli_command:
        command: "{{ item }}"
      loop: "{{ commands }}"
      failed_when: false
```

Listado 7.1: Playbook genérico de Huawei al cierre de la segunda iteración.

Funcionalidad "Transporte SSH y Autenticación por Clave"

Se incorporó la autenticación por par de claves y, con ella, la vía de transporte directa prevista en el diseño. En los equipos físicos del laboratorio se probaron las dos bibliotecas SSH de Python que puede usar Ansible. Paramiko, en dos versiones, no logró autenticar por clave contra su firmware. Ansible-pylibssh ni siquiera logró conectarse, porque por defecto rechaza los algoritmos antiguos que esos equipos exigen. OpenSSH, en cambio, lo hizo de forma consistente. El controlador elige la vía según el método de autenticación del equipo, y ambas devuelven el resultado con la misma forma, por lo que los controladores y los parsers no cambiaron. Volverla confiable requirió resolver cinco problemas:

- La autenticación quedaba en espera indefinida si la clave se pasaba como archivo, porque el cliente envía entonces la firma sin preguntar antes si el equipo acepta la clave, un mensaje que el firmware nunca responde. Por eso en cada sesión se inicia un agente SSH efímero que ofrece la clave, con el que el cliente consulta primero y firma recién cuando el equipo la acepta.
- Los cambios de configuración no se completan como un comando aislado, por lo que el bloque se envía línea por línea dentro de la sesión interactiva.
- Las lecturas extensas se detienen en el paginador del equipo, que se deshabilita en la primera línea de cada sesión.

- La transcripción incluye el banner y el eco de los comandos, que los parsers interpretaban como datos, por lo que se extrae sólo la salida entre el eco del comando y el cierre de la sesión.
- El cliente informa un error aunque la sesión haya terminado bien, porque el equipo no cierra la conexión de forma ordenada. Por eso se agrega al final quit en Huawei y exit en Cisco, y la sesión se da por completa si ese comando aparece en la transcripción.

Funcionalidad "Modelo de Lectura"

El estado almacenado se implementó con una tabla por tipo de información, de la que se resuelven las consultas. Una misma tarea de Celery sincroniza un tipo de información o la lectura combinada de VLAN, puertos e interfaces virtuales. En Cisco, la lectura combinada usa una sola sesión en lugar de tres, porque las interfaces virtuales se obtienen de la misma salida de configuración de interfaces que ya se pide para los puertos. En Huawei usa dos en lugar de cuatro, porque VRP no permite pedir juntas las configuraciones de todas las interfaces virtuales y hay que descubrir primero cuáles existen. Al registrar un *switch* se encola la lectura combinada, el usuario puede solicitar la de cada tipo por separado y cada cambio completado solicita la del tipo que modificó. Las tablas ARP y MAC y el registro local, que pueden ser extensos y no intervienen en ningún cambio, sólo se leen a pedido.

En esta iteración, tras un cambio, el orquestador guardaba primero el elemento modificado tal como lo describía la solicitud, para que la vista lo reflejara de inmediato, y la lectura posterior lo reemplazaba por el estado real. La configuración global era la excepción, porque se guarda como una sola fila con todos sus atributos y en ese caso se esperaba a la lectura. Si varios trabajos sobre el mismo equipo y tipo de recurso llegan seguidos, sólo el último solicita la lectura, porque antes de encolarla el orquestador comprueba en la base que no queden otros trabajos activos sobre ese equipo y tipo.

Funcionalidad "Configuración de Puertos Físicos"

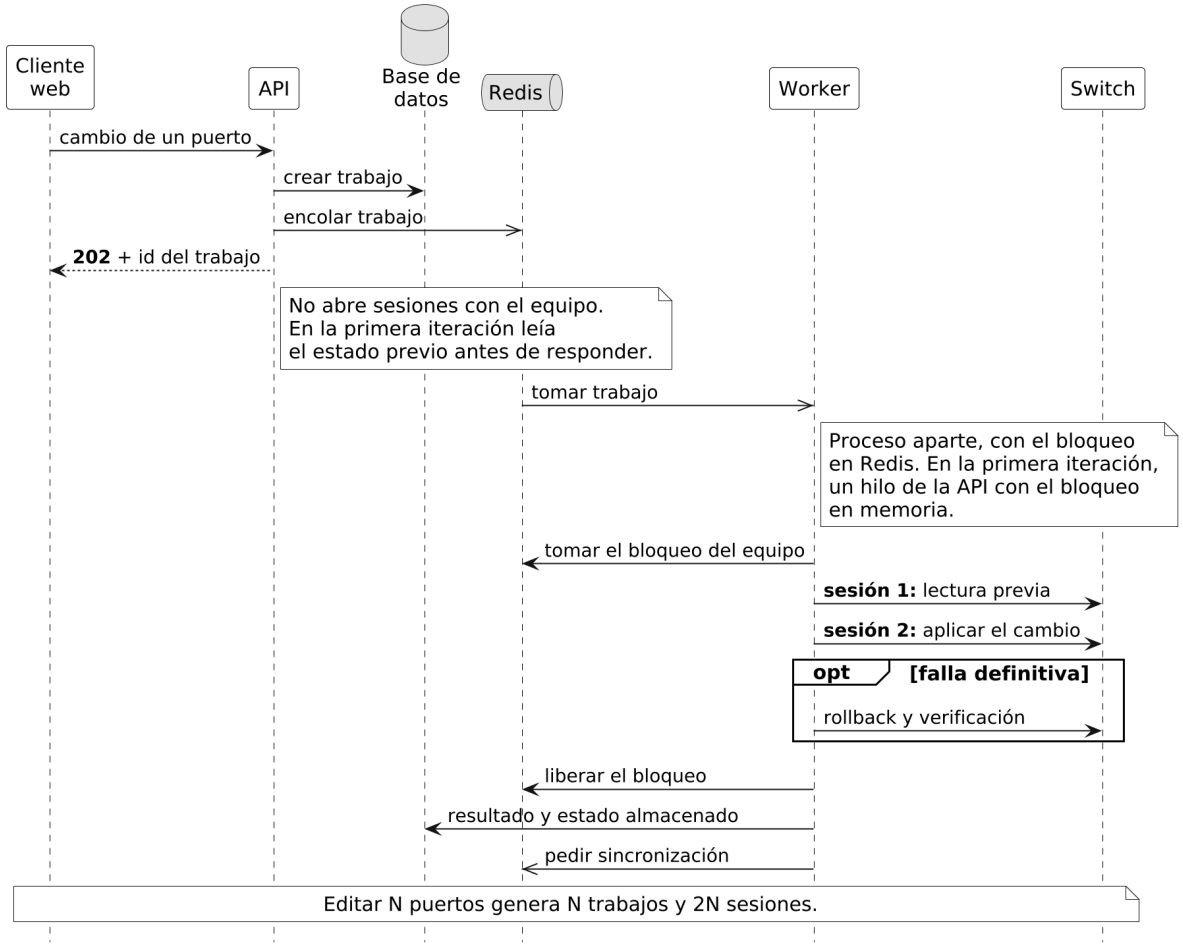


Figura 7.1: Secuencia de edición de puerto en la segunda iteración.

Fuente: elaboración propia.

La API permite consultar los puertos de un equipo con su modo, sus VLAN, su estado y su descripción, y ofrece una operación por tipo de cambio. Un puerto puede pasar a acceso, con su VLAN, o a troncal, con su VLAN nativa y sus VLAN permitidas. En un troncal se pueden agregar o quitar VLAN permitidas indicando sólo las que cambian. También se pueden modificar la descripción, el estado administrativo, la alimentación PoE y el storm-control, o restablecer la configuración por defecto del puerto. Antes de crear el trabajo, la API comprueba que la solicitud sea coherente, por ejemplo que un troncal incluya su VLAN nativa o que el umbral del storm-control esté entre 0 y 100. Después, el trabajo seguía el ciclo de un cambio, con una lectura previa y una sesión de aplicación por puerto (Figura 7.1). Una acción sobre varios puertos generaba así un trabajo por puerto, lo que motivó elevar el límite de trabajos por equipo y, en la tercera iteración, las operaciones en lote.

La validación sobre los equipos del laboratorio mostró cuatro diferencias de comportamiento que van más allá de la sintaxis:

- **VLAN inexistente:** asignada como VLAN de acceso, Cisco la crea con una advertencia, tratada como mensaje benigno para no revertir un cambio aplicado, mientras que Huawei rechaza el comando, tratado como error permanente para no reintentarlo. En la lista de permitidas de un troncal, Huawei sí la acepta.
- **Encapsulación troncal:** la imagen IOSvL2 exige fijar la encapsulación antes de pasar un puerto a troncal, lo que motivó la única alternativa del vocabulario de Cisco.
- **Storm-control:** la imagen IOSvL2 no lo implementa, y su rechazo hacía fallar la lectura completa de los puertos. Por eso la lectura usa la versión tolerante del playbook y deja el atributo como desconocido. Un cambio de storm-control sobre esa imagen, en cambio, falla sin reintentos, porque el rechazo se clasifica como permanente.
- **Confirmación al pasar a troncal:** en Huawei, el cambio de modo pide siempre confirmación. Sin ella, el equipo la tomaba como negativa y los comandos siguientes fallaban, dejando el puerto a medio configurar, por lo que el bloque incluye siempre la respuesta.

Funcionalidad "Configuración de Interfaces Virtuales y Configuración Global del Dispositivo"

Las interfaces virtuales admiten el alta y la baja asociadas a una VLAN, sus direcciones IPv4 primaria y secundarias y la IPv6, las ACL de entrada y de salida, y los servidores de reenvío DHCP, la descripción y el estado administrativo, con una operación por atributo. El alta crea la interfaz de la VLAN, Vlan en Cisco y Vlanif en Huawei, y la habilita en el mismo bloque. En Huawei hubo que resolver dos casos:

- **Reenvío DHCP:** la serie S asocia los servidores directamente a la interfaz, mientras que el CE12800 los reúne en un grupo vinculado a ella. El controlador intenta primero la forma directa y recurre al grupo mediante una alternativa del vocabulario. El parser reconoce ambas formas, ya que sin ello agregar un segundo servidor reemplaza al primero.
- **ACL con nombre:** para quitarla de la interfaz, VRP exige repetir cuál es, por lo que la operación la toma del estado leído antes del cambio.

La configuración global permite modificar el nombre del equipo, las rutas estáticas, los servidores NTP, DNS y syslog, la configuración SNMP y las ACL, y consultar la versión de *firmware*, la configuración en ejecución, las tablas ARP y MAC y el registro local. La búsqueda de una dirección MAC filtra la tabla almacenada del equipo. Los atributos que son listas, como las rutas o los servidores NTP, se modifican agregando o quitando un elemento, sin reenviar la lista completa.

La lectura reúne en una sola consulta la versión, el nombre, la tabla de rutas y la configuración en ejecución, de la que se extrae el resto de los atributos. El estado de SNMP se lee aparte, porque su comando falla cuando SNMP no está habilitado y haría fallar toda la consulta. La implementación se ajustó además a lo que mostraron los equipos:

- **Rutas estáticas:** la tabla de enrutamiento no incluye una ruta cuyo siguiente salto no es alcanzable, por lo que las rutas también se leen de la configuración en ejecución.
- **ACL:** sólo se crean y modifican las extendidas de Cisco y las avanzadas de Huawei, las únicas que admiten protocolo, destino y puertos, y las demás se muestran como de sólo lectura. Al revertir la baja de una regla, ésta se reinserta en su posición original, porque agregada al final quedaría detrás de una regla de rechazo general y nunca se aplicaría.
- **Servidores syslog:** la serie S de Huawei rechaza indicar la severidad de los mensajes en la misma línea que el servidor, una forma que el CE12800 sí acepta. Por eso el servidor se agrega solo y la severidad se fija en el canal que envía los mensajes a los servidores syslog, lo que funciona en ambas familias.

Funcionalidad "Interfaz Web"

El cliente web incorporó las vistas de puertos, interfaces virtuales y configuración global de cada equipo (Figura 7.2). La de puertos los mostraba en una tabla con filtros y aplicaba una acción por vez, como cambiar la descripción, el estado administrativo, el modo, las VLAN permitidas, PoE o storm-control, o restablecer el puerto. Si la acción se aplicaba a una selección de puertos, el cliente enviaba una solicitud por puerto y seguía cada trabajo por separado. La de interfaces virtuales permite crearlas, eliminarlas y modificar sus atributos. La de configuración global se organiza en seis pestañas: un resumen, desde el que se editan el nombre del equipo, SNMP, NTP, DNS y los servidores syslog, y las de rutas estáticas, listas de acceso, tablas ARP y MAC, registro local y configuración en ejecución. Al terminar un

trabajo, la notificación enlaza a su detalle, que muestra el cambio en términos legibles, los comandos aplicados y, si falló, el error junto con el comando que el equipo rechazó.

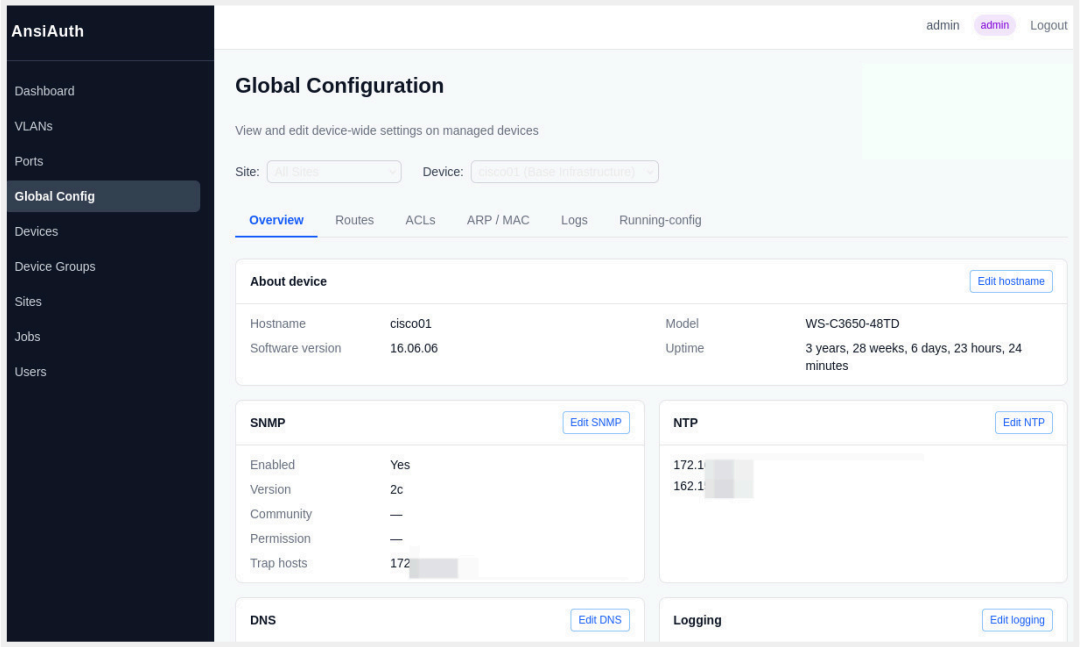


Figura 7.2: Interfaz web al cierre de la segunda iteración: vista de configuración global de un equipo. Fuente: elaboración propia.

7.3. Pruebas y Validaciones

La suite automatizada se adaptó a la nueva estructura. Las pruebas del ciclo de un cambio se reescribieron sobre el orquestador y el contrato del recurso, y la base de datos de pruebas pasó de SQLite a PostgreSQL, el mismo motor de producción. La cola se configura para ejecutar las tareas de forma síncrona. La iteración incorporó 400 pruebas en 34 archivos (Cuadro 7.2). Las del orquestador y del acceso a equipos emplean controladores de prueba que registran cada lectura y cada aplicación, lo que permite verificar, además del resultado, cuántas sesiones abre cada flujo, por ejemplo que la lectura combinada abre una sesión en Cisco y dos en Huawei.

La configuración global y las interfaces virtuales, cuya sintaxis depende en gran medida de cada familia de equipos, se validaron principalmente en el laboratorio, y sus pruebas se ampliaron en la tercera iteración. La configuración global contó en esta iteración con 10 pruebas incluidas en el área de orquestación, sobre sus *rollbacks* y el resumen legible de sus operaciones.

Área	Archivos	Pruebas	Aspectos verificados
Configuración de puertos	14	220	Rol mínimo, validación de parámetros, modos acceso y troncal, VLAN permitidas por reemplazo, suma y quita, descripción, estado administrativo, control de tormentas, confirmación interactiva de Huawei y parsers de ambos fabricantes.
Orquestación, reintentos y rollback	6	86	Clasificación de errores, esperas entre reintentos, resumen legible de la operación, rollback de cada tipo de recurso y su reintento manual, aislamiento de fallas y liberación del bloqueo.
Ejecución distribuida y seguimiento	6	55	Despacho a la cola, ejecución por equipo con aislamiento de fallas, semáforo global, agrupación de los trabajos de una operación sobre varios equipos, resumen de ejecución y depuración periódica.
Acceso a equipos	5	24	Comandos enviados por operación, tolerancia a comandos no soportados y resolución del controlador real o simulado.
Modelo de lectura	2	8	Sesiones de la lectura combinada por fabricante, toma única del bloqueo y persistencia parcial ante errores.
Interfaces virtuales	1	7	Alta y baja de direcciones IPv4 secundarias.
Total	**34**	**400**	—

Cuadro 7.2: Pruebas automatizadas incorporadas en la segunda iteración, por área.

En el laboratorio se ejecutó cada operación sobre las plataformas que la soportan y se contrastó la configuración resultante, leída en la CLI, con el trabajo registrado. Se evaluaron además escrituras consecutivas sobre un mismo equipo, para confirmar que la segunda espera el bloqueo en lugar de fallar, fallas inducidas para verificar el *rollback*, y la selección de alternativas del vocabulario ante las variantes de sintaxis.

7.4. Resultados y Ajustes

Las funcionalidades implementadas quedaron operativas en el laboratorio, con las excepciones propias de cada plataforma, como la alimentación PoE y el storm-control en los nodos emulados. El Cuadro 7.3 resume lo atendido y lo que quedó pendiente. Los tres recursos nuevos quedaron completos, salvo el aviso al modificar un puerto que transporta la VLAN de gestión. Entre los requerimientos retomados, el estado almacenado completó la consulta de VLAN y del inventario, y la ejecución asíncrona sumó los errores desconocidos y el reintento manual del *rollback*, sólo para puertos e interfaces virtuales. El guardado de la configuración, en cambio, se retiró al adoptar el orquestador único. Quedó pendiente la omisión del *rollback* cuando el cambio no llegó al equipo. En los no funcionales, el límite global de sesiones quedó aplicado sólo a las ejecuciones de Ansible.

Requerimiento		Atendido en esta iteración	Pendiente para la tercera
Nuevos	RF5: Puertos	Consulta y configuración del modo, las VLAN permitidas y los atributos de cada puerto	Aviso al modificar un puerto que transporta la VLAN de gestión
	RF6: Interfaces virtuales	Alta y baja, direcciones IP, ACL, reenvío DHCP y atributos	—
	RF7: Configuración global	Parámetros globales, consultas de diagnóstico y búsqueda de direcciones MAC	—
Retomados	RF1: Autenticación	—	Gestión de sesiones activas y reautenticación de operaciones sensibles
	RF3: Inventario	Autenticación por par de claves, lectura del estado al registrar y actualización manual	Guardado de la configuración, retirado al adoptar el orquestador único
	RF4: VLAN	Consulta desde el estado almacenado	—
	RF8: Trabajos	Errores desconocidos, actualización del estado tras un cambio y reintento manual del rollback en puertos e interfaces virtuales	Omisión del rollback cuando el cambio no llegó al equipo
	RF9: Auditoría	Filtro por resultado	Filtro por sitio y auditoría restringida al alcance
	RF10: Interfaz web	Vistas y acciones de los recursos de esta iteración	Vistas y acciones de la tercera iteración
No funcionales	RNF1: Protección del acceso	—	Vencimiento por inactividad, duración máxima de la sesión y reautenticación
	RNF3: Tolerancia a fallas	Bloqueo por equipo, límite de trabajos por equipo, límite global de ejecuciones de Ansible y reintentos acotados del rollback	Límite global para la vía SSH directa
	RNF4: Trazabilidad	Auditoría inmutable en la persistencia	—
	RNF5: Uso de sesiones	Consultas desde el estado almacenado y lectura completa en dos sesiones como máximo	Pausa de las consultas periódicas con la vista oculta
	RNF6: Extensibilidad	Incorporación de fabricantes sin modificar lo existente	—
	RNF7: Operación sin CLI	Resumen legible de cada trabajo y de su error	—

Cuadro 7.3: Requerimientos del sistema atendidos en la segunda iteración.

Para comparar con la primera iteración se repitió su medición sobre la versión de cierre de esta, en el mismo equipo y laboratorio, con 20 consultas por equipo y 15 ciclos de alta, renombrado y baja sobre los dos equipos en una misma operación. Cada sesión con el equipo seguía implicando iniciar Ansible y abrir una conexión SSH nueva, por lo que la mejora provino de reducir la cantidad de sesiones (Cuadro 7.4) y de ejecutar en paralelo los trabajos de equipos distintos (Cuadro 7.5):

- **Consultas sin sesiones:** se resuelven desde el estado almacenado en unos 30 ms, frente a los 2 a 3 segundos de leer el equipo. La lectura del equipo pasó a la sincronización, que corre en segundo plano y sólo cuando hace falta.
- **Respuesta inmediata:** la API responde en unos 35 ms, porque la lectura del estado previo pasó de la solicitud al trabajo. Por eso cada trabajo duraba más que en la primera iteración, ya que pasó a incluir esa lectura.

- **Bajas más cortas:** la baja pasó de cuatro a dos sesiones, porque la lectura previa verifica también la existencia y la verificación posterior la realiza la sincronización, fuera del trabajo, en unos 2,4 s.
- **Ejecución en paralelo:** los trabajos de una operación sobre varios equipos los toman workers distintos, por lo que la operación termina con el más lento. Sobre dos equipos, un alta pasó de 13,0 a 6,1 segundos y una baja de 22,7 a 6,1 segundos.

Operación	Primera iteración	Segunda iteración
Consulta de VLAN	1 por consulta	0 (estado almacenado)
Consulta de puertos o interfaces virtuales	—	0 (estado almacenado)
Alta o renombrado de VLAN	2	2 (lectura previa y aplicación)
Baja de VLAN	4	2 (lectura previa y aplicación)
Modificación de N puertos de un equipo	—	2N (un trabajo por puerto, cada uno con lectura previa y aplicación)
Lectura completa del equipo (VLAN, puertos e interfaces virtuales)	—	1 en Cisco, 2 en Huawei
Actualización del estado tras un cambio	—	Una sincronización del tipo de recurso modificado, fuera del trabajo (1 sesión, salvo las interfaces virtuales en Huawei y la configuración global, que requieren 2)

Cuadro 7.4: Sesiones SSH por operación en la primera y la segunda iteración.

Operación	Respuesta de la API	Ejecución en Cisco	Ejecución en Huawei	Total de la operación
Consulta de VLAN (20)	—	2,6 → 0,03	2,3 → 0,03	—
Alta (15)	4,8 → 0,04	5,3 → 6,0	2,2 → 4,4	13,0 → 6,1
Renombrado (15)	4,9 → 0,04	5,9 → 5,9	2,1 → 4,3	13,3 → 6,1
Baja (15)	4,9 → 0,03	10,4 → 5,8	6,8 → 4,4	22,7 → 6,1

Cuadro 7.5: Tiempos de ejecución en la primera y la segunda iteración, en segundos (mediana). Entre paréntesis, la cantidad de repeticiones.

Durante la iteración se aplicaron cinco ajustes:

- **Límite por equipo:** con el límite heredado de cinco trabajos por minuto, aplicar una misma acción a más de cinco puertos de un equipo, que genera un trabajo por puerto, demoraba hasta un minuto los trabajos a partir del sexto. Por eso se elevó a 30, valor que sigue frenando un ciclo descontrolado sin afectar el uso normal.
- **Espera del bloqueo:** un cambio sobre un puerto o una interfaz virtual de un equipo ocupado se rechazaba en la API con el código 409. Como el usuario no puede saber cuándo se liberará el *switch*, se optó por aceptarlo y dejar que el trabajo espere.
- **Estado almacenado de la configuración global:** guardar el resultado de un cambio sobrescribía con valores vacíos el resto de los atributos, por lo que el orquestador pasó a solicitar su sincronización, que hasta entonces no se disparaba tras estos cambios.

Separar además las tablas ARP y MAC en una tabla y una sincronización propias redujo la lectura de la configuración global de unos 21 segundos a 15 segundos.

- **Ancho de la terminal en Huawei:** en la serie S, las líneas extensas, como los destinos de notificaciones SNMP o las reglas de una ACL, se corrompían por las secuencias de control que la terminal inserta al superar su ancho. Se amplió el ancho al iniciar las sesiones de esas operaciones con un comando que no persiste en la configuración y que, como los CloudEngine no lo reconocen, se omite en ellos mediante una alternativa.
- **Guardado de la configuración:** en Cisco, cada playbook de escritura guardaba la configuración al terminar, lo que explica buena parte de su mayor duración en la primera iteración. Al unificar los playbooks, el guardado dejó de ejecutarse tras cada cambio en ambos fabricantes. La operación de guardado, que no correspondía a ningún recurso gestionable, se retiró de la API hasta reincorporarse como una operación explícita en la tercera iteración.

En síntesis, la segunda iteración extendió la plataforma a los puertos físicos, las interfaces virtuales y la configuración global, y la llevó a la estructura de módulos del Capítulo 5: workers con coordinación en Redis, un orquestador único, un vocabulario de comandos con alternativas, una vía SSH directa para la autenticación por clave y un estado almacenado de los equipos. La suite incorporó 400 pruebas. La validación mostró una variabilidad considerable entre familias de un mismo fabricante, con 76 alternativas en el vocabulario de Huawei frente a una en Cisco, que se incorporaron sin modificar los controladores. En rendimiento, las consultas pasaron a resolverse en milisegundos y una operación sobre dos equipos bajó de entre 13 y 23 segundos a unos 6 segundos.

Quedaron para la tercera iteración (Cuadro 8.1) tres funcionalidades. La organización en sitios y grupos cambia la autorización de todas las operaciones. Las operaciones en lote y el tablero respondían a que editar N puertos abría 2N sesiones (Cuadro 7.4) y a que una operación no podía abarcar un grupo o un sitio completo. Faltaban además cuatro ajustes, que eran un bloqueo que se renovara mientras el trabajo siguiera activo, una espera acotada para obtenerlo, un semáforo que cubriera también la vía SSH directa y una sincronización que no abriera sesiones innecesarias. Por último, la variabilidad de VRP anticipaba variantes nuevas en el parque en producción, de cuya validación dependían tres de los pendientes del Cuadro 7.3.

8. Tercera Iteración: Arquitectura Multisitio, Lote y Cliente Web

8.1. Planificación y Diseño

El objetivo de esta iteración fue completar el alcance del sistema a partir de las limitaciones de la segunda. Había que organizar los equipos en sitios y grupos, de modo que una misma operación pudiera aplicarse a todo un grupo o sitio, como crear una VLAN en todos los *switches* de un grupo. También había que mantener actualizado el estado almacenado sin abrir sesiones innecesarias y validar la plataforma sobre los equipos en producción.

Conforme a la priorización de requerimientos, se abordaron la organización multisitio y los permisos por alcance, de riesgo alto por su integración con la autorización de todas las operaciones existentes, y las operaciones en lote, de riesgo medio. Se sumaron la sincronización del estado y el tablero, de riesgo bajo y muy bajo, junto con el aislamiento entre alcances en la base de datos. Se retomaron además los requerimientos pendientes de las iteraciones anteriores.

Módulo (sección)	Cambio en esta iteración	Limitación de la segunda iteración que resuelve
Organización y acceso (5.5.2)	Sitios y grupos con roles por alcance, resueltos una vez por solicitud.	Los roles eran globales, por lo que todo usuario operaba sobre todo el inventario.
Persistencia (5.5.10)	Políticas de seguridad a nivel de fila y contexto de sistema para los workers.	La autorización se verificaba sólo en la API.
Ejecución asíncrona (5.5.3)	Operaciones sobre un grupo o un sitio, con un trabajo por equipo y un resultado agregado.	Una operación sólo abarcaba equipos elegidos uno por uno.
Orquestación (5.5.5)	Varios cambios de un equipo sobre VLAN, puertos o interfaces virtuales aplicados en un único trabajo, con una lectura previa y una sola sesión.	Cada cambio era un trabajo con su propia lectura y su propia sesión, por lo que editar N puertos abría N trabajos y 2N sesiones, y crear N VLAN en M equipos, N·M trabajos.
Orquestación (5.5.5)	Rollback por estado, que revierte sólo lo que sigue aplicado, y omisión del rollback cuando el cambio no llegó al equipo.	El rollback restauraba el estado previo a ciegas, por lo que podía deshacer un cambio posterior de otro usuario, y se ejecutaba aunque el cambio no hubiera llegado al equipo.
Coordinación (5.5.4)	Bloqueo con renovación y espera acotada, semáforo para ambas vías de transporte y unión de sincronizaciones.	El bloqueo vencía a los cinco minutos aunque el trabajo siguiera activo, la espera no tenía límite y la vía directa no pasaba por el semáforo.
Sincronización (5.5.9)	Lectura de los equipos desactualizados al abrir una vista y barrido periódico opcional.	El estado almacenado sólo se actualizaba al registrar un equipo, tras un cambio o a pedido.
Interfaz (5.6)	Cliente organizado por alcance, con el árbol de sitios, grupos y equipos y un tablero por nivel.	Las vistas eran por equipo.

Cuadro 8.1: Cambios de la tercera iteración y limitación que resuelve cada uno.

La iteración se organizó en tres etapas. La primera migró el modelo de datos y la autorización al esquema multisitio, porque las operaciones sobre grupos y sitios, el tablero y la navegación del cliente web se definen sobre esa jerarquía. La segunda construyó sobre esa base las operaciones en lote, la sincronización del estado, el tablero, la gestión de sesiones y el nuevo cliente web, organizado por alcance. La tercera desplegó la plataforma en producción y la validó sobre el parque real, lo que motivó los criterios de cambios seguros y buena parte de los ajustes finales. El Cuadro 8.1 resume los cambios de diseño de la iteración y la limitación de la segunda que resuelve cada uno.

8.2. Implementación

Funcionalidad "Organización Multisitio y Permisos por Alcance"

La migración al modelo multisitio modificaba a la vez el esquema, los datos ya cargados y la autorización de todas las rutas. Para no hacerla de una sola vez, se dividió en siete fases que se aplican y verifican por separado (Cuadro 8.2). Primero se amplía el esquema sin modificar lo existente, y lo anterior se elimina recién en la última fase, cuando lo nuevo ya está en uso. Así se conservan los equipos, usuarios y registros de las iteraciones anteriores, y hasta esa última fase cualquier paso puede revertirse.

En la jerarquía de la sección 5.5.2, el sitio, su grupo por defecto y la referencia entre ambos se crean en una misma transacción, para que una falla no deje un sitio sin grupo. Al iniciar, la API crea el sitio de infraestructura base o completa su grupo si un arranque anterior se interrumpió. Un administrador de sitio sólo puede crear usuarios con una asignación inicial en su sitio, porque sin ella quedarían fuera de su alcance y no podría gestionarlos.

La autorización hacía inicialmente una consulta a la base por cada verificación, de modo que listar N equipos implicaba N consultas. Se reemplazó por el alcance del usuario, que el repositorio de asignaciones construye con una única consulta como una lista de sitio, grupo y rol, y sobre el que cada verificación se resuelve en memoria. El rol efectivo era al principio el de la asignación más específica, pero así un administrador de sitio con un rol menor sobre uno de sus grupos perdía sus permisos sobre ese grupo sin advertirlo. Por eso se adoptó el mayor de los roles aplicables. Los traslados exigen permiso en el origen y en el destino, lo que está fuera del alcance responde como inexistente y la auditoría se filtra por alcance.

Fase	Cambio	Efecto sobre el sistema en servicio
0. Preparación	Principios comunes, línea base de pruebas y respaldo de la base.	Ninguno.
1. Esquema aditivo	Tablas y columnas nuevas, todas opcionales, y el sitio de infraestructura base.	Ninguno, porque nada las usa todavía.
2. Migración de datos	Grupo por defecto en cada sitio, asignación de cada equipo a un grupo y roles globales convertidos en asignaciones por sitio, de forma idempotente y verificada al finalizar.	Ninguno.
3. Corte de la aplicación	Servicios, autorización y rutas nuevas detrás de un indicador de configuración.	Solo con el indicador activo.
4. Imposición	Indicador activo por defecto, restricciones de integridad en la base y cliente web adaptado.	Primera versión incompatible con la anterior.
5. Limpieza	Eliminación de columnas, tablas, rutas y código heredados.	Sin vuelta atrás.
6. Aislamiento	Políticas de seguridad a nivel de fila.	Refuerzo de la autorización de las lecturas en la base.

Cuadro 8.2: Fases de la migración al modelo multisitio.

Las políticas de seguridad a nivel de fila se instalaron sobre las tablas de sitios, grupos, equipos y auditoría. Al abrir cada transacción, un evento de SQLAlchemy informa a la base el usuario y si es administrador del sistema. Esos valores rigen sólo en esa transacción, para que no pasen a otra solicitud que reutilice la conexión. Las políticas de lectura los comparan con las asignaciones de rol. La de sitios oculta además el sitio de infraestructura base.

Las políticas también afectan a las escrituras, porque PostgreSQL rechaza cualquier escritura que ninguna política habilite. Como las escrituras ya las autoriza la API, se agregaron políticas de escritura que admiten todo. Se definió una por operación, para el alta, la modificación y la baja, y no una general, porque PostgreSQL admite lo que permita cualquiera de las políticas de una tabla. Una política general que admitiera todo habría alcanzado también a las lecturas y anulado su restricción.

Por defecto, el dueño de las tablas queda exento de las políticas, por lo que se activó FORCE ROW LEVEL SECURITY para que también lo alcancen. Como no actúan en nombre de un usuario, los workers se identifican ante la base como procesos del sistema, mediante la señal de inicio de cada proceso de Celery, que la clase base de las tareas repite en cada ejecución.

Funcionalidad "Operaciones en Lote"

Es habitual combinar varios cambios sobre un mismo equipo, como editar varios puertos o varios atributos de una interfaz virtual. En VLAN, puertos e interfaces virtuales,

esos cambios se aplican ahora como un único trabajo, de modo que editar N puertos de un equipo pasó de N trabajos y 2N sesiones a un trabajo y dos sesiones (Figura 8.1).

Tras la lectura previa, el orquestador descarta las entradas que ya están aplicadas, y si no queda ninguna, el trabajo termina sin abrir la sesión de aplicación. El controlador arma los comandos de las restantes a partir del vocabulario y los envía en una sola conexión. En Ansible, una tarea del playbook genérico los recorre reutilizando la conexión, y en la vía directa se concatenan en la misma sesión. Si el equipo rechaza el lote, cada paso que tenga una alternativa para ese error la usa y el lote se reenvía completo. Cada tipo de recurso lo aprovecha de la siguiente forma:

- **VLAN:** una solicitud combina altas, renombres y bajas de varias VLAN y las aplica en una sesión en cada equipo elegido. Si incluye una baja, exige el rol de administrador sobre todos ellos, como la baja individual.
- **Puertos:** los atributos de un puerto, como el modo, las VLAN, la descripción, el estado, el storm-control y PoE, se aplican en un único trabajo. Al elegir puertos de varios equipos, el cliente envía una solicitud por equipo con todos sus puertos. El restablecimiento de la configuración queda fuera del lote, porque restablece todos los valores del puerto. Como todas las entradas se calculan contra la misma lectura previa, dos cambios sobre las VLAN de un mismo puerto podrían contradecirse, por ejemplo agregar y quitar la misma VLAN, por lo que la API rechaza el lote y pide unirlos en una sola entrada.
- **Interfaces virtuales:** los atributos de una misma interfaz forman un único trabajo. Cada entrada se valida además contra lo que fijan las demás, porque agregar un DHCP relay junto con la primera dirección IPv4 de la interfaz fallaba al compararse con la lectura previa, que todavía no tenía la dirección.

Las operaciones sobre varios equipos crean un trabajo por equipo. Su resultado no se guarda aparte, sino que se calcula a partir de esos trabajos cada vez que se consulta, por lo que siempre coincide con lo ocurrido en cada equipo. En el servidor, esa operación sobre varios equipos existe sólo para las VLAN. En los puertos, las interfaces virtuales y la configuración global, el cliente envía una solicitud por equipo y muestra al terminar el resultado de cada una.

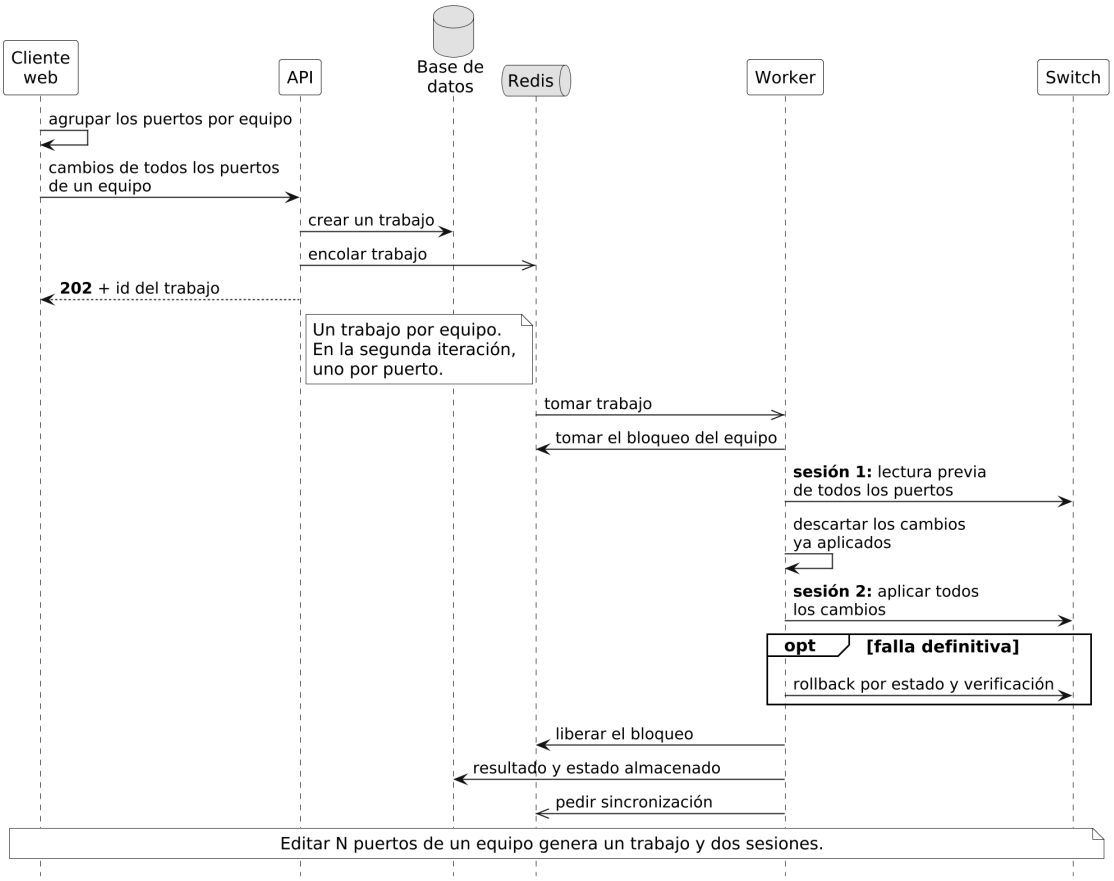


Figura 8.1: Secuencia de una edición conjunta de puertos en la tercera iteración.

Fuente: elaboración propia.

Funcionalidad "Autenticación de Usuarios"

Los límites de sesión se aplican en el servidor al renovar el *token*. Cada renovación hereda el identificador y el inicio de la sesión original, así que la inactividad se mide desde el último uso y la duración máxima desde el inicio. Vencer por inactividad revoca sólo esa sesión, porque, a diferencia de un *token* reutilizado, no indica un robo. El cliente web comparte la última actividad entre pestañas mediante BroadcastChannel y el almacenamiento local, para no cerrar una sesión en uso desde otra, propaga el cierre de sesión y avisa 30 segundos antes del cierre por inactividad. La lista de sesiones activas agrupa las renovaciones de un mismo inicio y permite cerrar todas excepto la actual.

Las operaciones sensibles, como eliminar un sitio o revocar un permiso, exigen reautenticarse. Si falta el *token* elevado, el servidor responde con un código propio, ante el cual el cliente pide la contraseña y reintenta la solicitud sin que el usuario deba repetirla. Ese *token* se guarda sólo en memoria, se reutiliza mientras le queden más de 30 segundos de

validez y lleva un tipo distinto del de acceso, para que ninguno pueda usarse en lugar del otro.

Funcionalidad "Sincronización del Estado y Tablero"

Al entrar a un sitio, un grupo o un equipo, el cliente pide una sola vez que se relean los equipos con algún dato leído hace más de 15 minutos. No lo hace en la vista de la organización, que releería todo el inventario en cada inicio de sesión, ni con la configuración global, la lectura de mayor carga para el *switch*, que se lee sólo a pedido. Si varios usuarios abren la misma vista, sus pedidos se unen mediante una clave en Redis por equipo y tipo de información, que sólo se crea si no existe y que la tarea borra al comenzar. Así se evita una lectura por cada usuario, del mismo modo que la unión de la segunda iteración evitaba una por cada trabajo.

Para no tener que consultar cada equipo por separado, el tablero reúne la cantidad de equipos, VLAN, puertos, interfaces virtuales y trabajos de un sitio, un grupo o toda la organización. El servidor los calcula en una única consulta sobre el estado almacenado, cuando antes el cliente pedía los datos de cada equipo y los sumaba. El tablero señala además lo que requiere atención, como las lecturas fallidas, una VLAN con nombres distintos según el equipo, los puertos caídos y los trabajos fallidos. En sitios y grupos, la configuración global de cada equipo se muestra lado a lado, de modo que se detecta a simple vista uno que no siga la del resto.

Mientras una lectura está en curso, las vistas consultan su estado cada 8 segundos en lugar de cada 2, porque una lectura demora varios segundos y consultar más seguido sólo repetía la misma respuesta. El seguimiento de los trabajos mantuvo su intervalo de 2,5 segundos, porque el usuario espera ver el resultado apenas termina. Además, las consultas se suspenden mientras la pestaña del navegador no está visible.

Funcionalidad "Interfaz Web"

El cliente web se reorganizó según el mapa de navegación de la sección 5.6.1 (Figura 8.2). Cada nivel tiene sus propias rutas, con las mismas pestañas de resumen, VLAN, puertos, interfaces virtuales y configuración global, y cada pestaña es un único componente que recibe el alcance como parámetro, de modo que la vista de VLAN de un sitio, un grupo o un equipo es el mismo código. Lo que cambia por nivel es su marco, con la ruta de navegación, las

acciones propias del nivel y la lectura al abrirlo. Un árbol lateral muestra los sitios, grupos y equipos del alcance del usuario, con un buscador de equipos.

La vista de puertos, que en la segunda iteración era una tabla con una solicitud por puerto, pasó a dibujar cada equipo como un chasis, con los puertos ordenados por módulo y coloreados según su estado. Las interfaces agregadas (Eth-Trunk y Port-channel), que no ocupan una posición física en el equipo, se muestran aparte. Los puertos que se eligen en el chasis se editan juntos en un único formulario por pestañas, que envía un lote por equipo (Figura 8.1).

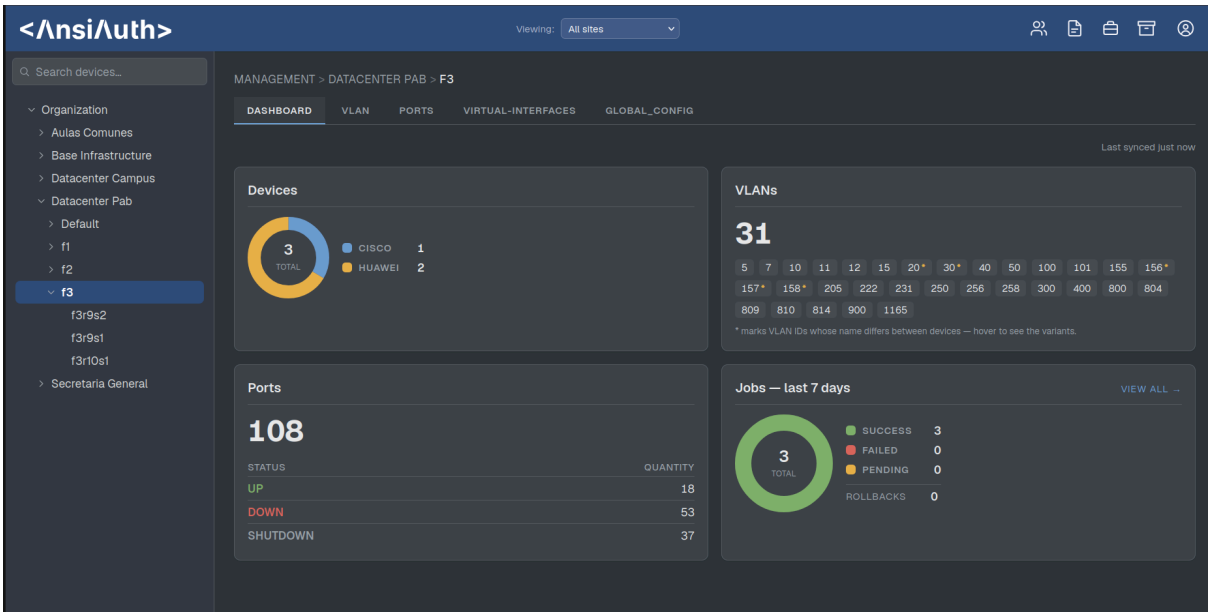
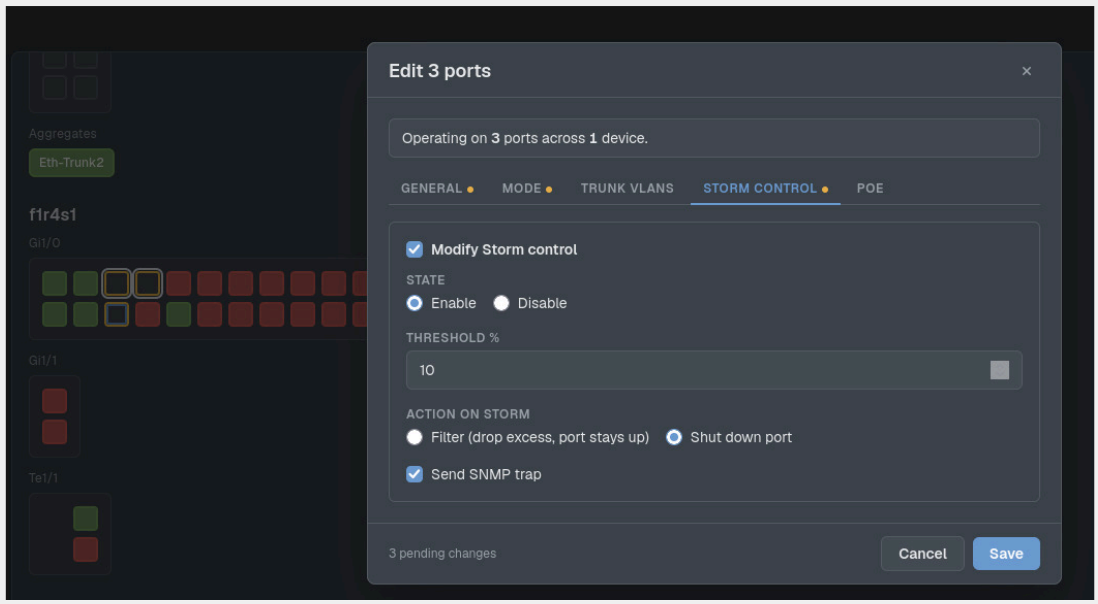


Figura 8.2: Cliente web al cierre de la tercera iteración: (a) tablero de un sitio

Para ocultar las acciones que exceden el rol del usuario, el cliente calcula ese rol en cada alcance con la misma regla que la API y lo compara con una copia de la tabla de rol mínimo por operación, que sólo decide qué botones se muestran, porque la API vuelve a verificar cada solicitud. La gestión de usuarios muestra los permisos de cada uno por alcance y permite asignarlos y revocarlos. Un selector de sitio filtra el inventario, la auditoría, los trabajos y los usuarios entre los sitios que la API devolvió como visibles.



(b) vista de puertos y edición conjunta. Fuente: elaboración propia.

Funcionalidad "Cambios Seguros sobre Equipos"

Operar sobre equipos en producción exigió revisar cómo se aplican y revierten los cambios. Un caso analizado durante la validación lo ilustra. Para agregar una VLAN a un troncal, en Huawei el sistema enviaba un comando que quitaba todas las VLAN permitidas, seguido de la lista completa. Como el equipo aplica cada línea de inmediato, si la segunda era rechazada, el troncal quedaba sin VLAN y el equipo podía quedar inaccesible. A partir de ese caso se aplicaron tres criterios:

1. **Cambios incrementales**: los cambios sobre un troncal envían sólo las VLAN permitidas que se agregan o se quitan, y la nativa si cambia, en lugar de borrar la lista y volver a escribirla. En Huawei, por ejemplo, undo port trunk allow-pass vlan 30 quita sólo la VLAN 30. Las VLAN que se permiten van antes que la nativa y las que se quitan, después, para que la nativa anterior siga permitida hasta ser reemplazada. Lo mismo se aplicó a los servidores de DHCP relay de Huawei, que antes se quitaban y volvían a configurar todos para cambiar uno. Ahora se agrega o quita sólo el que cambia, salvo cuando están en un grupo compartido con otras interfaces, porque modificar el grupo afectaría a todas.
2. ***Rollback* por estado**: el *rollback* de la sección 5.5.5 se aplicó a cada atributo, y en los cambios incrementales consiste en el inverso de la propia diferencia, sólo sobre lo que sigue presente. Se omite cuando ningún intento de aplicación llegó a conectarse con el equipo.

3. **Aviso sobre el acceso de gestión**: la API deduce la VLAN de gestión del estado almacenado, como la de la interfaz virtual cuya dirección coincide con la que usa la plataforma para conectarse al equipo o, si ninguna coincide, la de la subred más específica que la contiene. El aviso de la sección 5.6.1 se extendió a los cambios sobre puertos, VLAN, interfaces virtuales, rutas y ACL que pueden cortar ese camino.

El guardado de la configuración se implementó como un recurso gestionable más, que obtiene así el bloqueo, los reintentos, el registro del trabajo y la auditoría. En Cisco se usa '*write'*, y en Huawei '*save'*, que pide confirmar con '*y'*. Se da por exitoso sólo si el equipo lo confirma en su respuesta, porque el código de salida no alcanza. Se descartó '*copy running-config startup-config',* que en un Catalyst pide de forma interactiva el nombre del destino y terminaba con código 0 sin haber guardado nada. El guardado nunca es automático, así que, mientras un cambio no se guarda, reiniciar el equipo lo deshace, lo que sirve como última salida si un cambio lo deja inaccesible. Además, el guardado fija también los cambios que alguien haya hecho por consola y no guardó.

Funcionalidad "Instalación y Protección de Credenciales"

La instalación se automatizó con un script para Ubuntu que instala Docker, Node.js y Caddy, genera los secretos y deja la plataforma desplegada. Los secretos son la clave con la que se cifran las credenciales de los equipos, la firma de los *tokens* y las contraseñas del administrador inicial y de la base de datos. La API, los workers, el planificador de Celery, la base de datos y el servicio en memoria corren en contenedores, mientras que Caddy y el servidor que entrega el cliente web lo hacen como servicios del sistema. Caddy atiende las conexiones HTTPS con un certificado de la organización o, si no hay uno, con uno autofirmado que genera él mismo, y puede quedar expuesto directamente o detrás de otro proxy que ya use el puerto 443. El script puede volver a ejecutarse tras cada actualización y conserva los secretos y la configuración local, como un certificado propio.

Las credenciales de los equipos se guardan cifradas y sólo se descifran por un instante en dos casos que exigen escribirlas en un archivo, el inventario de cada ejecución de Ansible y la clave privada de la vía SSH directa. Ambos se escriben en sistemas de archivos en memoria montados en los contenedores de la API y de los workers, de modo que nunca llegan al disco y desaparecen si el contenedor se detiene antes de borrarlos.

Los contenedores de la API, los workers y el planificador de Celery corren con un usuario sin privilegios ni capacidades del sistema. El código queda en modo de sólo lectura, de modo que quien tomara control de un proceso no podría modificarlo. El servicio en memoria exige una contraseña, porque desde él podrían encolarse trabajos sin pasar por la API, los permisos ni la auditoría.

8.3. Pruebas y Validaciones

La iteración incorporó 716 pruebas, en 59 archivos nuevos y 20 ampliados (Cuadro 8.3). La suite completa alcanzó 1430 pruebas en 117 archivos, ya que 95 pruebas de la primera iteración se retiraron o reescribieron al reemplazarse los módulos que verificaban. Las fases de la migración multisitio se probaron sobre una base separada, comprobando que cada usuario con sitios asignados ve exactamente los mismos equipos que antes, que la migración de datos puede repetirse sin cambiar el resultado, que se detiene si deja un equipo sin grupo y que las fases que agregan o eliminan estructuras pueden deshacerse. Las fallas detectadas en equipos reales se fijaron con pruebas construidas a partir de la salida capturada del equipo afectado.

Área	Archivos	Pruebas	Aspectos verificados
Lote, cambios incrementales y rollback por estado	14	328	Lote por equipo y su rollback conjunto, rechazo de entradas en conflicto, cambios incrementales de troncal y de reenvío DHCP en ambos fabricantes, rollback por estado de cada atributo y tipo de recurso, y omisión del rollback sin conexión.
Organización multisitio y aislamiento	30	121	Jerarquía y restricciones, migración, rol efectivo, asignación y delegación de permisos, traslados, alcance de la auditoría y de las operaciones de grupo, y políticas a nivel de fila.
Comportamiento de equipos reales	20	110	Rechazos no detectados, lecturas vacías o truncadas, formatos de salida, nombres de interfaz, interfaces agregadas, algoritmos SSH y búsqueda en tablas MAC extensas.
Coordinación y estado almacenado	9	82	Renovación y tope de espera del bloqueo, cancelación atómica, recuperación de trabajos huérfanos, semáforo de la vía directa, unión de sincronizaciones, lectura bajo demanda y escritura parcial del estado tras un cambio.
Guardado y aviso de gestión	3	43	Variantes de guardado por fabricante con confirmación textual y deducción de la VLAN de gestión.
Sesiones y reautenticación	3	32	Inactividad, duración máxima, listado y revocación de sesiones, y token elevado.
Total	**79**	**716**	—

Cuadro 8.3: Pruebas automatizadas incorporadas en la tercera iteración, por área.

En el laboratorio, cada funcionalidad se probó por las dos vías de transporte, y las pruebas por Ansible sobre los nodos emulados IOSvL2 y CE12800 revelaron cuatro problemas. El CE12800 no tiene el comando ‘*save force’* y toma *‘force’* como nombre de

archivo, por lo que la variante principal de guardado pasó a ser ‘*save’* con la confirmación ‘*y’*. Cuando la sesión se cortaba, Ansible devolvía todos los comandos vacíos sin marcar error, y la sincronización borraba los puertos del estado almacenado, por lo que una lectura enteramente vacía pasó a tratarse como una falla. Tras un cambio, el orquestador guardaba el pedido completo, con los atributos que no tocaba en blanco, de modo que si la sincronización posterior fallaba la vista mostraba el puerto incompleto. Ahora guarda sólo los atributos que fija la operación. Por último, un lote con dos cambios sobre la misma interfaz virtual enviaba dos veces la misma restauración al reintentar el *rollback*, y la segunda fallaba porque el dato ya no existía. Ahora cada paso se envía una sola vez.

En producción, la plataforma gestionó 40 *switches*: Cisco Catalyst 3650 con IOS-XE 16.6 a 16.12, Huawei de la serie S (S5720, S5731 y S6730) con VRP 5.170 y Huawei CloudEngine (CE5855, CE6810, CE6863, CE12808S y CE16808) con VRP 8.191 a 8.220, además de Cisco Catalyst 2960 con IOS 15.0 y un equipo Cisco con IOS-XE 3.6. Cada corrección se verificó contra el equipo afectado antes de desplegarse, y las pruebas de escritura se hicieron en ventanas de mantenimiento, sobre puertos sin uso, con una consola abierta y la configuración previa resguardada.

8.4. Resultados y Ajustes

Todas las funcionalidades de la iteración quedaron operativas en producción. El Cuadro 8.4 resume los requerimientos atendidos, con los que quedaron cubiertos todos los especificados en el Capítulo 4, incluidos los pendientes de las iteraciones anteriores.

La operación sobre el parque real expuso fallas que el laboratorio no había mostrado, concentradas en los equipos de mayor tamaño y en los *firmware* más antiguos (Cuadro 8.5). Casi todas afectan la lectura del estado y no la aplicación de cambios. Se resolvieron en el transporte, en los parsers o en el vocabulario de comandos, las partes que el diseño aísla detrás de contratos, sin modificar el ciclo de un cambio. Las de negociación SSH se deben a que el cliente OpenSSH deshabilita por defecto algunos algoritmos de intercambio de claves, de clave del equipo y de cifrado, y la firma ssh-rsa, que el *firmware* de algunos *switches* todavía exige. Cuando la negociación falla, la vía directa toma de la respuesta del equipo los algoritmos que ofrece y vuelve a intentar con ellos, y desde esta iteración los recuerda para las siguientes sesiones de ese equipo.

Requerimiento		Atendido en esta iteración
Nuevos	RF11: Multisitio	Sitios y grupos, traslado de equipos, roles por sitio o grupo y ocultamiento de lo que está fuera del alcance
	RF12: Lote	Operaciones sobre un grupo o sitio, con un trabajo por equipo y resultado agregado
	RF13: Sincronización del estado	Lectura al abrir una vista, barrido periódico opcional y unión de pedidos
	RF14: Tablero	Totales por organización, sitio, grupo y equipo
Retomados	RF1: Autenticación	Sesiones activas y reautenticación
	RF3: Inventario	Guardado de la configuración como operación explícita, confirmado por el equipo
	RF5: Puertos	Aviso sobre la VLAN de gestión
	RF8: Trabajos	Omisión del rollback si el cambio no llegó al equipo
	RF9: Auditoría	Filtro por sitio y auditoría por alcance
	RF10: Interfaz web	Vistas de los recursos de esta iteración
No funcionales	RNF1: Protección del acceso	Límites de la sesión y reautenticación
	RNF2: Aislamiento entre alcances	Seguridad a nivel de fila en la base y contexto de sistema para los procesos en segundo plano
	RNF3: Tolerancia a fallas	Límite global en la vía SSH directa
	RNF5: Uso de sesiones	Pausa de consultas con la vista oculta

Cuadro 8.4: Requerimientos del sistema atendidos en la tercera iteración.

Categoría	Caso observado	Tratamiento
Lectura incompleta tomada como dato	Lecturas fallidas o cortadas a mitad de sesión se guardaban como estado vacío, y siete equipos grandes perdieron del estado almacenado sus interfaces virtuales.	Una lectura sólo se acepta si la sesión llegó a su comando de cierre y devolvió contenido. Si no, se conserva el último estado conocido.
Salidas extensas	Un switch con una ventana SSH reducida cortaba la conexión cuando la plataforma terminaba de enviar los comandos, antes de devolver unos 66 KB de salida.	Espera de 5 s antes de cerrar el envío, con 9 de 9 lecturas consistentes (277 VLAN, 67 puertos y 39 interfaces virtuales).
Ráfagas de conexiones	La vía SSH directa abría una conexión por comando, unas 24 en 16 s para una sola lectura, que algunos firmware cortaban.	Todos los comandos de una lectura en una única sesión.
Algoritmos SSH antiguos	Cada sesión repetía negociaciones rechazadas, y cinco Huawei rechazaban la clave correcta porque sólo firman con ssh-rsa.	Adaptación a la oferta del equipo, recordada por equipo (de cuatro conexiones por lectura a una), y habilitación explícita de esa firma.
Rechazo no detectado	Con autenticación por contraseña, un Huawei rechazó el nombre abreviado de una interfaz y el trabajo se informó como exitoso sin haberse aplicado.	Detección del rechazo en el texto devuelto por el equipo, en ambas vías de transporte.
Formatos de salida	Columnas adicionales en la tabla MAC y en las VLAN de los troncales, sintaxis alternativa de ACL y nombres de interfaz que comienzan con un dígito.	Parsers ampliados y probados con salidas capturadas, con 6084 de 6084 entradas MAC leídas en el equipo más grande.
Interfaces	Módulos de servicio internos listados como puertos, y agregados de enlaces ausentes.	Filtrado de los primeros y soporte de Eth-Trunk y Port-channel.
Recursos del worker	Los procesos del agente SSH quedaban sin recolectar y agotaban los identificadores de proceso del contenedor.	Agente en primer plano y un proceso init en cada contenedor.

Cuadro 8.5: Fallas detectadas durante la validación en producción, por categoría.

El Cuadro 8.6 compara las sesiones que abre cada operación con las de la segunda iteración. En el laboratorio, tres modificaciones aplicadas en una sola sesión demoraron 4,86 segundos en el nodo IOSvL2, frente a 9,24 segundos como tres ejecuciones separadas, y 6,17 segundos frente a 14,97 segundos en el Huawei S5731, sin contar la lectura previa que cada trabajo separado requería.

Operación	Segunda iteración	Tercera iteración
Modificación de N puertos de un equipo	N trabajos y 2N sesiones	1 trabajo y 2 sesiones
Alta de N VLAN en M equipos	N·M trabajos y 2N·M sesiones	M trabajos y 2M sesiones
Lectura de un equipo por la vía SSH directa	Una conexión por comando	Una sesión por lectura
Conexión a un equipo que exige algoritmos SSH deshabilitados por defecto	Hasta 4 conexiones por sesión, hasta dar con los algoritmos que acepta	1 conexión, con los algoritmos recordados de ese equipo
Rollback de un cambio que no llegó a conectarse con el equipo	Se ejecutaba igual, con sus sesiones de escritura y verificación	0, porque se omite

Cuadro 8.6: Sesiones por operación en la segunda y la tercera iteración.

Además de las correcciones del Cuadro 8.5, se aplicaron siete ajustes:

- **Rol propio para la base de datos:** las políticas de seguridad a nivel de fila no se aplicaban, porque la aplicación se conectaba con el usuario inicial de la imagen de PostgreSQL, que es superusuario y está exento de ellas. La aplicación pasó a usar un rol propio sin esa exención, y el usuario inicial quedó para mantenimiento.
- **Lectura bajo demanda:** el barrido periódico leía todos los equipos cada 20 minutos aunque nadie los consultara, y cada lectura de un Catalyst 2960 elevaba el uso de su procesador del 30 % al 92 %. Se deshabilitó por defecto en favor de la lectura al abrir una vista, ya que todo cambio relee el equipo antes de aplicarse. Sigue disponible como opción, empezando por los equipos más desactualizados y con 3 segundos entre lecturas.
- **Bloqueo con renovación:** un trabajo que superaba los cinco minutos perdía el bloqueo en pleno *rollback*, y otro podía abrir una segunda sesión sobre el mismo equipo. El bloqueo pasó a vencer a los 60 segundos y a renovarse cada 20 segundos mientras el trabajo corre, de modo que la caída de un *worker* libera el equipo en menos de un minuto. La espera para obtenerlo se limitó a 10 minutos.
- **Límite global de sesiones:** el semáforo, que sólo cubría las ejecuciones de Ansible, se extendió a la vía SSH directa, que ahora reserva un lugar antes de abrir cada sesión.

- **Trabajos huérfanos:** los trabajos que la caída de un worker dejaba en ejecución sólo se cerraban al reiniciar la API. Ahora un barrido cada 5 minutos los pasa a fallidos, una vez que su equipo ya no está bloqueado.
- **Conexión persistente de Ansible:** cada ejecución dejaba su sesión SSH abierta hasta que vencía el tiempo de inactividad, y ninguna reutiliza la de otra. La lectura previa, la aplicación y la sincronización de operaciones seguidas agotaban así las cinco líneas VTY del CE12800 del laboratorio. El playbook genérico pasó a cerrar la conexión al terminar cada ejecución, aun cuando una tarea falla.
- **Alcance de las operaciones de grupo:** el resultado agregado de una operación sobre varios equipos se mostraba a cualquier usuario autenticado que conociera su identificador. Ahora sólo incluye los equipos del alcance del usuario, y si no ve ninguno, responde como si la operación no existiera.

Quedaron identificadas, además, tres limitaciones:

- **Algoritmos SSH por proceso:** los algoritmos aprendidos de cada equipo se guardan en la memoria de cada worker, que los vuelve a aprender al reiniciarse.
- **Aviso de gestión:** no cubre la ACL que filtra las sesiones remotas del equipo, porque no se lee a qué líneas de acceso está aplicada. Quitar de ella la dirección de la plataforma puede dejarla sin acceso sin aviso.
- **Operaciones de grupo en el cliente:** salvo en las VLAN, una operación sobre varios equipos se envía como una operación por equipo, por lo que el historial no las muestra agrupadas y su resultado conjunto sólo se ve al aplicarla.

En síntesis, la tercera iteración completó el alcance funcional previsto: la organización en sitios y grupos con permisos por alcance y políticas a nivel de fila, las operaciones en lote, la sincronización al abrir cada vista, el tablero y un cliente web organizado por alcance. La validación sobre 40 *switches* en producción expuso fallas propias de los equipos más grandes y de los *firmware* más antiguos. Casi todas se resolvieron en el transporte, los parsers o el vocabulario de comandos, sin modificar el ciclo de un cambio, lo que confirma la separación entre la lógica de los cambios y la heterogeneidad de los equipos que plantea el diseño. Operar sobre equipos en servicio motivó además los cambios incrementales, el *rollback* por estado y el aviso sobre el acceso de gestión, que reducen el riesgo de dejar un equipo inaccesible.

9. Conclusiones

El objetivo general del proyecto fue desarrollar una plataforma centralizada para administrar *switches* de distintos fabricantes desde una interfaz única, que abstrajera la sintaxis de cada uno y garantizara operaciones idempotentes, trazables y reversibles. La plataforma resultante administra *switches* Cisco IOS e IOS-XE y Huawei VRP por SSH, sin instalar agentes en los equipos, y al cierre de la tercera iteración gestionaba 40 *switches* en producción. Con esa iteración quedaron atendidos todos los requerimientos funcionales y no funcionales especificados. La plataforma permite consultar y modificar las VLAN, los puertos, las interfaces virtuales y la configuración global sin recurrir a la CLI, con cambios que se reintentan, se revierten ante fallas y quedan auditados, sobre una organización por sitios y grupos con permisos por alcance.

El análisis de las herramientas y de los modelos de solución existentes mostró que ninguno combinaba la abstracción multivendor con la viabilidad sobre el parque existente y la confiabilidad de la operación, lo que fundamentó una plataforma propia organizada en torno al recurso gestionable. Cada recurso incorporado después de la VLAN recibió el bloqueo, los reintentos y la auditoría sin modificar la lógica central. Las diferencias entre fabricantes quedaron aisladas en un controlador y un vocabulario de comandos por fabricante. La validación mostró que la variabilidad también se da entre familias de un mismo fabricante, con 76 alternativas en el vocabulario de Huawei frente a una en Cisco. Todas se incorporaron sin modificar los controladores.

La validación incremental, primero en un laboratorio emulado, luego sobre equipos físicos y por último sobre 40 *switches* en producción, resultó decisiva. La operación en producción expuso fallas que el laboratorio no había mostrado. Casi todas se resolvieron en el transporte, los parsers o el vocabulario, sin tocar el ciclo de un cambio. Esa experiencia motivó además criterios de cambios seguros, como los cambios incrementales sobre los troncales y el *rollback* por estado. En rendimiento, las consultas pasaron a resolverse en milisegundos sin abrir sesiones con los equipos. Una operación sobre dos equipos bajó de entre 13 y 23 segundos a unos 6, y editar varios puertos de un equipo pasó a requerir dos sesiones en lugar de dos por puerto.

El desarrollo deja tres lecciones. La primera es que el equipo de red es un recurso limitado. Una lectura completa llegaba a elevar el procesador de un Catalyst 2960 del 30 % al 92 %, por lo que las consultas se resuelven desde el estado almacenado y los equipos se leen sólo cuando hace falta. La segunda es que la heterogeneidad no se agota en el fabricante. Aparece entre familias, versiones de *firmware* y hasta en la forma de autenticarse, como ocurrió con la autenticación por clave, que en algunos equipos las bibliotecas SSH de Python que usa Ansible no lograron completar y que llevó a una vía directa con OpenSSH. Aislarla detrás de contratos permitió absorberla sin rediseñar el sistema. La tercera es que los equipos emulados no alcanzan para validar una herramienta de red, porque las funciones ausentes en las imágenes, las diferencias entre familias y las fallas de los equipos más grandes sólo aparecieron sobre equipos físicos y en producción.

El proyecto integró conocimientos de distintas áreas de la carrera. La ingeniería de software guió la planificación iterativa, la especificación de requerimientos y un diseño modular basado en patrones. Las redes de computadoras aportaron el conocimiento de los equipos, sus protocolos y su configuración, sin el cual no habría sido posible traducir cada operación a la sintaxis de cada fabricante. La concurrencia, las bases de datos y la seguridad informática sostuvieron la ejecución en paralelo, el aislamiento entre alcances y la protección de las credenciales. A lo largo del desarrollo se mantuvieron prácticas como las pruebas automatizadas, el modo simulado y el control de versiones, que permitieron validar cada cambio antes de llevarlo a los equipos en servicio.

La plataforma se construyó íntegramente con herramientas de código abierto y opera sobre los equipos existentes, sin licencias ni reemplazo de hardware, lo que la vuelve aplicable a otras organizaciones con redes de campus heterogéneas. Para quienes administran la red, reemplaza el recorrido equipo por equipo por operaciones desde una interfaz única, con cambios verificados, reversibles y registrados. El resultado es una plataforma en uso sobre la red de la organización, que cumple el propósito que dio origen al trabajo, administrar equipos de distintos fabricantes como si fueran uno solo, y cuyo diseño permite seguir extendiéndola.

10. Trabajos Futuros

Las líneas de trabajo futuro siguen cuatro direcciones: completar lo construido a partir de las limitaciones detectadas en la validación, asumir tareas de la operación diaria que hoy se resuelven a mano, ampliar el alcance que el proyecto dejó fuera de forma deliberada e integrar la plataforma con los servicios de la organización.

En primer lugar, sería posible completar lo ya construido. La configuración global podría incorporar el lote que ya tienen las VLAN, los puertos y las interfaces virtuales. Estos recursos podrían, a su vez, admitir solicitudes sobre varios equipos con un resultado agregado y un historial agrupado, como ocurre con las VLAN. El reintento manual del *rollback*, hoy disponible sólo para puertos e interfaces virtuales, podría extenderse a las VLAN y a la configuración global. Los algoritmos SSH aprendidos de cada equipo podrían conservarse en un almacenamiento compartido, para no perderse al reiniciarse cada worker. Por último, el aviso sobre el acceso de gestión podría cubrir también la ACL que filtra las sesiones remotas del equipo.

En segundo lugar, la plataforma podría asumir tareas que hoy se resuelven a mano. Una ACL podría definirse una única vez y asignarse a varios equipos, de modo que modificarla genere un trabajo por equipo, con su lectura previa y su *rollback*. Dos puertos podrían intercambiar su configuración con una sola acción, por ejemplo cuando un usuario cambia de lugar o un puerto falla. Como ya se conoce cómo están interconectados los *switches*, la plataforma podría además mostrar esas conexiones y el camino entre dos equipos, de modo que se vea de un vistazo por qué troncales debe permitirse una VLAN para llevarla de un *switch* a otro. La plataforma podría también advertir cuando un equipo fue modificado por consola, al notar diferencias entre una lectura y la anterior, y registrar ese cambio en la auditoría. Los trabajos fallidos, por su parte, podrían notificarse por otros canales mediante un nuevo suscriptor de eventos, sin modificar a los emisores.

En tercer lugar, el alcance podría ampliarse. La plataforma podría incorporar otros fabricantes presentes en redes de campus, cada uno con su controlador y su vocabulario, del mismo modo en que hoy conviven Cisco y Huawei. El mismo contrato del recurso gestionable permite sumar funciones que hoy quedan fuera del alcance, como completar la gestión de la agregación de enlaces (LACP), el árbol de expansión (STP) o la duplicación de tráfico (port mirroring).

Una ampliación con impacto inmediato sería el aprovisionamiento de los equipos nuevos. Hoy, un equipo recién encendido obtiene por DHCP la ubicación de su configuración inicial y la descarga de un servidor de archivos mediante el proceso de ZTP. Ese archivo se genera con un script independiente de la plataforma. Integrado a ella, se generaría a partir de plantillas con los valores predeterminados de cada sitio, como la dirección de gestión, el usuario local y la clave pública de la plataforma. El equipo quedaría registrado en el inventario apenas responda por SSH, y el aprovisionamiento, en la auditoría. Así, el día 0 dejaría de ser un supuesto y pasaría a formar parte del ciclo de vida que gestiona la plataforma.

Por último, la autenticación podría delegarse en Keycloak, un proveedor de identidad de código abierto, conectado mediante SAML al servicio de autenticación central (CAS) que ya utiliza la organización. Los usuarios ingresarían con su cuenta institucional y un inicio de sesión único compartido con los demás servicios. La plataforma recibiría su identidad desde Keycloak, mientras que los permisos por alcance seguirían definiéndose en ella.

Bibliografía

[1] Cisco Systems, Inc., "Enterprise Campus 3.0 Architecture: Overview and Framework," San Jose, CA, USA, Apr. 2008. Accessed: Oct. 12, 2026. [Online]. Available: https://www.cisco.com/c/en/us/td/docs/solutions/Enterprise/Campus/campover.pdfcisco

[2] IEEE Standard for Local and Metropolitan Area Networks—Bridges and Bridged Networks, IEEE Std 802.1Q-2022, 2022. Accessed: Oct. 12, 2026. [Online]. Available: https://standards.ieee.org/ieee/802.1Q/10323/

[3] Cisco Systems, Inc., "Cisco IOS Configuration Fundamentals Configuration Guide," Cisco IOS Release 15S. Accessed: Oct. 12, 2026. [Online]. Available: https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/fundamentals/configuration/15-s/fundamentals-15-s-book.htmlcsrc.nist

[4] Huawei Technologies Co., Ltd., "Versatile Routing Platform (VRP) Documentation." Accessed: Oct. 12, 2026. [Online]. Available: https://support.huawei.com/enterprise/en/software/vrpnvlpubs.nist

[5] Cisco Systems, Inc., "Managing Configuration Files," in *System Management Configuration Guide, Cisco IOS XE Everest 16.6.x (Catalyst 9400 Switches)*. Accessed: Oct. 12, 2026. [Online]. Available: https://www.cisco.com/c/en/us/td/docs/switches/lan/catalyst9400/software/release/16-6/configuration_guide/sys_mgmt/b_166_sys_mgmt_9400_cg/b_166_sys_mgmt_9400_cg_chapter_01000.pdfstandards.ieee

[6] IEEE Standard for Local and Metropolitan Area Networks—Bridges and Bridged Networks, IEEE Std 802.1Q-2022, 2022. Accessed: Oct. 12, 2026. [Online]. Available: https://standards.ieee.org/ieee/802.1Q/10323/csrc.nist

[7] Cisco Systems, Inc., "Configuring Access and Trunk Interfaces," in *Cisco Nexus 5000 Series NX-OS Layer 2 Switching Configuration Guide, Release 5.0(3)N1(1)*, 2011. Accessed: Oct. 12, 2026. [Online]. Available: https://www.cisco.com/c/en/us/td/docs/switches/datacenter/nexus5000/sw/layer2/503_n1_1/Cisco_n5k_layer2_config_gd_rel_503_N1_1_chapter6.htmlcisco

[8] Cisco Systems, Inc., "Configuring IP Routing Between VLANs," in *IP Routing Configuration Guide, Cisco IOS Release 15.2(6)E (Catalyst 2960-X Switch)*. Accessed: Oct. 12, 2026. [Online]. Available: https://www.cisco.com/c/en/us/td/docs/switches/lan/catalyst2960x/software/release/15-2_6_e/configuration_guide/ip/b_1526e_ip_2960x_cg/ip_routing_between_vlans.htmlnist

[9] D. Harrington, R. Presuhn, and B. Wijnen, "An Architecture for Describing Simple Network Management Protocol (SNMP) Management Frameworks," IETF RFC 3411, Dec. 2002. Accessed: Oct. 12, 2026. [Online]. Available: https://www.rfc-editor.org/rfc/rfc3411nvlpubs.nist

[10] J. Postel and J. Reynolds, "Telnet Protocol Specification," IETF RFC 854, May 1983. Accessed: Oct. 12, 2026. [Online]. Available: https://www.rfc-editor.org/rfc/rfc854nvlpubs.nist

[11] T. Ylonen and C. Lonvick, "The Secure Shell (SSH) Protocol Architecture," IETF RFC 4251, Jan. 2006. Accessed: Oct. 12, 2026. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4251shop.standards

[12] R. Enns, M. Bjorklund, J. Schoenwaelder, and A. Bierman, "Network Configuration Protocol (NETCONF)," IETF RFC 6241, Jun. 2011. Accessed: Oct. 12, 2026. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6241

[13] A. Bierman, M. Bjorklund, and K. Watsen, “RESTCONF Protocol,” IETF RFC 8040, Jan. 2017. Accessed: Oct. 12, 2026. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8040

[14] T. Ylonen and C. Lonvick, “The Secure Shell (SSH) Authentication Protocol,” IETF RFC 4252, Jan. 2006. Accessed: Oct. 12, 2026. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4252wikipedia

[15] K. Watsen, I. Farrer, and M. Abrahamsson, “Secure Zero Touch Provisioning (SZTP),” IETF RFC 8572, Apr. 2019. Accessed: Oct. 12, 2026. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8572pages.nist

[16] G. Gooch, *Network Automation with Ansible*. Sebastopol, CA, USA: O’Reilly Media, 2016. Accessed: Oct. 12, 2026. [Online]. Available: https://www.oreilly.com/content/network-automation-with-ansible/pages.nist

[17] E. Haleplidis, K. Pentikousis, S. Denazis, J. Hadi Salim, D. Meyer, and O. Koufopavlou, “Software-Defined Networking (SDN): Layers and Architecture Terminology,” IETF RFC 7426, Jan. 2015. Accessed: Oct. 12, 2026. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7426pages.nist

[18] Paramiko Project, “Paramiko Documentation.” Accessed: Oct. 12, 2026. [Online]. Available: https://docs.paramiko.org/csrc.nist

[19] K. Byers, “Netmiko: Multi-vendor Library to Simplify Legacy CLI Connections to Network Devices.” Accessed: Oct. 12, 2026. [Online]. Available: https://github.com/ktbyers/netmiko

[20] C. Montanari, “Scrapli Documentation.” Accessed: Oct. 12, 2026. [Online]. Available: https://carlmontanari.github.io/scrapli/pages.nist

[21] NAPALM Project, “NAPALM Documentation.” Accessed: Oct. 12, 2026. [Online]. Available: https://napalm.readthedocs.io/csrc.nist

[22] Nornir Project, “Nornir Documentation.” Accessed: Oct. 12, 2026. [Online]. Available: https://nornir.readthedocs.io/iso

[23] Ansible Project, “How Network Automation is Different,” *Ansible Documentation*. Accessed: Oct. 12, 2026. [Online]. Available: https://docs.ansible.com/projects/ansible/latest/network/getting_started/network_differences.htmlstandards.ieee

[24] Ansible Project, “ansible.netcommon.network_cli Connection Plugin,” *Ansible Documentation*. Accessed: Oct. 12, 2026. [Online]. Available: https://docs.ansible.com/projects/ansible/latest/collections/ansible/netcommon/network_cli_connection.htmliso

[25] Cisco Systems, Inc., “Cisco DevNet.” Accessed: Oct. 12, 2026. [Online]. Available: https://developer.cisco.com/

[26] D. Kreutz, F. M. V. Ramos, P. E. Veríssimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, “Software-Defined Networking: A Comprehensive Survey,” *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, Jan. 2015, doi: 10.1109/JPROC.2014.2371999.csrc.nist

[27] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. dissertation, University of California, Irvine, Irvine, CA, USA, 2000. Accessed: Oct. 12, 2026. [Online]. Available: https://ics.uci.edu/~fielding/pubs/dissertation/top.htmstandards.ieee

[28] R. Fielding, M. Nottingham, and J. Reschke, “HTTP Semantics,” IETF RFC 9110, Jun. 2022. Accessed: Oct. 12, 2026. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9110

[29] OpenAPI Initiative, “OpenAPI Specification.” Accessed: Oct. 12, 2026. [Online]. Available: https://spec.openapis.org/oas/latest.htmliso

[30] National Institute of Standards and Technology, “Security and Privacy Controls for Information Systems and Organizations,” NIST Special Publication 800-53 Rev. 5, 2020.

Accessed: Oct. 12, 2026. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/53/r5/finalcsrc.nist

[31] National Institute of Standards and Technology, "Guide to Attribute Based Access Control (ABAC) Definition and Considerations," NIST Special Publication 800-162, includes updates as of Aug. 2, 2019. Accessed: Oct. 12, 2026. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/162/upd2/finalcommittee.iso

[32] ISO/IEC/IEEE 12207:2026, *Systems and software engineering—Software life cycle processes*. Geneva, Switzerland: International Organization for Standardization, 2026. Accessed: Oct. 12, 2026. [Online]. Available: https://www.iso.org/standard/90219.htmliso

[33] S. Brown, "The C4 Model for Visualising Software Architecture." Accessed: Oct. 12, 2026. [Online]. Available: https://c4model.com/standards.ieee

[34] A. Cockburn, "Hexagonal Architecture," 2005. Accessed: Oct. 12, 2026. [Online]. Available: https://alistair.cockburn.us/hexagonal-architecture/

[35] ISO/IEC/IEEE 29148:2018, *Systems and software engineering—Life cycle processes—Requirements engineering*. Geneva, Switzerland: International Organization for Standardization, 2018. Accessed: Oct. 12, 2026. [Online]. Available: https://www.iso.org/standard/72089.htmlstore.sfs

[36] ISO/IEC 25010:2023, *Systems and software engineering—Systems and software Quality Requirements and Evaluation (SQuaRE)—Product quality model*. Geneva, Switzerland: International Organization for Standardization, 2023. Accessed: Oct. 12, 2026. [Online]. Available: https://www.iso.org/standard/78176.htmlstandards.ieee

[37] National Institute of Standards and Technology, "Digital Identity Guidelines: Authentication and Authenticator Management," NIST Special Publication 800-63B-4, Jul. 2025. Accessed: Oct. 12, 2026. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/63/b/4/finalstandards.ieee

[38] OWASP Foundation, "Logging Cheat Sheet," *OWASP Cheat Sheet Series*. Accessed: Oct. 12, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.htmlstandards.ieee

[39] OWASP Foundation, "Session Management Cheat Sheet," *OWASP Cheat Sheet Series*. Accessed: Oct. 12, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.htmlstandards.ieee

[40] ANSI INCITS 359-2012, *Information Technology—Role Based Access Control*. Washington, DC, USA: American National Standards Institute, 2012.

[41] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.standards.ieee

[42] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002.standards.ieee

[43] PostgreSQL Global Development Group, "Row Security Policies," *PostgreSQL 16 Documentation*. Accessed: Oct. 12, 2026. [Online]. Available: https://www.postgresql.org/docs/16/ddl-rowsecurity.htmlieee802

[44] GNS3 Technologies, Inc., "GNS3 Documentation." Accessed: Oct. 12, 2026. [Online]. Available: https://docs.gns3.com/1.ieee802

[45] Ansible Project, "Ansible Runner Documentation." Accessed: Oct. 12, 2026. [Online]. Available: https://ansible.readthedocs.io/projects/runner/en/latest/ieee802

[46] Ansible Project, "ansible-pylibssh: Python Bindings for libssh Client Specific to Ansible Use Case." Accessed: Oct. 12, 2026. [Online]. Available: https://github.com/ansible/pylibssh

[47] Ansible Project, “AWX.” Accessed: Oct. 12, 2026. [Online]. Available: https://github.com/ansible/awx

[48] PagerDuty, Inc., “Rundeck Documentation.” Accessed: Oct. 12, 2026. [Online]. Available: https://docs.rundeck.com/

[49] Semaphore UI, “Semaphore UI Documentation.” Accessed: Oct. 12, 2026. [Online]. Available: https://docs.semaphoreui.com/

[50] Ansiauth, “Ansiauth.” GitHub repository. Accessed: Oct. 18, 2026. [Online]. Available: https://github.com/Marconia40/AnsiAuth-1